

数字逻辑设计

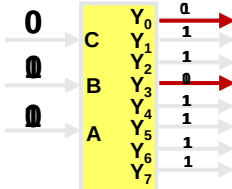
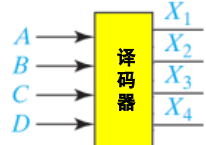
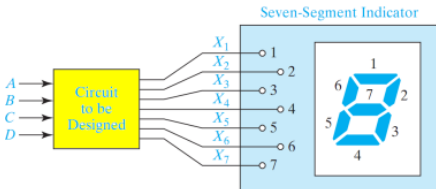
曲龙跃
信息科学与技术学院
qulongyue@hit.edu.cn

常用组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

译码器及分类

- 特点：多输入、多输出的组合逻辑电路
- 功能：将一种编码转换为另一种编码

分类	特点	译码演示
二进制译码器	• 输入：n 位二进制码 • 输出：N 位 ($N=2^n$)，每根输出线都与一个输入最小项唯一对应 (输出线编号值=最小项编号值) • 每个最小项输入，只能使 N 根输出线中的一个输出有效 • N ($N=2^n$) 中取一译码器，也称最小项译码器。	 (3 线-8 线译码器)
代码转换译码器	从一种编码转换为另一种编码 (例如：8421BCD码→余3码)	
显示译码器	将输入的编码信号转换为十进制码或其它特定编码，用来驱动显示器件显示相应的文字符号。	

二进制译码器举例——3线-8线译码器

译码器输出：低电平有效

$$y_i = \overline{m_i}$$

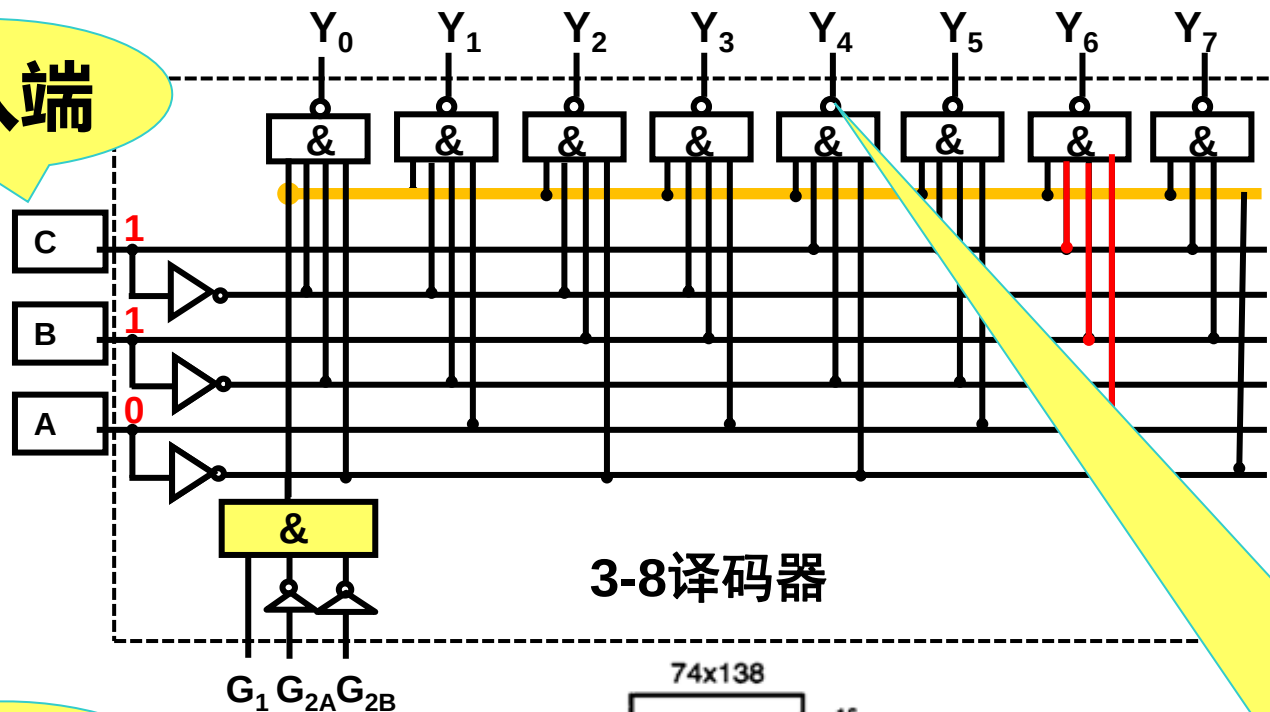
称为最小项译码器

无法实现级联扩展等

输入			译码输出							
C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

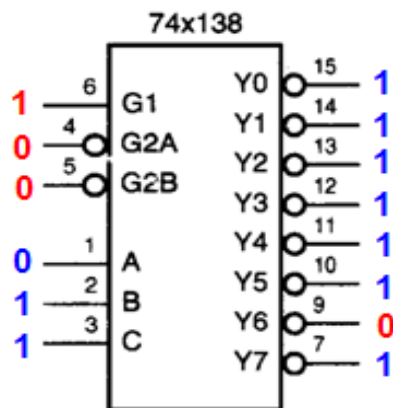
二进制译码器举例——3线-8线译码器

输入端



3-8译码器

附加使能端



典型芯片: 74LS138

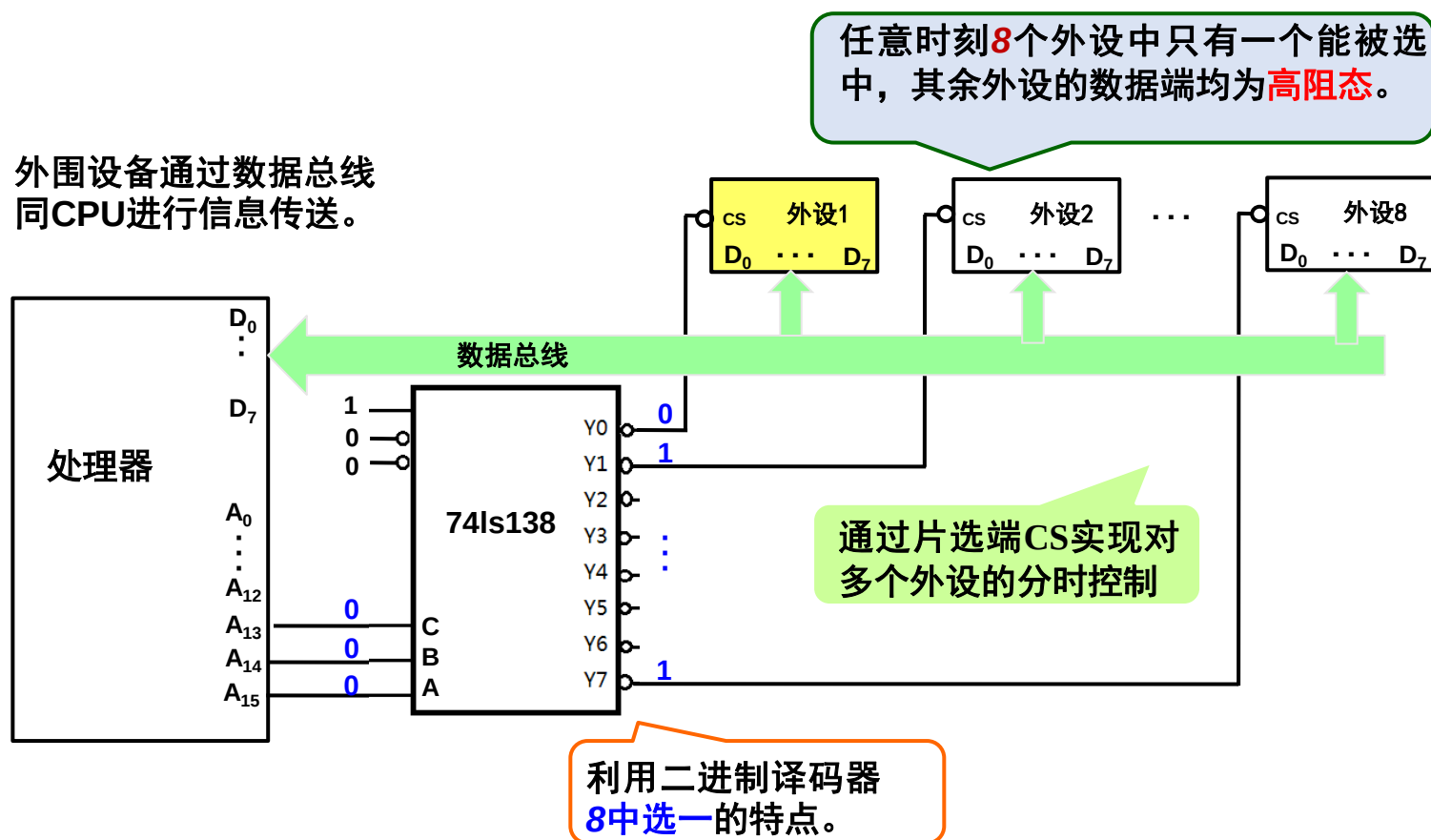
使能端			输入			译码输出							
G ₁	G _{2A}	G _{2B}	C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
0	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	1	0	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	1	1	1	0	1

取反, 输出端
低电平有效

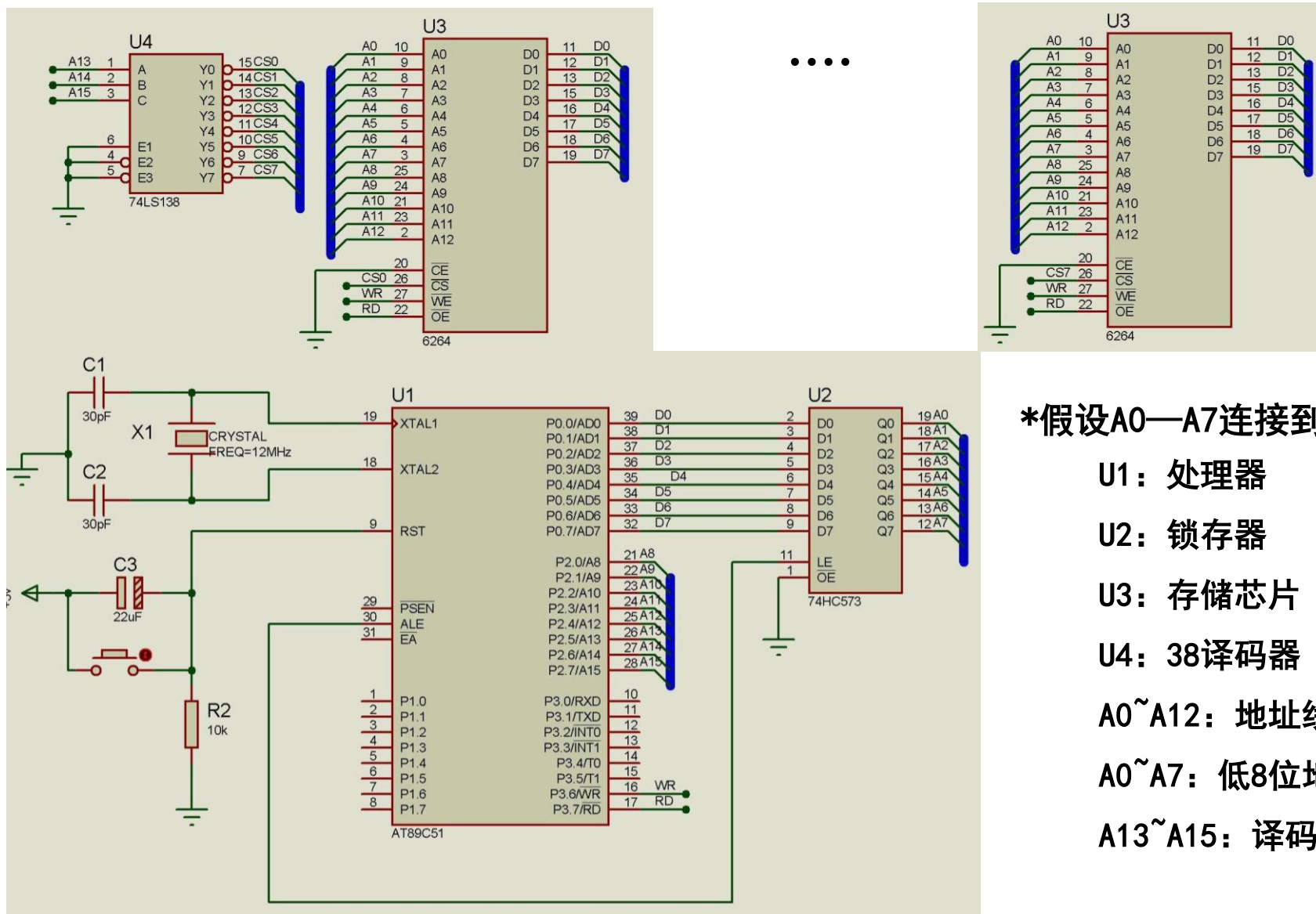
二进制译码器的典型应用——地址译码

■ 微处理器的地址译码

*假设A0—A7连接到各个外设的低8位地址线。



CPU连接8个存储芯片



*假设A0—A7连接到各个外设的低8位地址线。

U1: 处理器

U2: 锁存器

U3: 存储芯片

U4: 38译码器

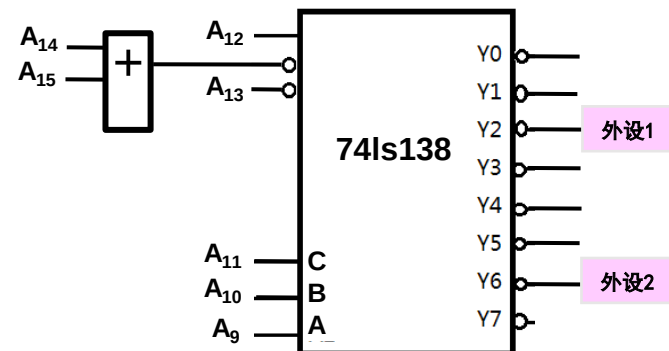
A0~A12: 地址线

A0~A7: 低8位地址线

A13~A15: 译码器输入端

地址译码

- 图示电路的整个地址译码范围？
- 各个外设的地址译码范围？



整个译码器的地址译码范围：

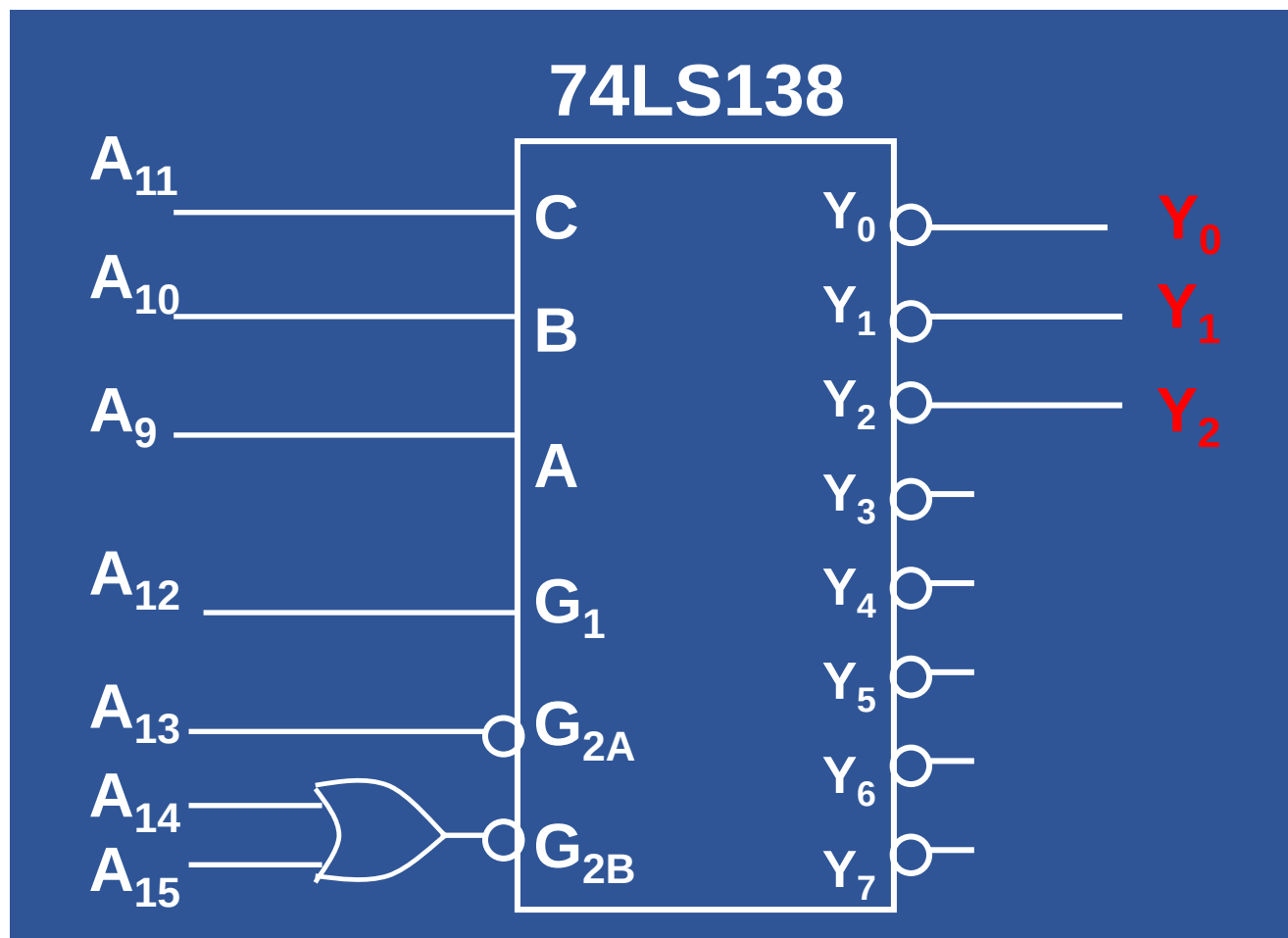
A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
5	4	3	2	1	0											
0	0	0	1	0	0	0	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	最小取值 1000H
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	最大取值 1FFFH

外设1的地址译码范围：

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
5	4	3	2	1	0											
0	0	0	1	0	1	0	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	ⓧ	最小取值 1400H
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	
0	0	0	1	0	1	0	1	1	1	1	1	1	1	1	1	最大取值 15FFH

地址译码

- 图示电路整个地址译码范围？各个外设的地址译码范围？



请给出8个存储芯片的地址范围

第一片存储: 0000H-1FFFH

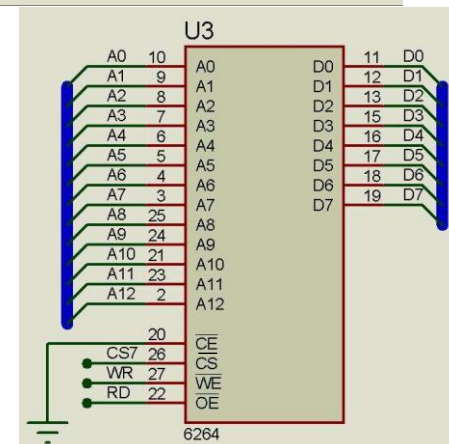
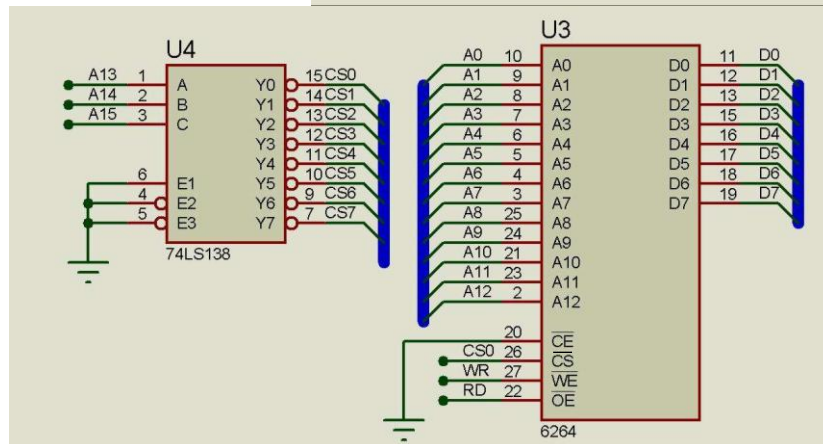
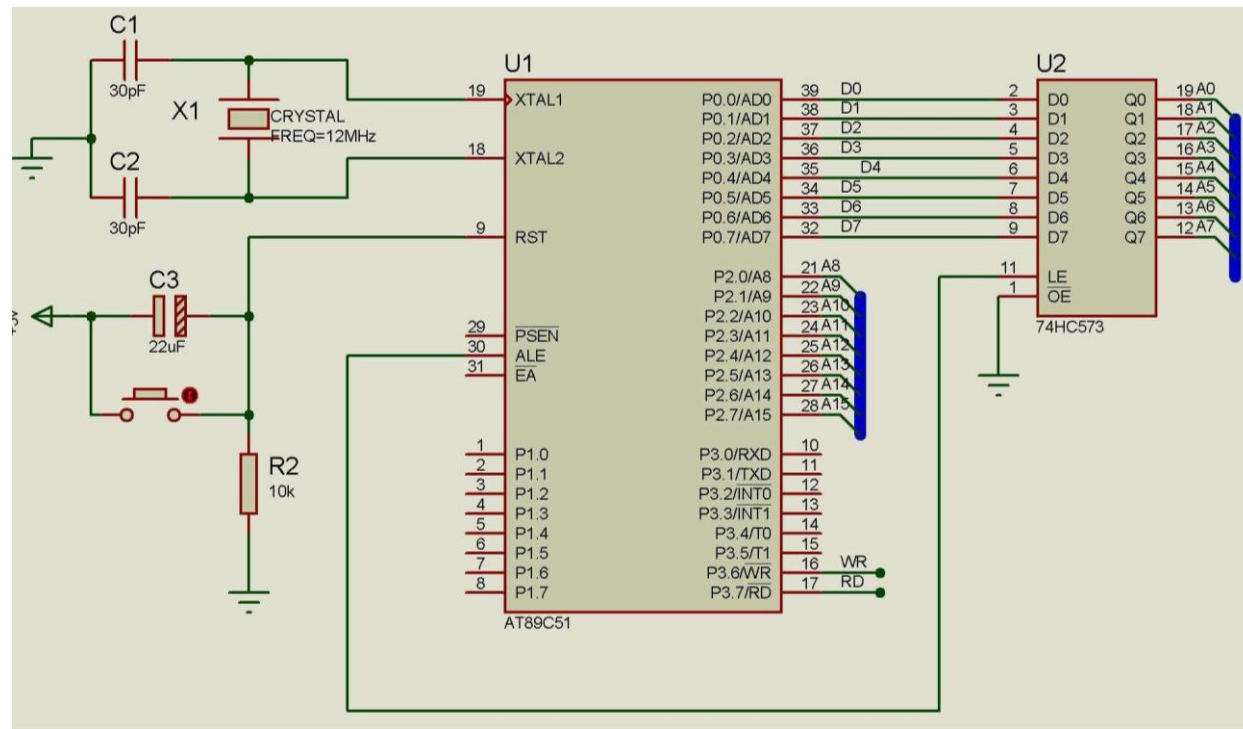
第二片存储: 2000H-3FFFH

.....

第七片存储: C000H-DFFFH

第八片存储: E000H-FFFFH

连接的芯片超过8个怎么办?



二进制译码器的典型应用——译码器级联

• 3线-8线译码器扩展为4线-16线译码

输入				译码输出L								译码输出H							
D	C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y _{0H}	Y _{1H}	Y _{2H}	Y _{3H}	Y _{4H}	Y _{5H}	Y _{6H}	Y _{7H}
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1
1	0	1	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

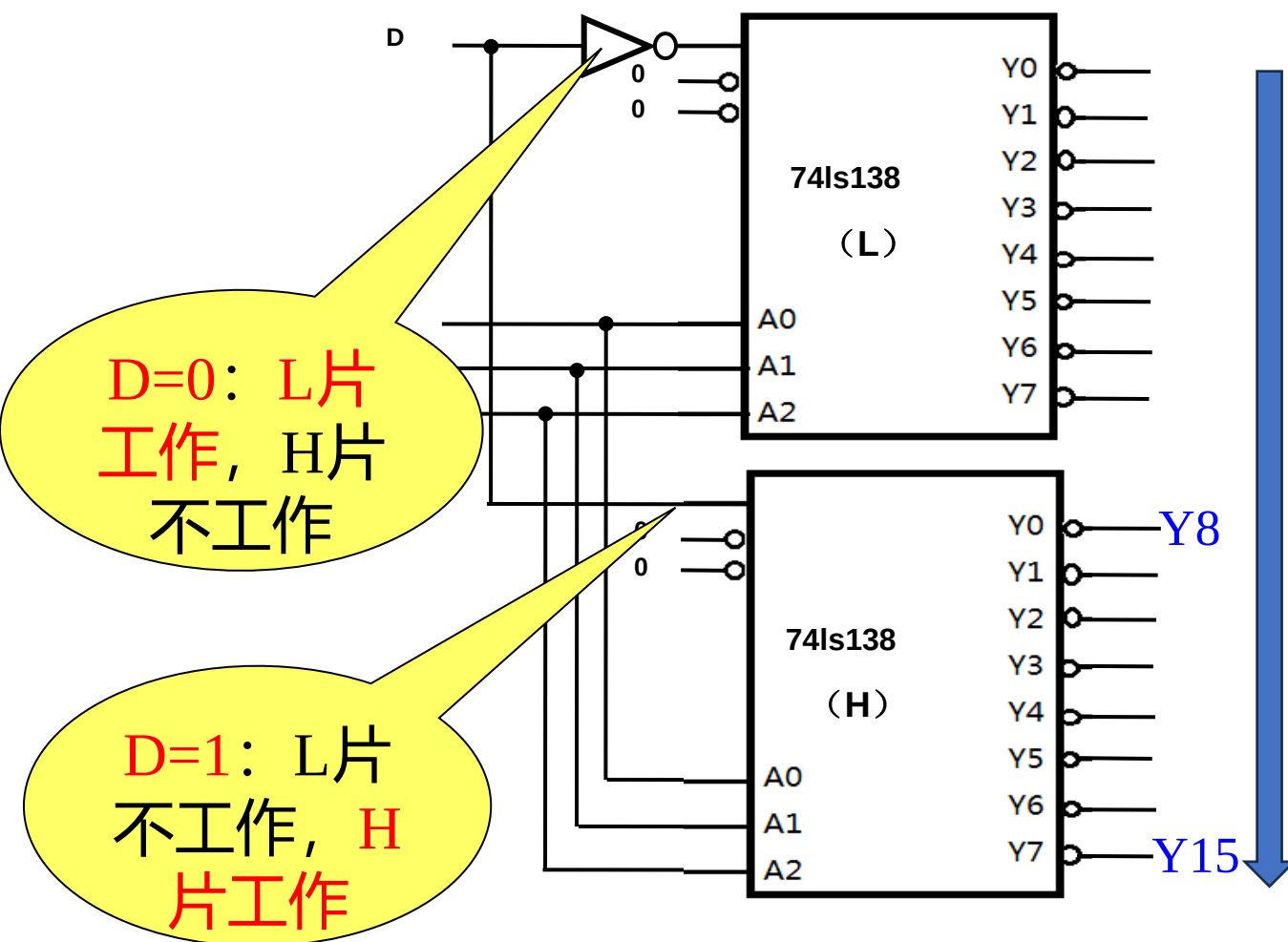
解：要构成4 - 16译码器，需要2片3 - 8译码器（16个输出端），4位输入地址线。如何控制2片3-8译码器，让其中一片工作，另一片不工作？

利用4位输入的最高位D进行控制，如何控制？

D接入2片3-8译码器的附加使能端的进行控制。

二进制译码器的典型应用——译码器级联

• 3线-8线译码器扩展为4线-16线译码



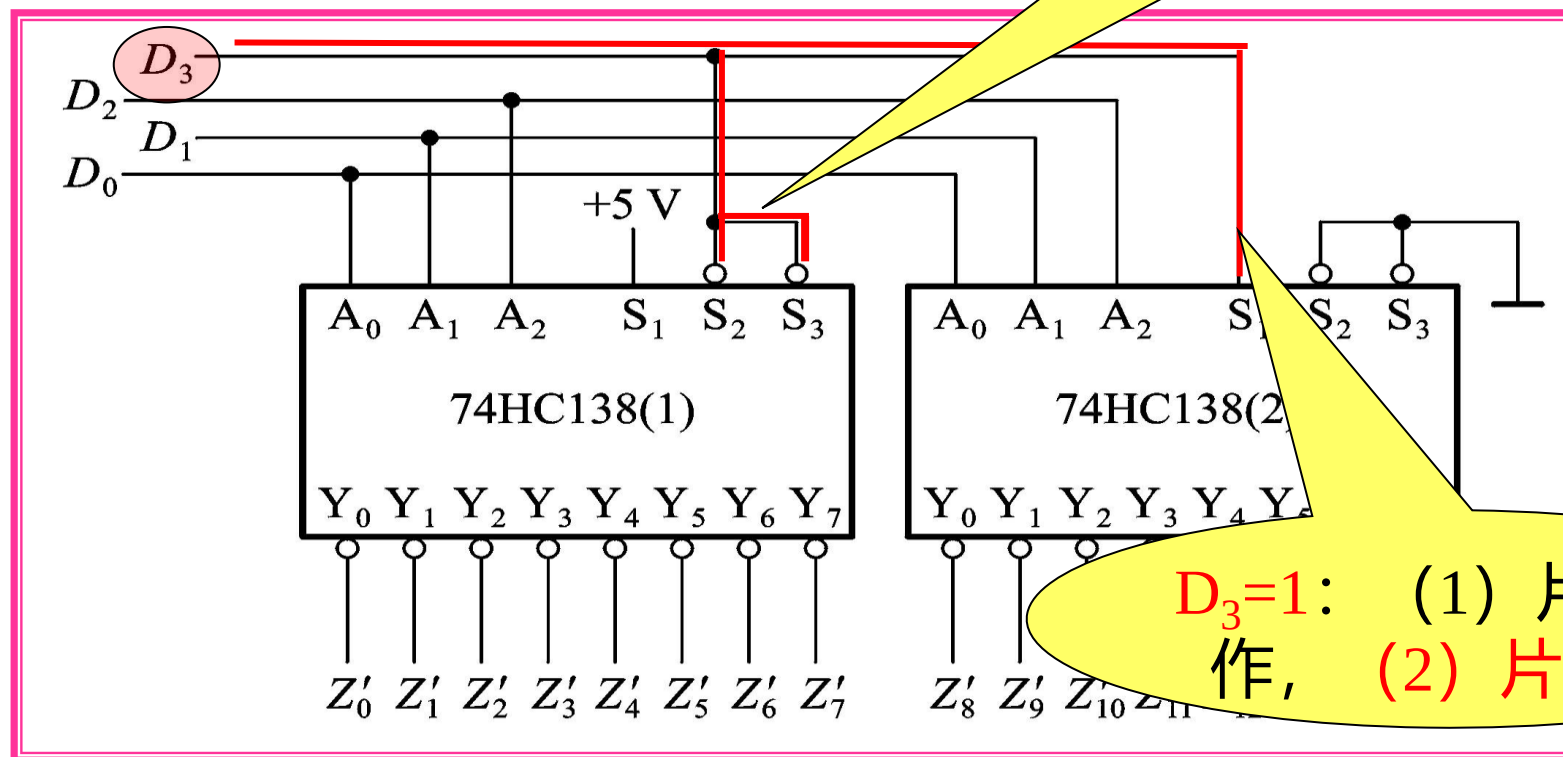
解：要构成4 - 16译码器，需要2片3 - 8译码器（16个输出端），4位输入地址线。
如何控制2片3-8译码器，让其中一片工作，另一片不工作？

利用4位输入的最高位D进行控制，如何控制？

D接入2片3-8译码器的附加控制端的进行控制。

二进制译码器的典型应用——译码器级联

- 不添加任何逻辑门，如何实现？



$D_3=0$: (1) 片工作, (2) 片不工作

$D_3=1$: (1) 片不工作, (2) 片工作

利用74LS138设计多输出的组合电路

1. 基本原理

由于译码器的输出为最小项取反，而逻辑函数可以写成最小项之和的形式，故可以利用附加的门电路和译码器实现逻辑函数。

2. 举例

例 利用3-8译码器设计一个多输出的组合逻辑电路，输出逻辑函数式为：

$$Z_1 = AC' + A'BC + AB'C$$

$$Z_2 = BC + A'B'C$$

$$Z_3 = A'B + AB'C$$

$$Z_4 = A'BC' + B'C' + ABC$$

利用74LS138设计多输出的组合电路

解：先将要输出的逻辑函数化成**最小项之和**的形式，即

$$Z_1 = AC' + A'BC + AB'C = m_3 + m_4 + m_5 + m_6$$

$$Z_2 = BC + A'B'C = m_1 + m_3 + m_7$$

$$Z_3 = A'B + AB'C = m_2 + m_3 + m_5$$

$$Z_4 = A'BC' + B'C' + ABC = m_0 + m_2 + m_4 + m_7$$

将要实现的输出逻辑函数的最小项之和的形式**两次取反**，即

$$Z_1 = ((m_3 + m_4 + m_5 + m_6)')' = (m'_3 \cdot m'_4 \cdot m'_5 \cdot m'_6)'$$

$$Z_2 = ((m_1 + m_3 + m_7)')' = (m'_1 \cdot m'_3 \cdot m'_7)'$$

$$Z_3 = ((m_2 + m_3 + m_5)')' = (m'_2 \cdot m'_3 \cdot m'_5)'$$

$$Z_4 = ((m_0 + m_2 + m_4 + m_7)')' = (m'_0 \cdot m'_2 \cdot m'_4 \cdot m'_7)'$$

利用74LS138设计多输出的组合电路

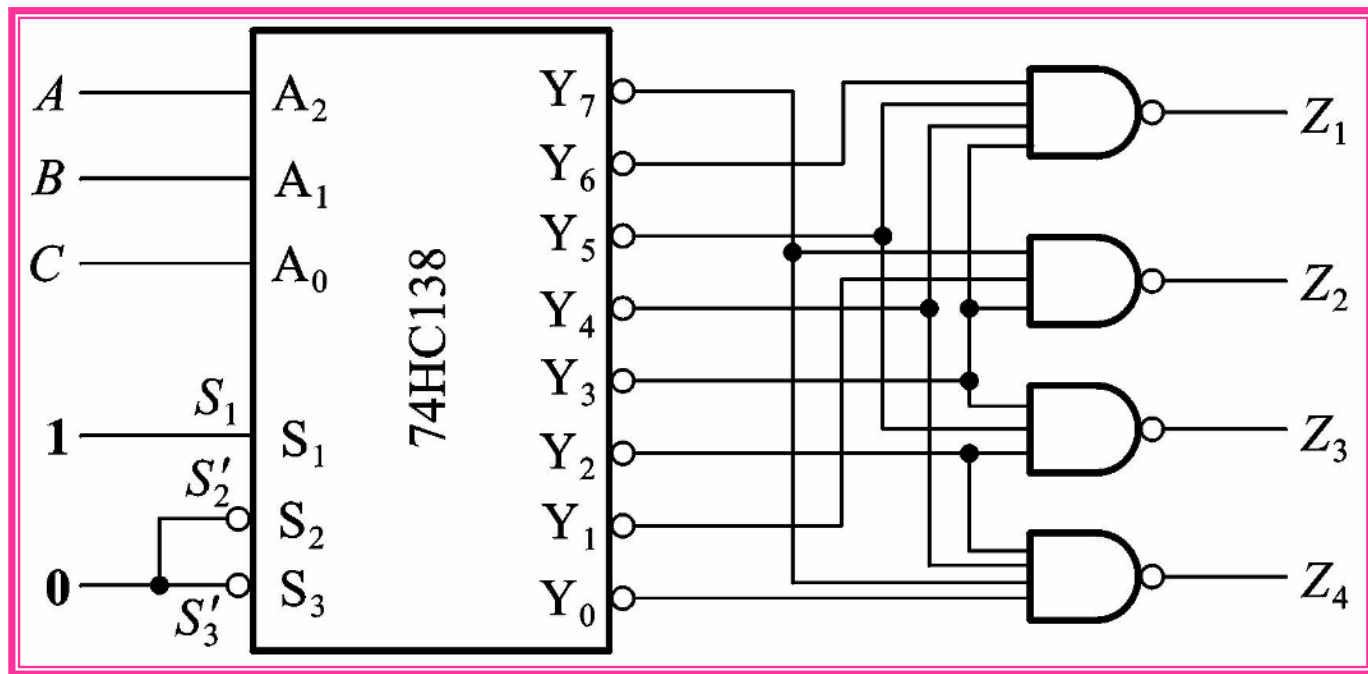
解：将要实现的输出逻辑函数的最小项之和的形式**两次取反**，即

$$Z_1 = ((m_3 + m_4 + m_5 + m_6))' = (m'_3 \cdot m'_4 \cdot m'_5 \cdot m'_6)'$$

$$Z_2 = ((m_1 + m_3 + m_7))' = (m'_1 \cdot m'_3 \cdot m'_7)'$$

$$Z_3 = ((m_2 + m_3 + m_5))' = (m'_2 \cdot m'_3 \cdot m'_5)'$$

$$Z_4 = ((m_0 + m_2 + m_4 + m_7))' = (m'_0 \cdot m'_2 \cdot m'_4 \cdot m'_7)'$$



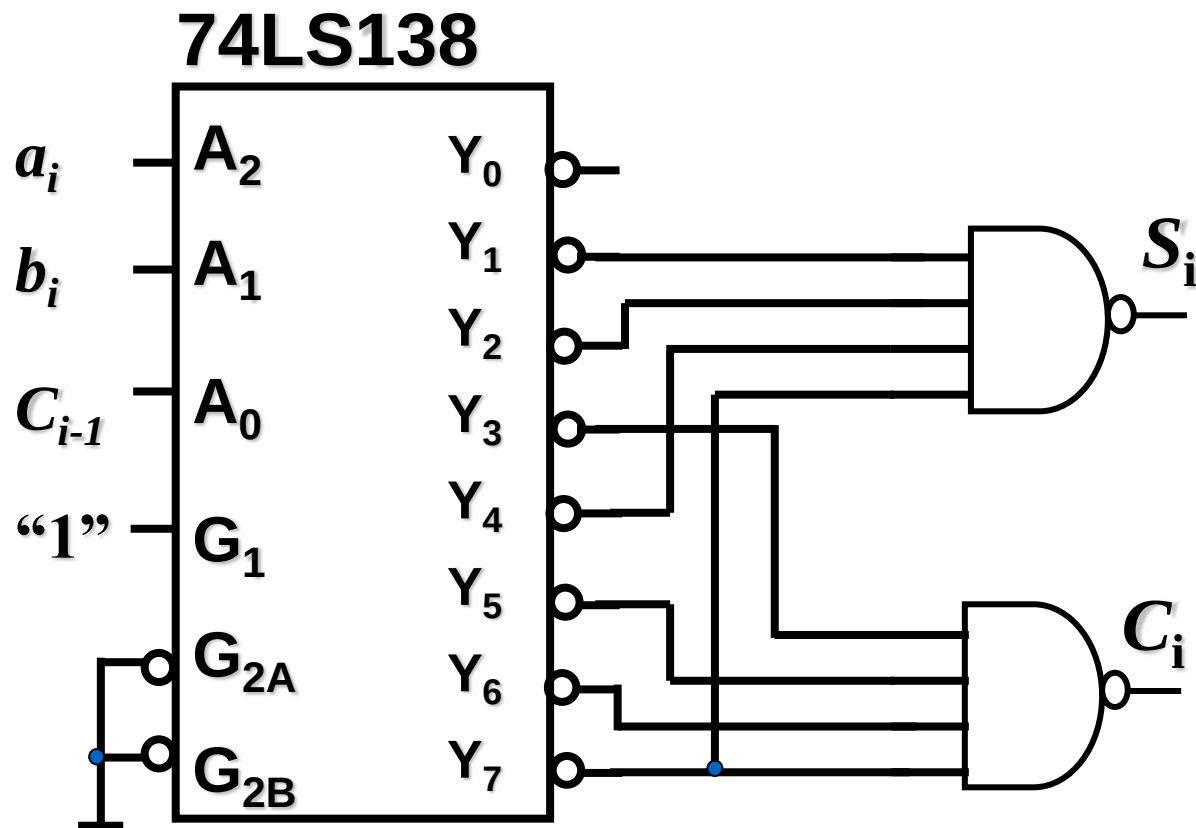
利用74LS138设计1位二进制全加器

全加器真值表

a_i	b_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

74LS138的输出 $y_i = \overline{m_i}$

$$S_i = \sum (1,2,4,7) = \overline{\overline{m_1 m_2 m_4 m_7}}$$



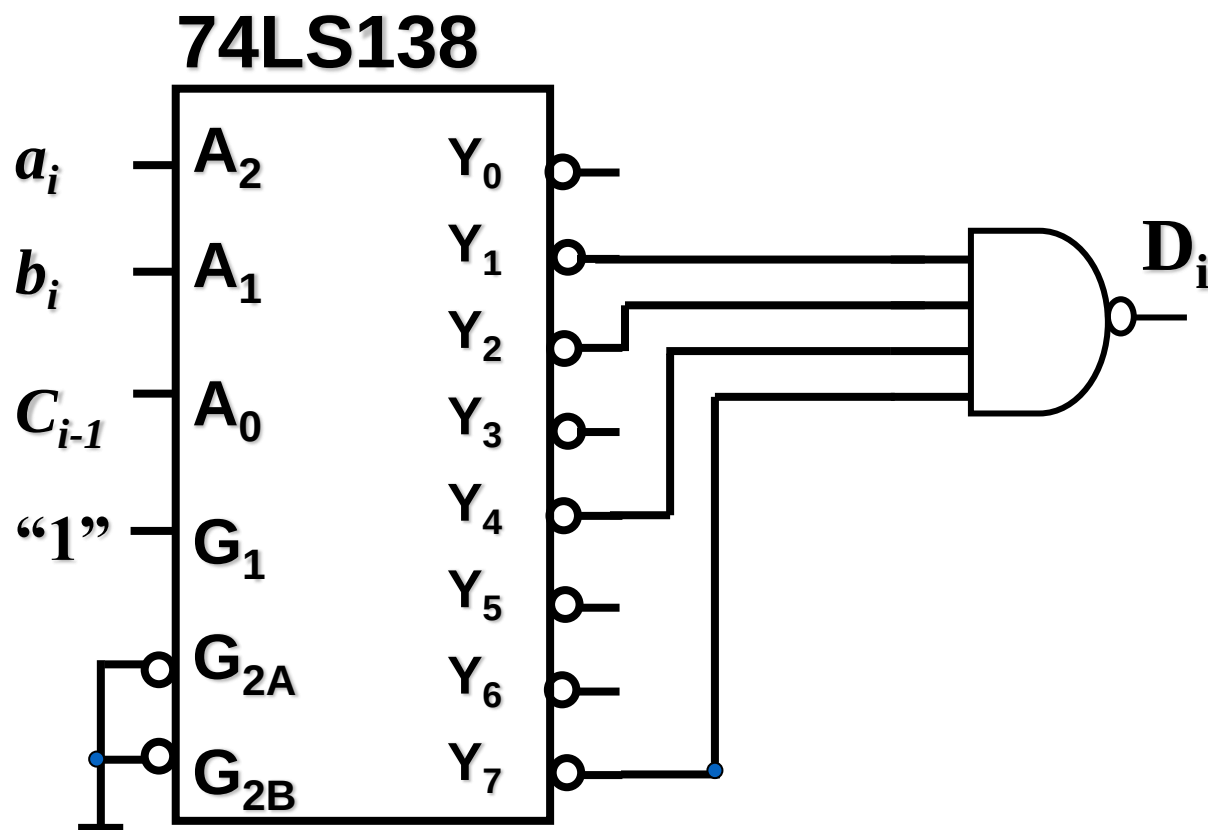
$$C_i = \sum (3,5,6,7) = \overline{\overline{\overline{m_3 m_5 m_6 m_7}}}$$

利用74LS138设计1位二进制全减器

a_i	b_i	C_{i-1}	D_i	C_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

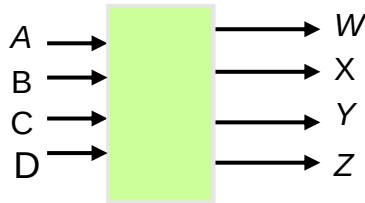
74LS138的输出 $y_i = \overline{m_i}$

$$D_i = \sum (1,2,4,7) = \overline{m_1} \overline{m_2} \overline{m_4} \overline{m_7}$$



编码转换译码器

例：设计一个译码器，
将输入的4位二进制数转换为典型格雷码。



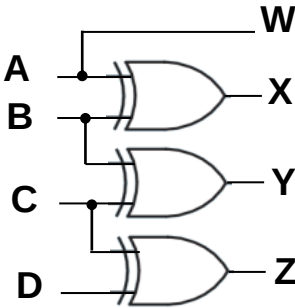
CD \ AB	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

CD \ AB	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

$W = A$
 $X = A \oplus B$
 $Y = B \oplus C$
 $Z = C \oplus D$

CD \ AB	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	1	1	0	0
10	0	0	1	1

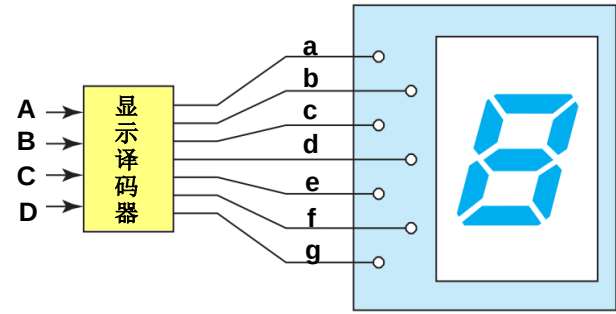
CD \ AB	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1



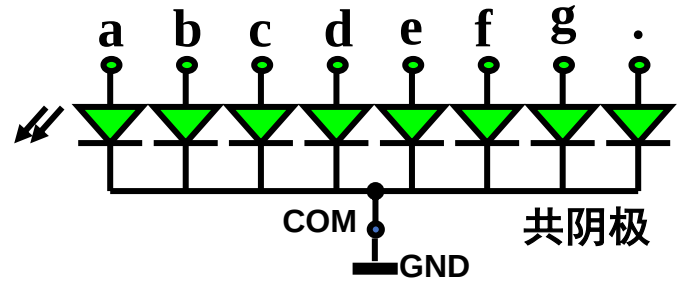
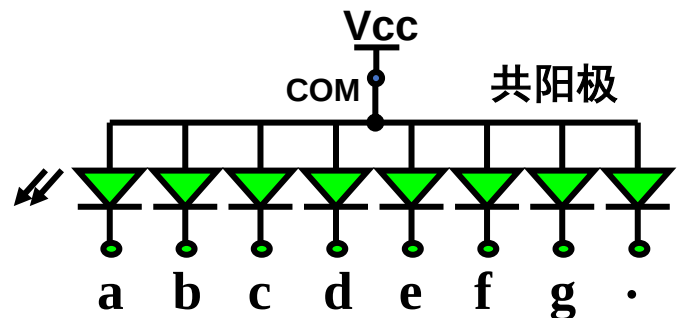
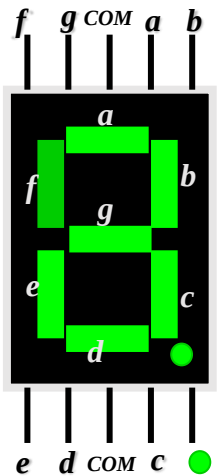
ABCD	WXYZ	ABCD	WXYZ
0000	0000	1000	1100
0001	0001	1001	1101
0010	0011	1010	1111
0011	0010	1011	1110
0100	0110	1100	1010
0101	0111	1101	1011
0110	0101	1110	1001
0111	0100	1111	1000

显示译码器

- 与显示器件（如数码管）配合，将输入代码转换为十进制码或特定编码，并在显示器件上显示相应的字形



七段数码管

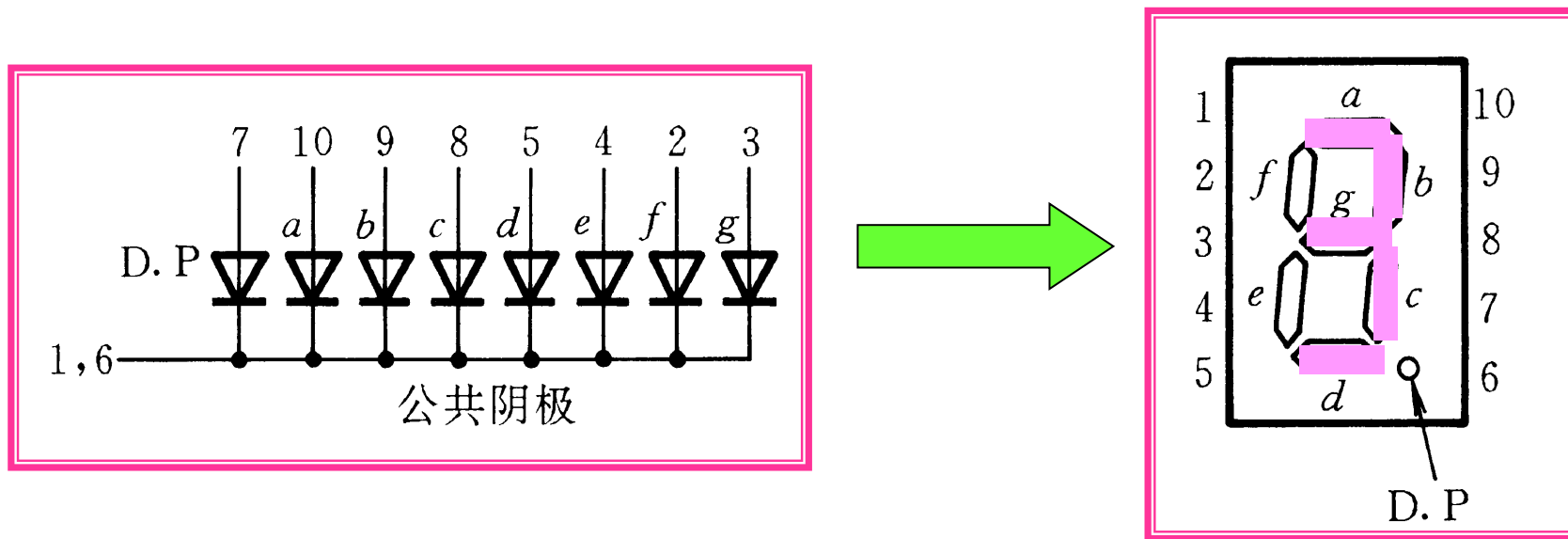


8421BCD码驱动的**共阴极**七段数码管显示译码器功能表

输入				译码输出						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

显示译码器

- 与显示器件（如数码管）配合，将输入代码转换为十进制码或特定编码，并在显示器件上显示相应的字形



当某段加高电平时，则点亮，加低电平时，熄灭。

那么如果显示某一数字如“3”，则 $abcdg = 11111$ ， $fe = 00$ 。

共阴极七段数码管显示译码器逻辑表达式

a

CD \ AB	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

$$a = A + C + BD + \overline{B}\overline{D}$$

b

CD \ AB	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

$$b = \overline{B} + \overline{C}\overline{D} + CD$$

c

CD \ AB	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

$$c = B + \overline{C} + D$$

组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

多路复用器/数据选择器/多路选择器/多路开关

- 从 n 个数据源里选择1个数据，送到输出端 Y
- 多路复用器与二进制译码器相似，都实现了选择功能，而且都会基于选择实现数据传送。
- 多路复用器是一种数据开关，可以看做是一个由二进制译码器控制的单个开关的集合。

用一个译码器和开关实现的多路复用器

- 多路复用器可以看做是一个由二进制译码器控制的单个开关的集合。

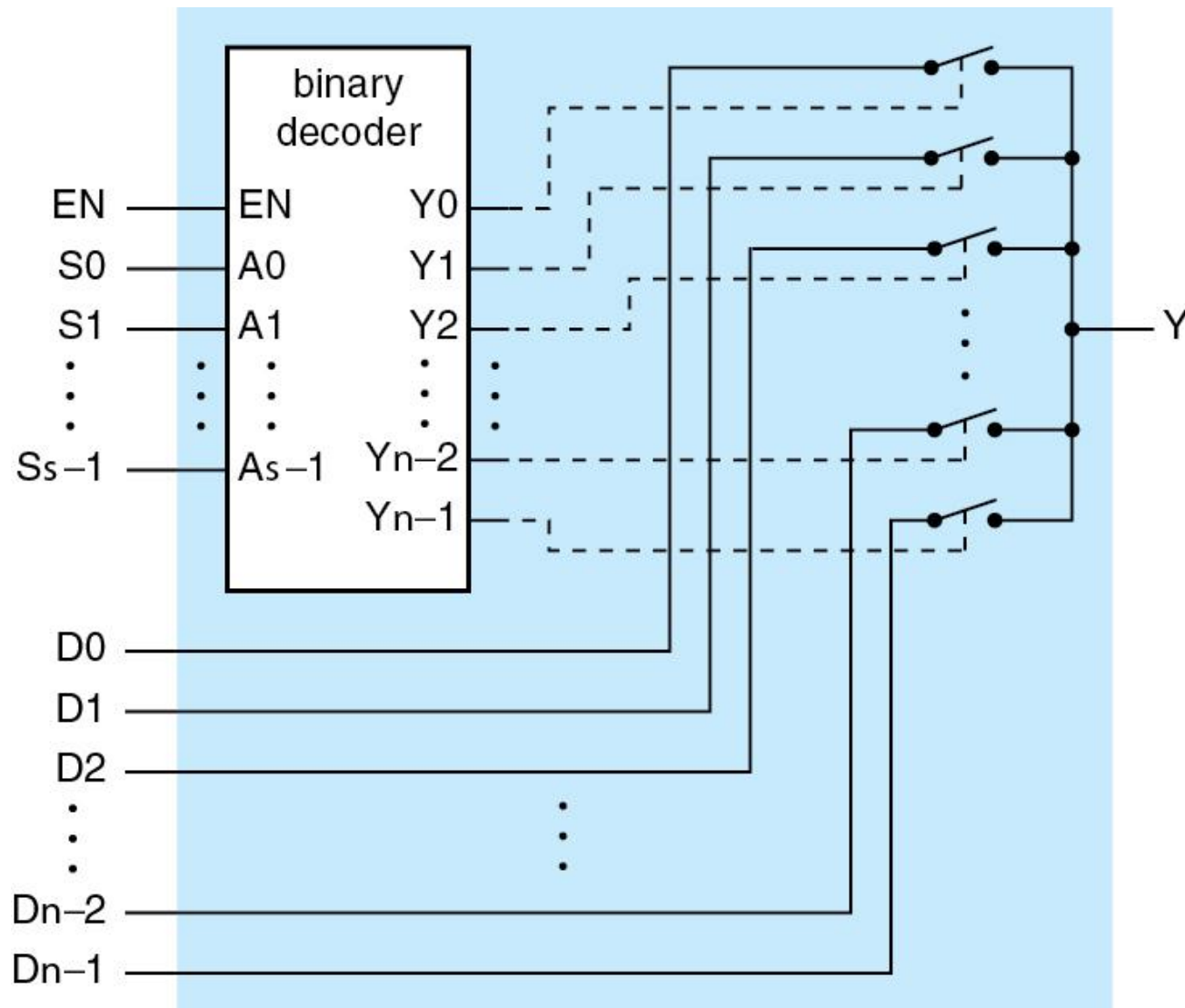
Y_i : 高电平有效

$$Y = \sum_{k=0}^{2^n-1} m_k D_k$$

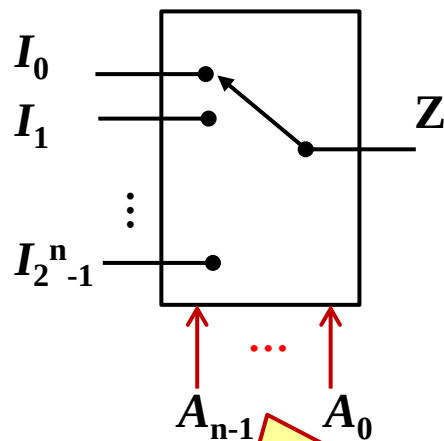
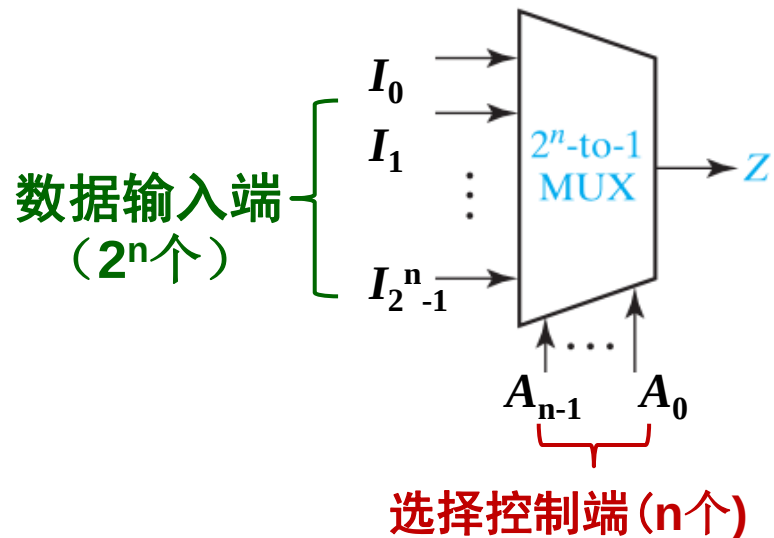
选择端最小项 m_k 的序号 k , 指向了第 k 路数据输入端 D_k 。

m_k —— n 个选择变量的最小项

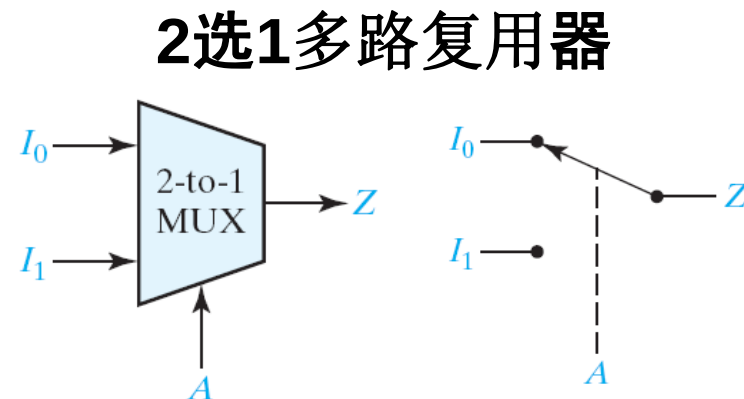
D_k —— 第 k 路数据输入



多路复用器



可构成 2^n 个最小项



$$Z = A' I_0 + A I_1$$

$$Z = \sum_{k=0}^{2^n-1} m_k I_k$$

控制端最小项 m_k 的序号 k , 指向了第 k 路数据输入端 I_k 。

m_k —— n 个控制变量的最小项

I_k —— 第 k 路数据输入

多路复用器的功能:

- 从多路输入中选择1个送往输出端(2^n 选1);
- 选择哪一路输入送到输出端由控制信号决定;

用途: 实现多通道的数据传送。

多路复用器的典型应用——实现逻辑函数

- 多路复用器的标准与或表达式

$$Z = \sum m_k I_k$$

$$Z = A'B'C'I_0 + A'B'CI_1 + A'BC'I_2 + A'BCI_3 + AB'C'I_4 + AB'CI_5 + ABC'I_6 + ABCI_7$$

- 逻辑函数的标准与或式

$$F = \sum m_k$$

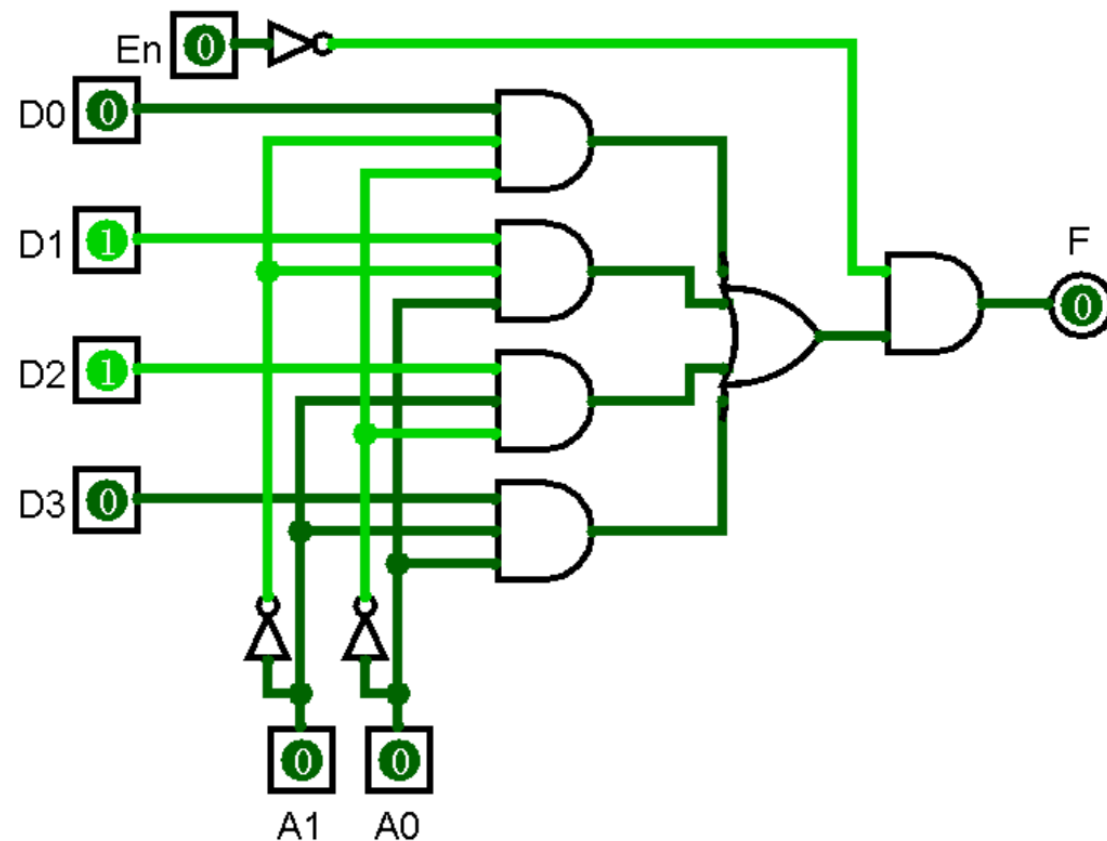
$$= \sum_{k=0}^{2^n-1} m_k F_k$$

第 k 行的最小项与输出 F_k 的积

多路复用器实现的例子

多路复用器功能表

E_n	A_1	A_0	F
1	x	x	0
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3

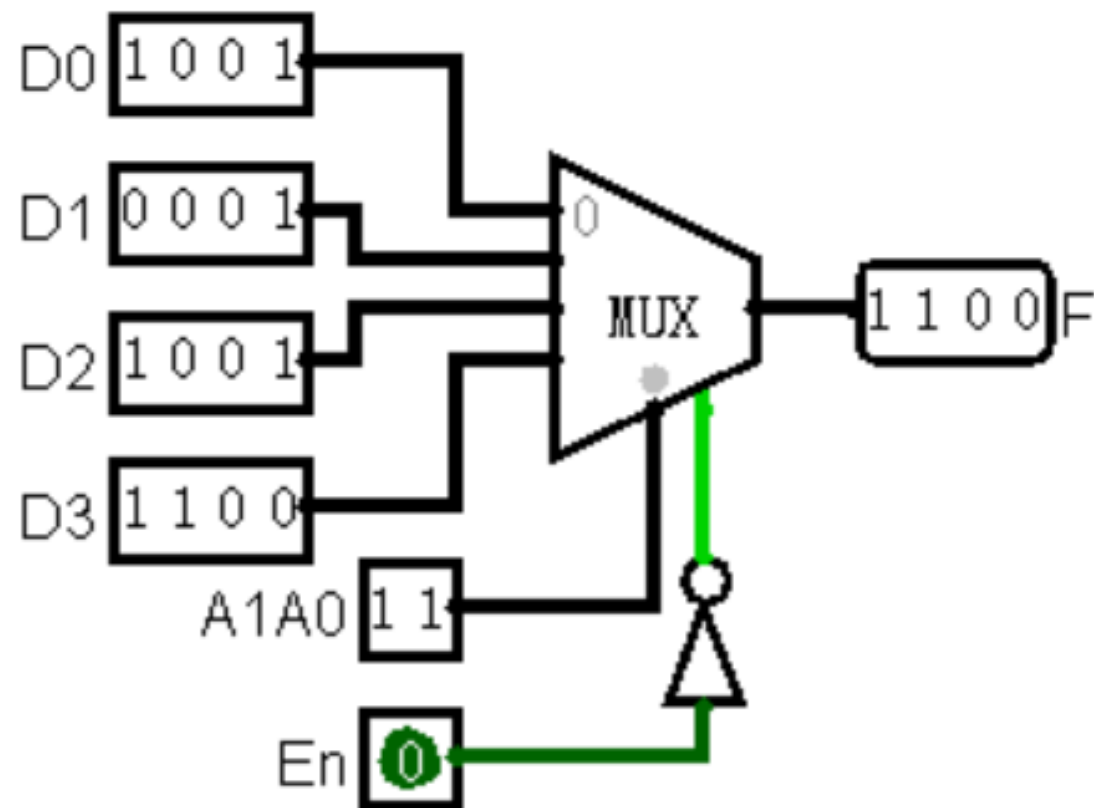


$$F = E_n' (D_0 A_1' A_0' + D_1 A_1' A_0 + D_2 A_1 A_0' + D_3 A_1 A_0)$$

多路复用器应用的例子

多路复用器功能表

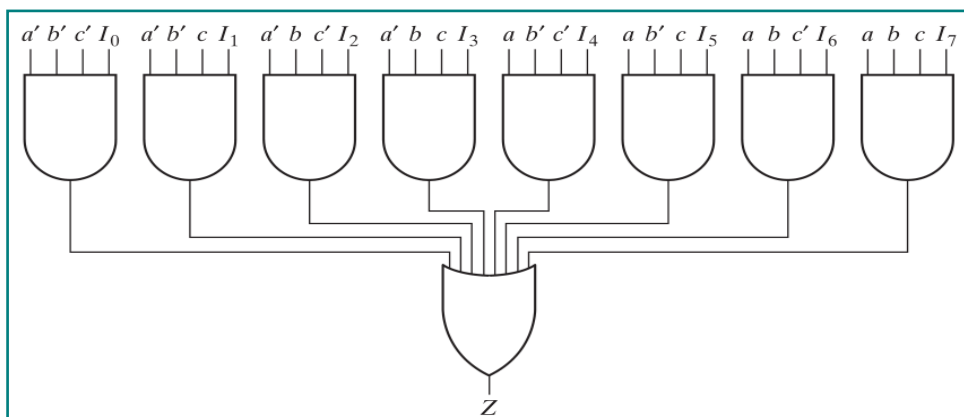
E_n	A_1	A_0	F
1	×	×	0
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3



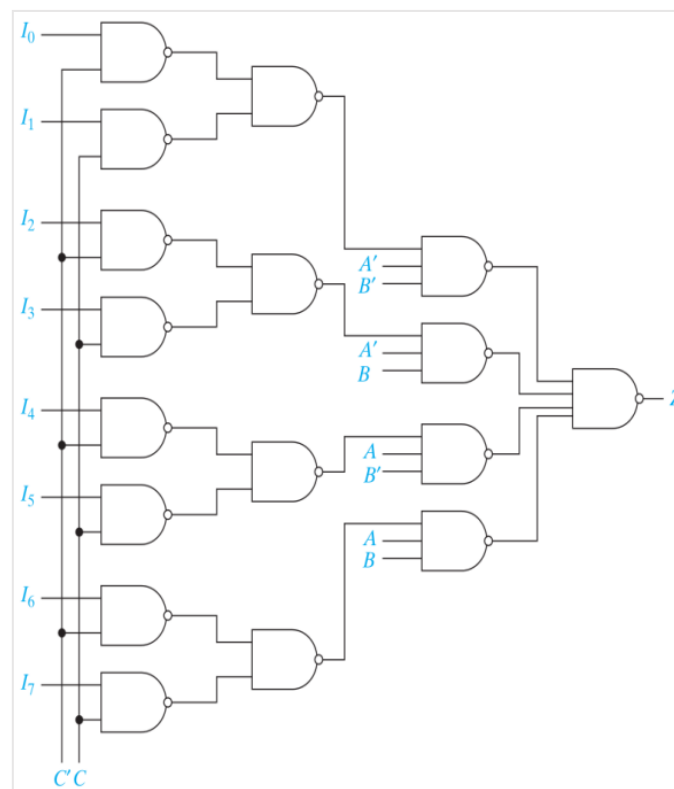
例：用四选一多路复用器如何实现 $F = A \text{ XOR } B$

8选1多路复用器的实现

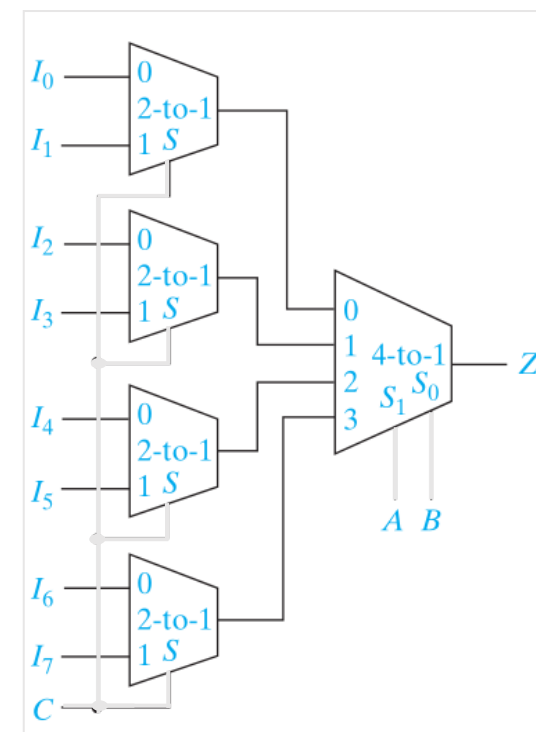
二级逻辑门电路实现



单一逻辑门实现



多路复用器级联实现



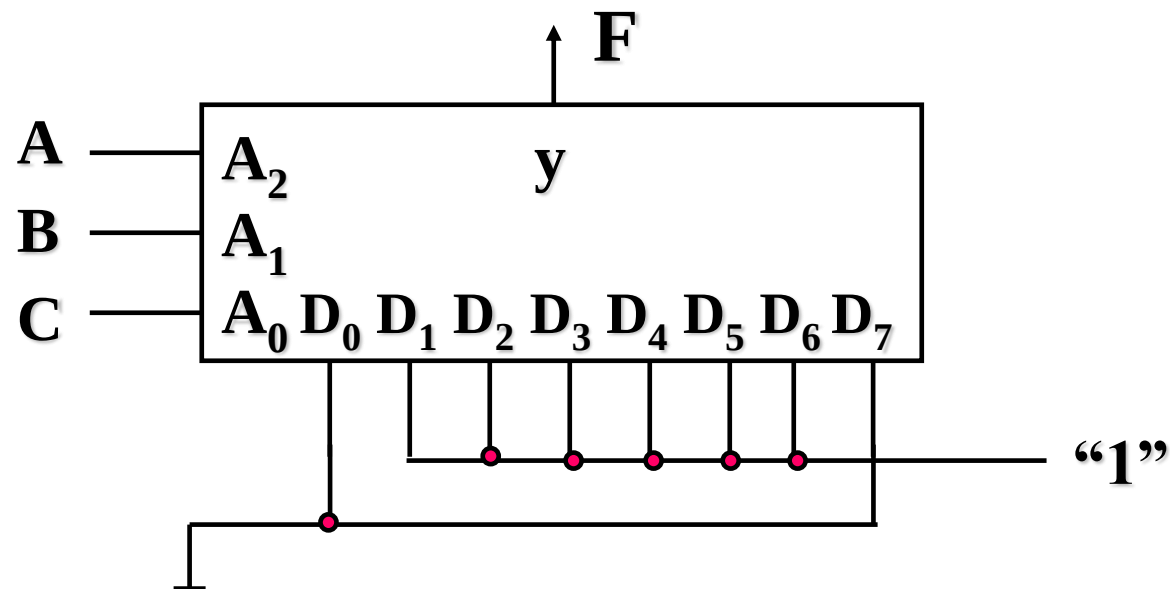
$$Z = A'B'C'I_0 + A'B'CI_1 + A'BC'I_2 + A'BCI_3 + AB'C'I_4 + AB'CI_5 + ABC'I_6 + ABCI_7$$

使用八选一多路复用器实现逻辑函数

$$F = AB' + A'C + BC'$$

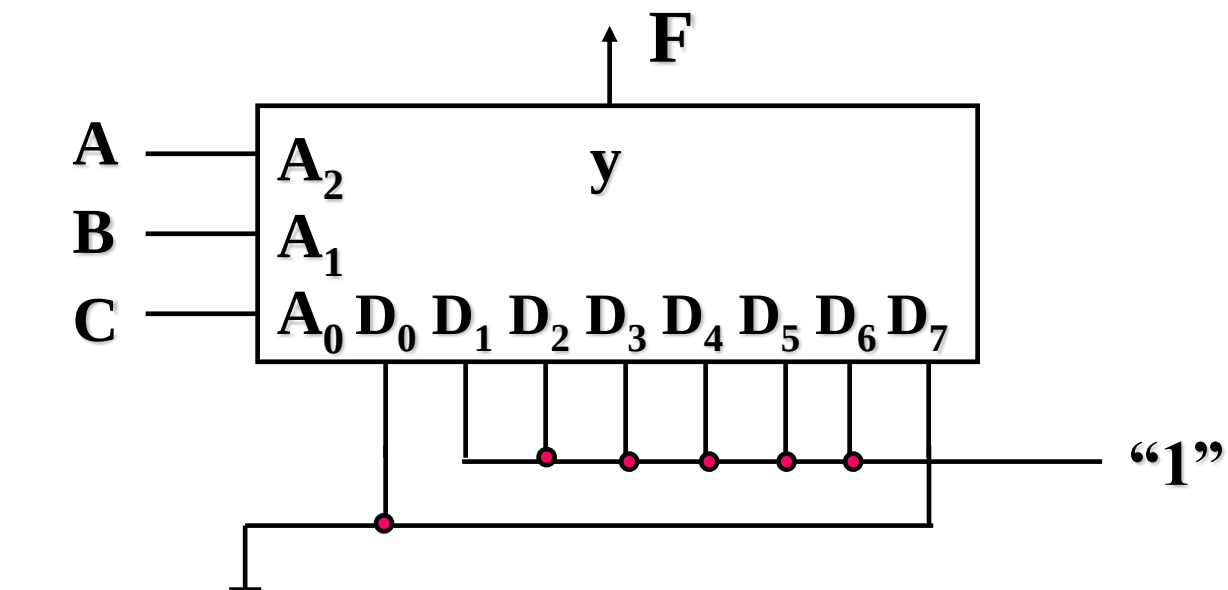
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

A ₂	A ₁	A ₀	y
0	0	0	D ₀
0	0	1	D ₁
0	1	0	D ₂
0	1	1	D ₃
1	0	0	D ₄
1	0	1	D ₅
1	1	0	D ₆
1	1	1	D ₇



使用八选一多路复用器实现逻辑函数—续

$$F = AB' + A'C + BC'$$



A	BC			
	00	01	11	10
0	0	1	1	1
1	1	1	0	1

A_2	$A_1 A_0$			
	00	01	11	10
0	D_0	D_1	D_3	D_2
1	D_4	D_5	D_7	D_6

如果变量更多呢？

F

多路复用器

=

四变量卡诺图变换

$$F=f(A, B, C, D)$$

AB \ CD	CD			
	00	01	11	10
00	1	0	1	0
01	1	1	0	1
11	0	0	1	0
10	0	1	0	1

AB \ C	C			
	0		1	
00	$1 * D'$	$0 * D$	$1 * D$	$0 * D'$
01	$1 * D'$	$1 * D$	$0 * D$	$1 * D'$
11	$0 * D'$	$0 * D$	$1 * D$	$0 * D'$
10	$0 * D'$	$1 * D$	$0 * D$	$1 * D'$

四变量卡诺图降维

AB \ C	0		1	
00	$1 * D'$	$0 * D$	$1 * D$	$0 * D'$
01	$1 * D'$	$1 * D$	$0 * D$	$1 * D'$
11	$0 * D'$	$0 * D$	$1 * D$	$0 * D'$
10	$0 * D'$	$1 * D$	$0 * D$	$1 * D'$

AB \ C	0		1	
00	$1 * D' + 0 * D$		$1 * D + 0 * D'$	
01	$1 * D' + 1 * D$		$0 * D + 1 * D'$	
11	$0 * D' + 0 * D$		$1 * D + 0 * D'$	
10	$0 * D' + 1 * D$		$0 * D + 1 * D'$	

四变量卡诺图降维

		C	
		0	1
AB	00	$1*D' + 0*D$	$1*D + 0*D'$
	01	$1*D' + 1*D$	$0*D + 1*D'$
	11	$0*D' + 0*D$	$1*D + 0*D'$
	10	$0*D' + 1*D$	$0*D + 1*D'$



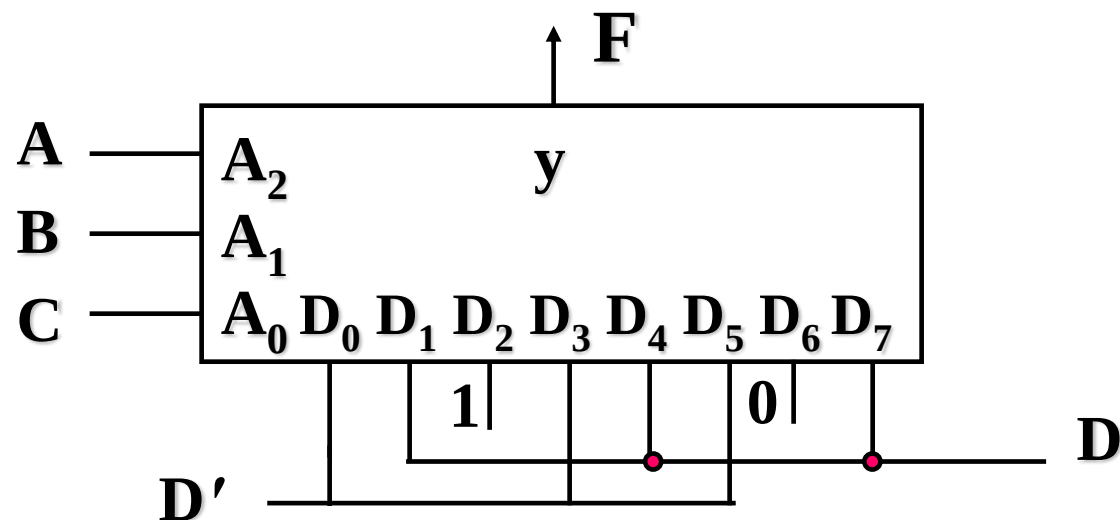
		C	
		0	1
AB	00	D'	D
	01	1	D'
	11	0	D
	10	D	D'



A	BC			
	00	01	11	10
0	D'	D	D'	1
1	D	D'	D	0

使用八选一多路复用器实现逻辑函数—续

$$F=f(A, B, C, D)$$



A \ BC				
	00	01	11	10
0	D'	D	D'	1
1	D	D'	D	0

A2 \ A1A0				
	00	01	11	10
0	D ₀	D ₁	D ₃	D ₂
1	D ₄	D ₅	D ₇	D ₆

F

=

多路复用器

单个 n 控制变量的多路复用器 最多可以表示 $n+1$ 个变量的函数式

五变量卡诺图降维

$F=f(A,B,C,D,E)$

		CDE							
		000	001	011	010	110	111	101	100
AB	00	0	1	3	2	6	7	5	4
	01	8	9	11	10	14	15	13	12
	11	24	25	27	26	30	31	29	28
	10	16	17	19	18	22	23	21	20

		CD			
		00	01	11	10
AB	00	$f_0(E,E')$	$f_1(E,E')$	$f_3(E,E')$	$f_2(E,E')$
	01	$f_4(E,E')$	$f_5(E,E')$	$f_7(E,E')$	$f_6(E,E')$
	11	$f_{12}(E,E')$	$f_{13}(E,E')$	$f_{15}(E,E')$	$f_{14}(E,E')$
	10	$f_8(E,E')$	$f_9(E,E')$	$f_{11}(E,E')$	$f_{10}(E,E')$

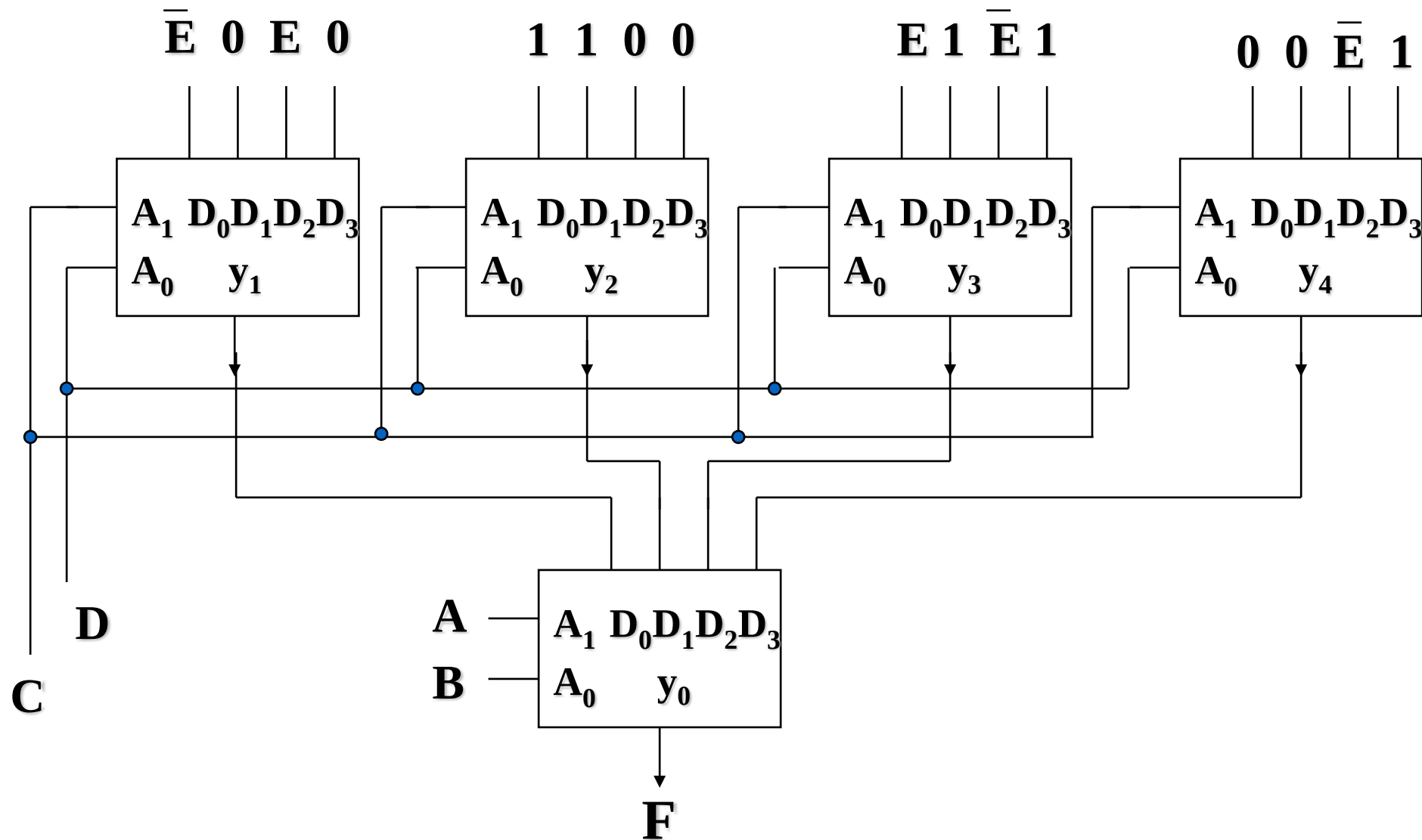
利用四选一多路复用器设计组合逻辑

- $F(A,B,C,D,E) = \sum m(0,5,8,9,10,11,17,18,19,20,22,23,28,30,31)$

A B	C D	E	F
00	0 0	0	1
		1	0
	0 1	0	0
		1	0
	1 0	0	0
		1	1
	1 1	0	0
		1	0
01	0 0	0	1
		1	1
	0 1	0	1
		1	1
	1 0	0	0
		1	0
	1 1	0	0
		1	0

A B	C D	E	F
10	0 0	0	0
		1	1
	0 1	0	1
		1	1
	1 0	0	1
		1	0
	1 1	0	1
		1	1
11	0 0	0	0
		1	0
	0 1	0	0
		1	0
	1 0	0	1
		1	0
	1 1	0	1
		1	1

利用四选一多路复用器设计组合逻辑

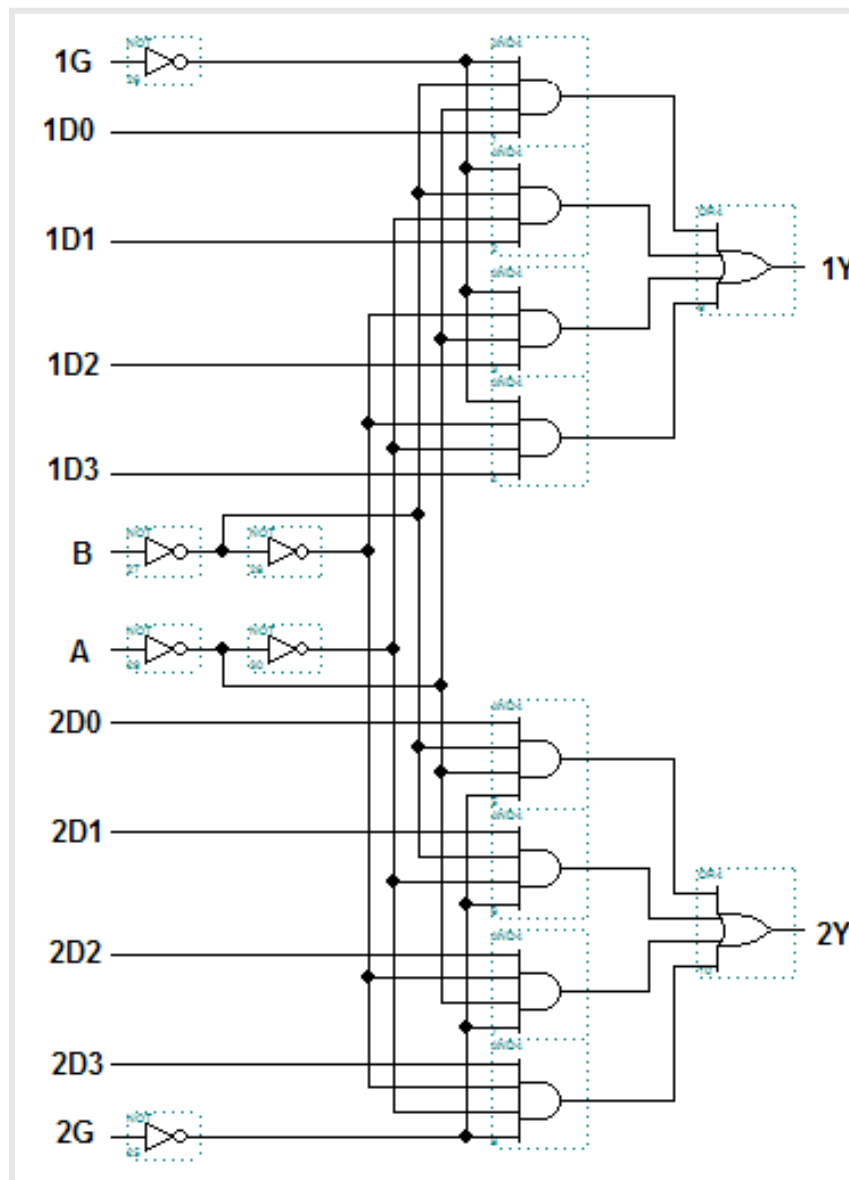
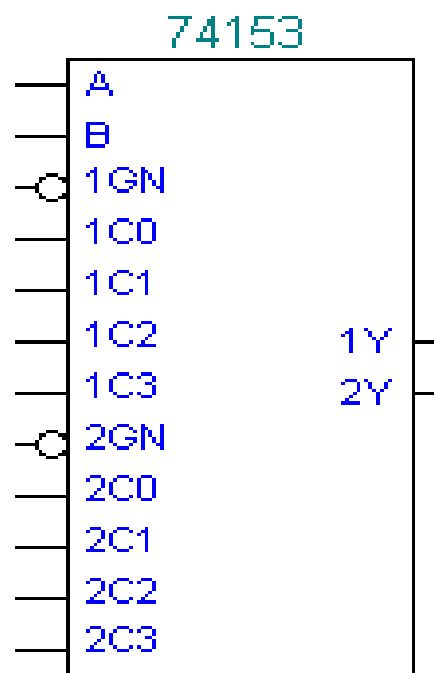


用多路复用器实现组合逻辑电路

- 组合逻辑电路输入变量数**等于**多路复用器的控制端数
 - 输入变量 接到 选择控制端
 - 多路复用器的输入端按照真值表连接相应的逻辑常量
- 组合逻辑电路输入变量数**少于**多路复用器的控制端数
 - 输入变量接到选择控制端的低位或高位
 - 多路复用器的输入端按照真值表连接相应的逻辑常量
 - 不用的选择控制端接常量0/1
- 组合逻辑电路输入变量数**大于**多路复用器的控制端数
 - 一部分输入变量（或其表达式）接到选择控制端
 - 另一部分按照卡诺图降维的结果在多路复用器的输入端连接相应的逻辑表达式（由剩余输入变量组成）

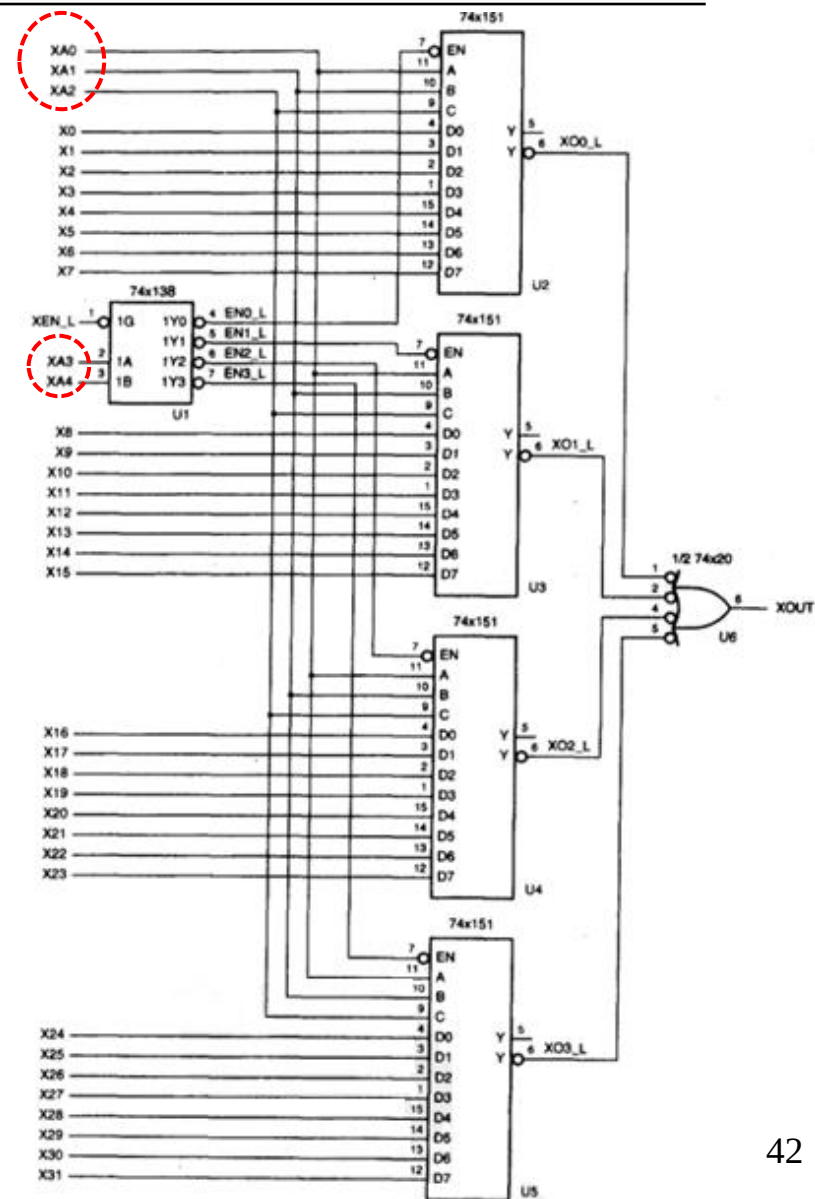
双4选1多路复用器典型器件74LS153

1Gn	2Gn	A	B	1Y	2Y
1	1	x	x	0	0
0	0	0	0	1C ₀	2C ₀
0	0	0	1	1C ₁	2C ₁
0	0	1	0	1C ₂	2C ₂
0	0	1	1	1C ₃	2C ₃



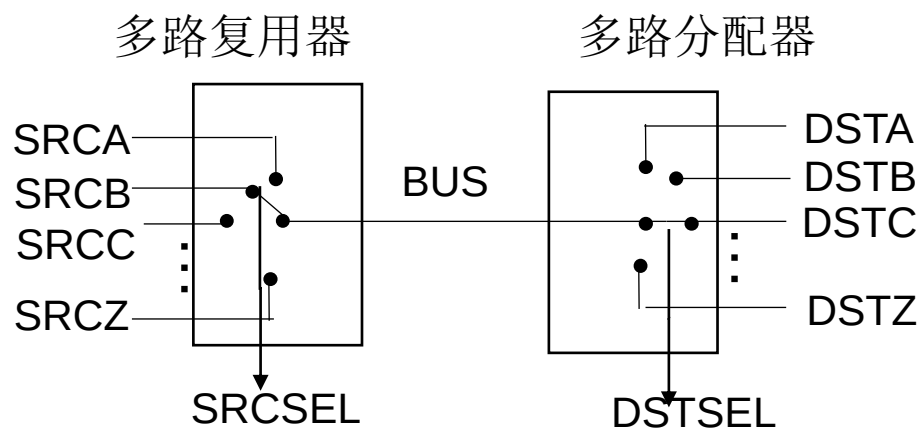
扩展多路复用器

- 给定8选1多路复用器74LS151
- 实现32选1多路复用器
- 用一个2-4译码器对2个最高选择位进行译码，以从4个74LS151多路复用器中选择1个

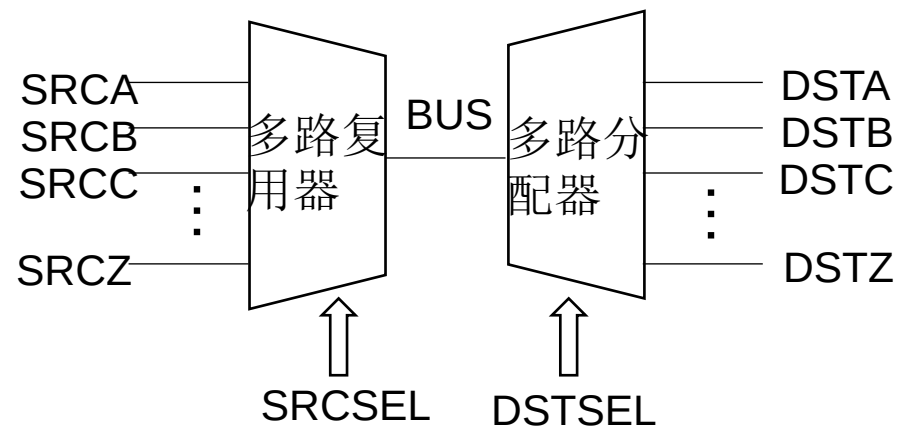


多路复用器、多路分配器和总线

- 多路复用器可以用于选择发往总线的n个数据源之一，即从多路信号中选择一路信号输出。
- 在总线的远端，多路分配器可以用于把总线数据送到m个目的地之一，即将总线数据传送到所选择的输出端口。



开关等效示意图



符号框图

组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

三态缓冲器 three-state buffer

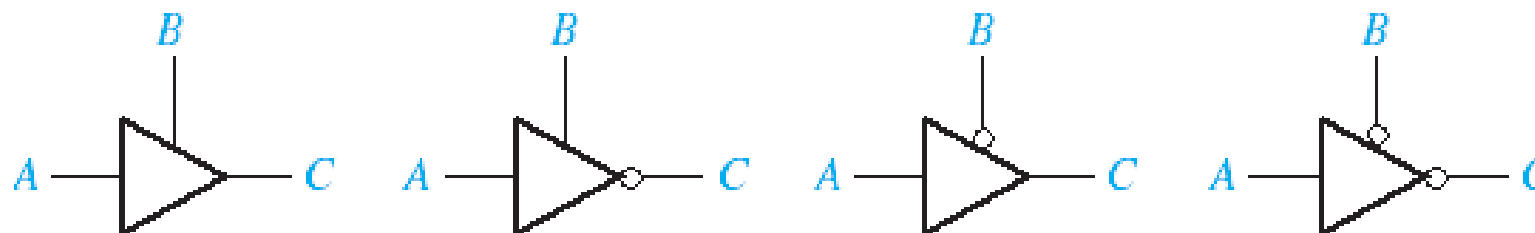
三态——

■ 0

■ 1

■ Z: 高阻态电阻很大，相当于开路

➤ 4种不同的三态缓冲器的逻辑符号



B	A	C
0	0	Z
0	1	Z
1	0	0
1	1	1

(a)

B	A	C
0	0	Z
0	1	Z
1	0	1
1	1	0

(b)

B	A	C
0	0	0
0	1	1
1	0	Z
1	1	Z

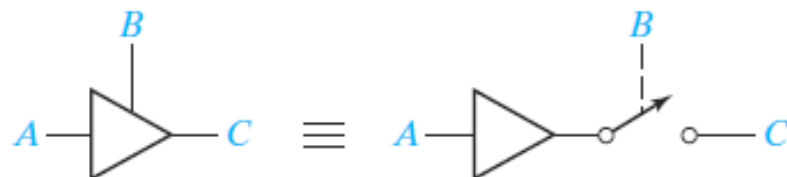
(c)

B	A	C
0	0	1
0	1	0
1	0	Z
1	1	Z

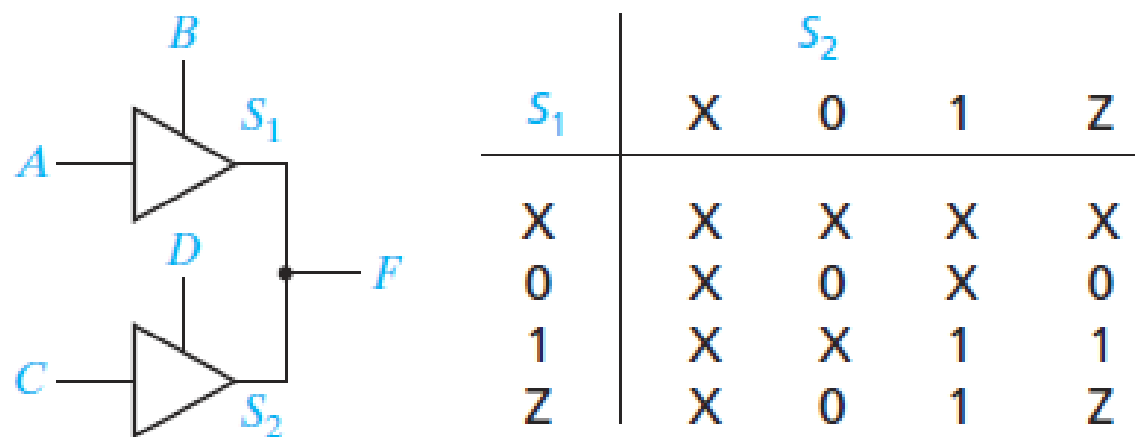
(d)

■ 又称三态驱动器

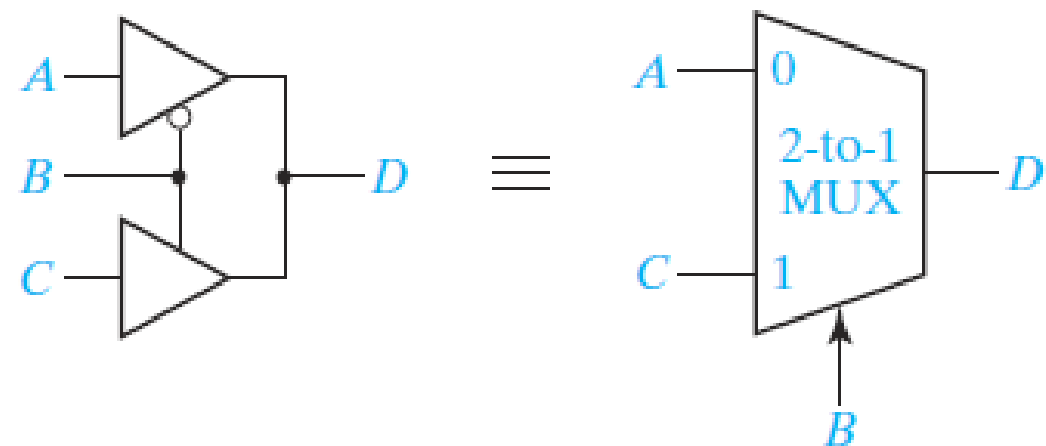
三态缓冲器还可用来增强输出驱动能力。



三态缓冲器的应用



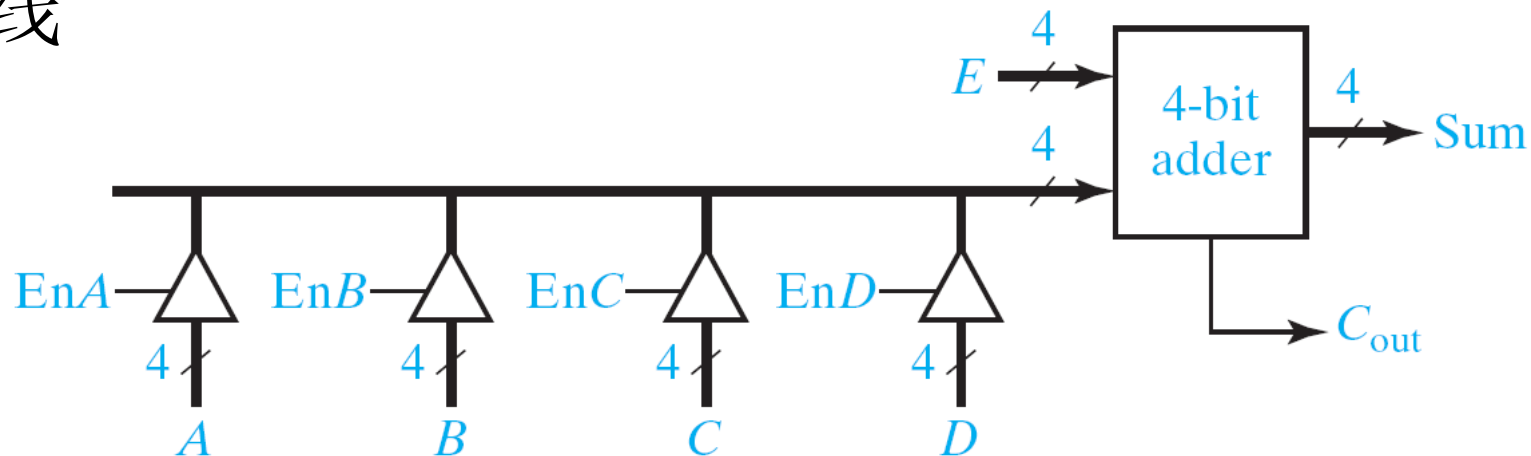
具有两个三态缓冲器的电路



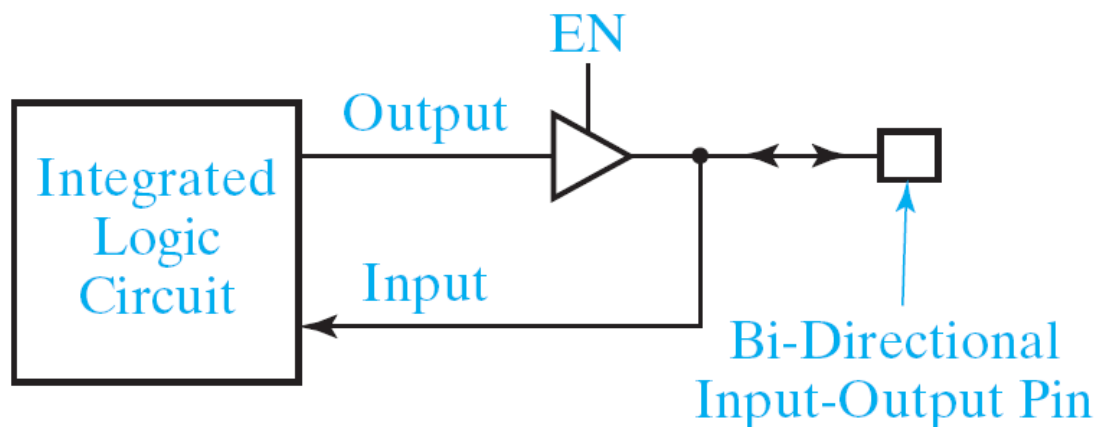
用三态缓冲器实现数据选择

三态缓冲器的应用——续

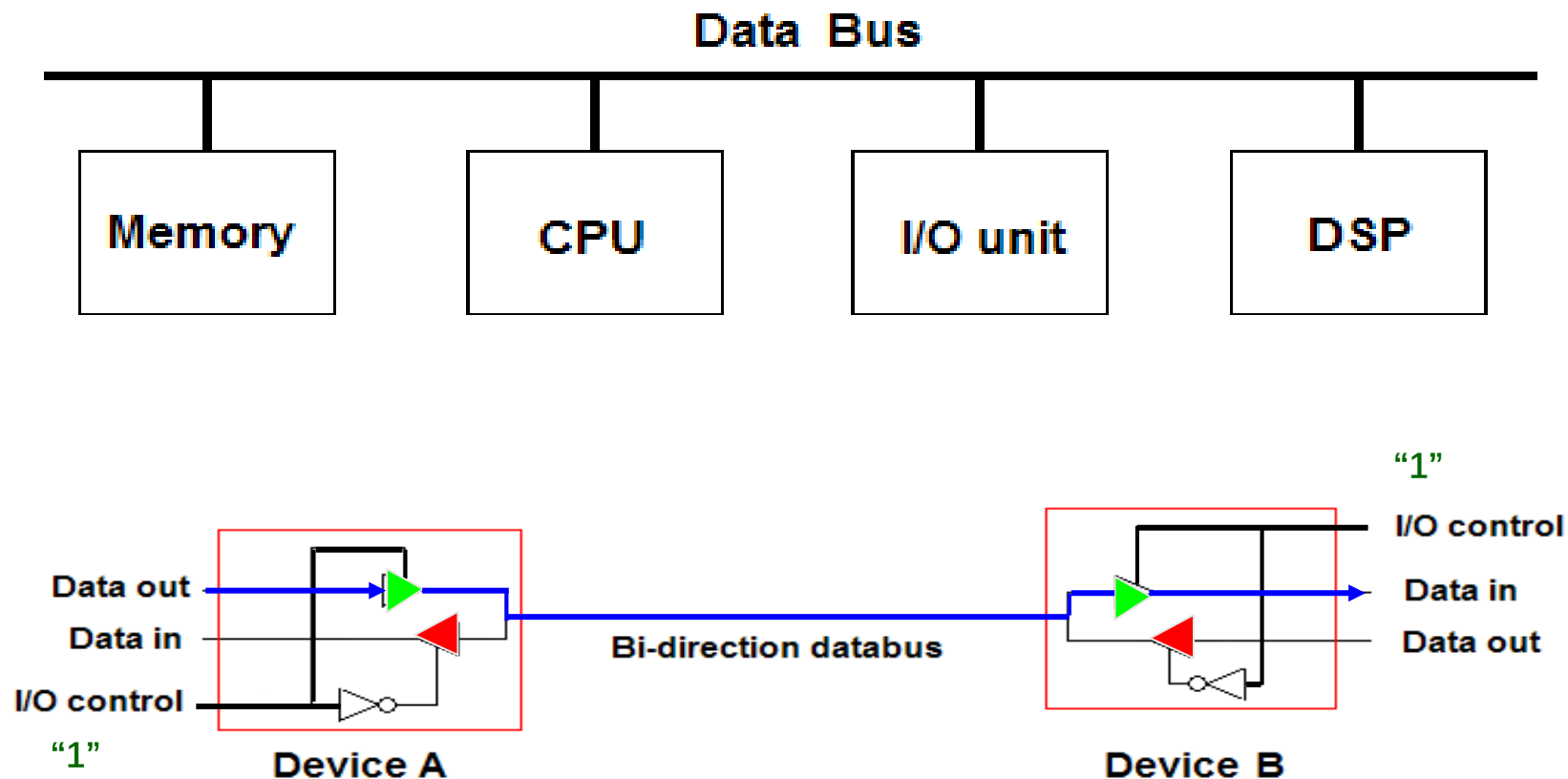
- 三态总线



- 管脚输入输出可编程



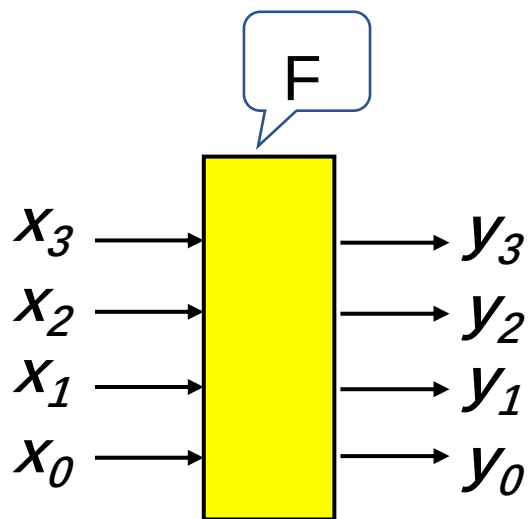
三态缓冲器的应用——双向数据总线



三态缓冲器的应用——MOD 5选择电路

- $X=X_3X_2X_1X_0$ 为8421BCD码，设计一个MOD 5选择电路，要求选择那些能被5整除的数输出。

①真值表(F为控制信号)



$X_3 X_2 X_1 X_0$	F	$X_3 X_2 X_1 X_0$	F
0 0 0 0	1	1 0 0 0	0
0 0 0 1	0	1 0 0 1	0
0 0 1 0	0	1 0 1 0	×
0 0 1 1	0	1 0 1 1	×
0 1 0 0	0	1 1 0 0	×
0 1 0 1	1	1 1 0 1	×
0 1 1 0	0	1 1 1 0	×
0 1 1 1	0	1 1 1 1	×

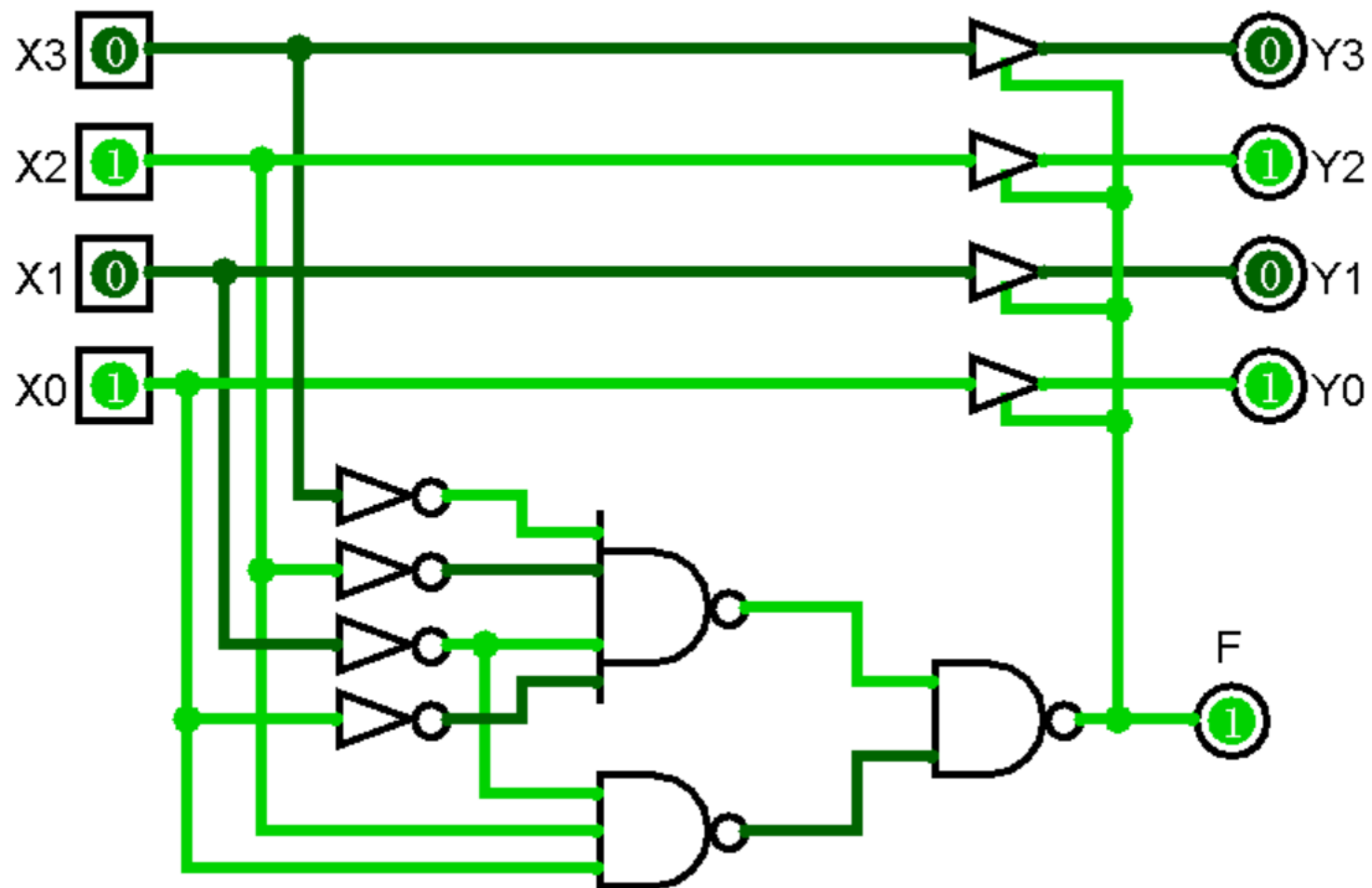
MOD 5选择电路——续

② 化简

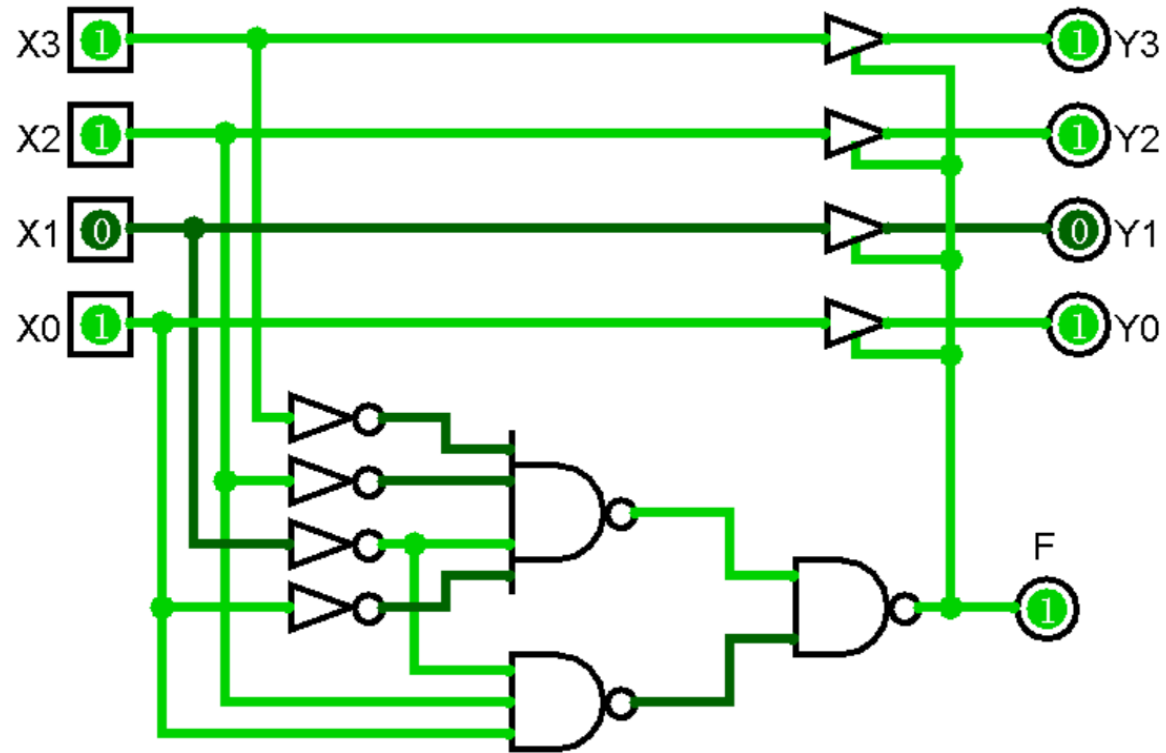
x_1x_0					
x_3x_2	00	01	11	10	
00	1	0	0	0	
01	0	1	0	0	
11	x	x	x	x	
10	0	0	x	x	

$$\begin{aligned} F &= \overline{\overline{X_2\overline{X_1}X_0} + \overline{\overline{X_3}\overline{X_2}\overline{X_1}\overline{X_0}}} \\ &= (\overline{X_2\overline{X_1}X_0})(\overline{\overline{X_3}\overline{X_2}\overline{X_1}\overline{X_0}}) \end{aligned}$$

③ 逻辑图



MOD 5选择电路——续



×

MOD 5选择电路——改进

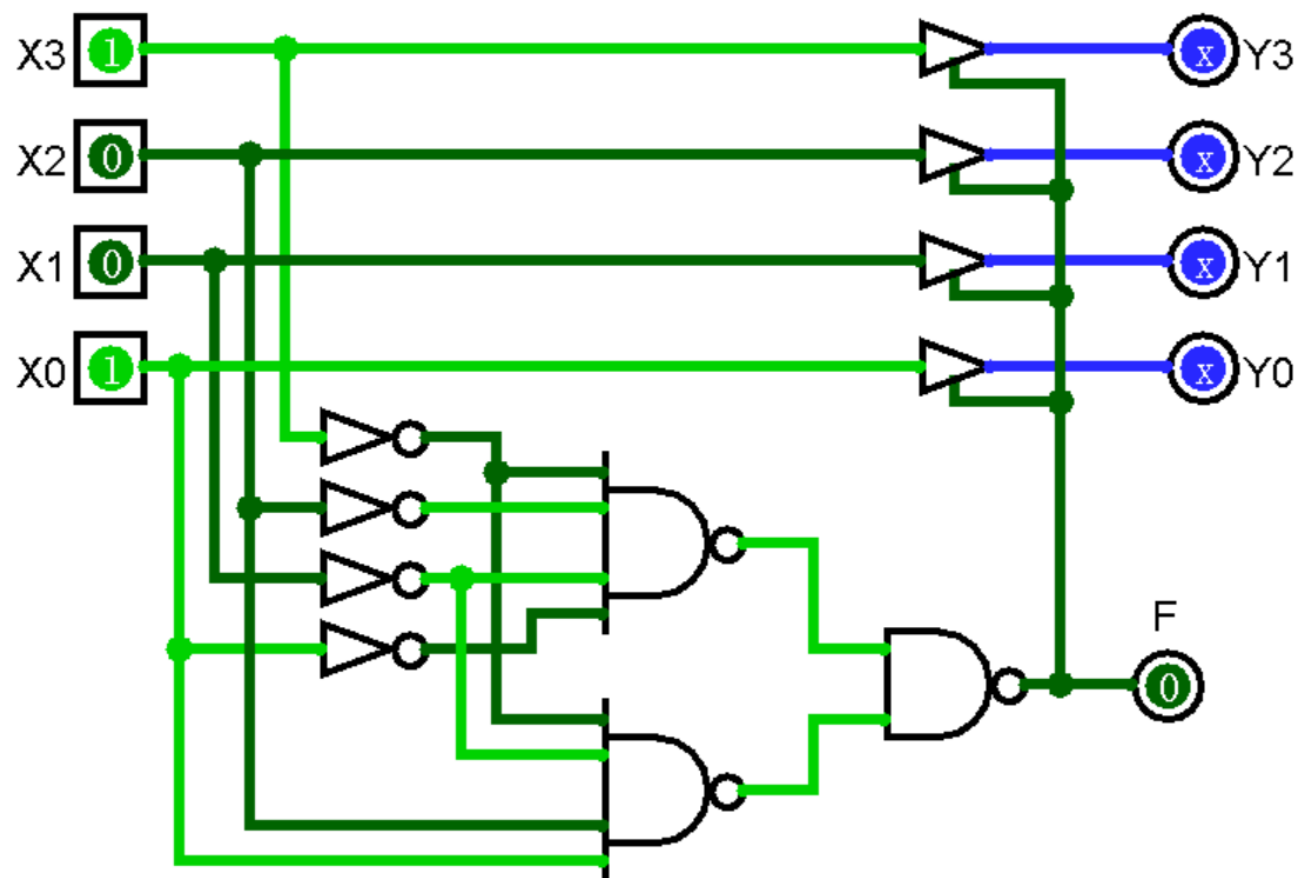
② 化简

$x_3x_2 \backslash x_1x_0$		00	01	11	10
		00	01	11	10
00	1	0	0	0	
01	0	1	0	0	
11	0	0	0	0	
10	0	0	0	0	

$$F = (X_3' X_2 X_1' X_0 + X_3' X_2' X_1' X_0')'' \\ = ((X_3' X_2 X_1' X_0)' (X_3' X_2' X_1' X_0'))'$$

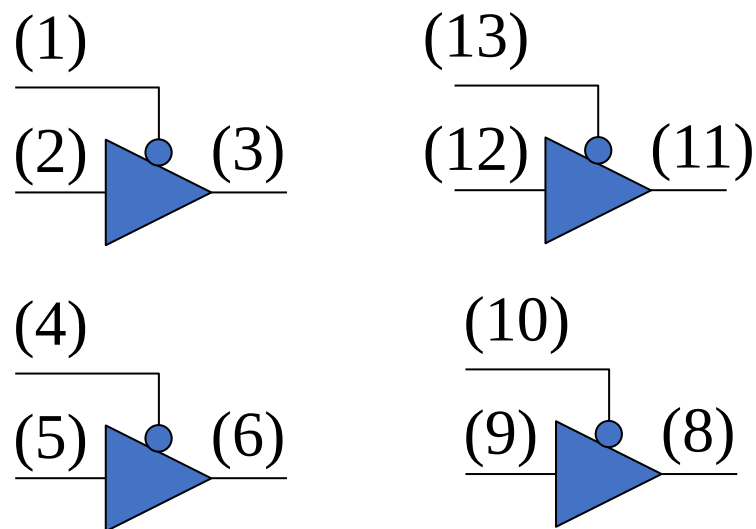
将无关项视为0

③ 逻辑图



典型三态缓冲器——四线缓冲器74LS125

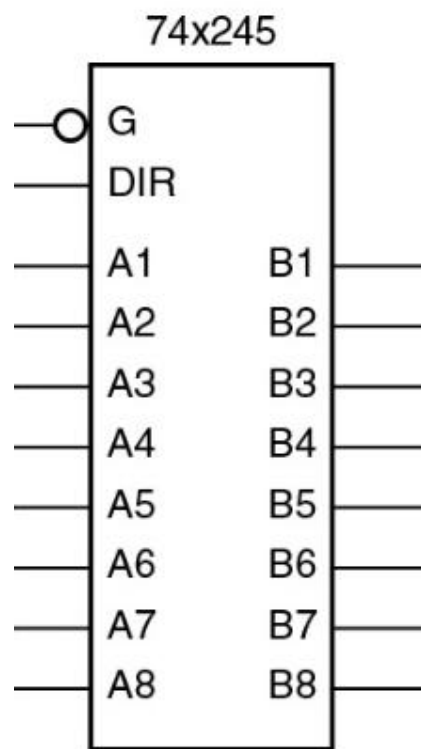
- 几个独立的三态缓冲器可以封装在单个IC中



74x245三态缓冲器的管脚引线

典型三态缓冲器——八缓冲器74x541

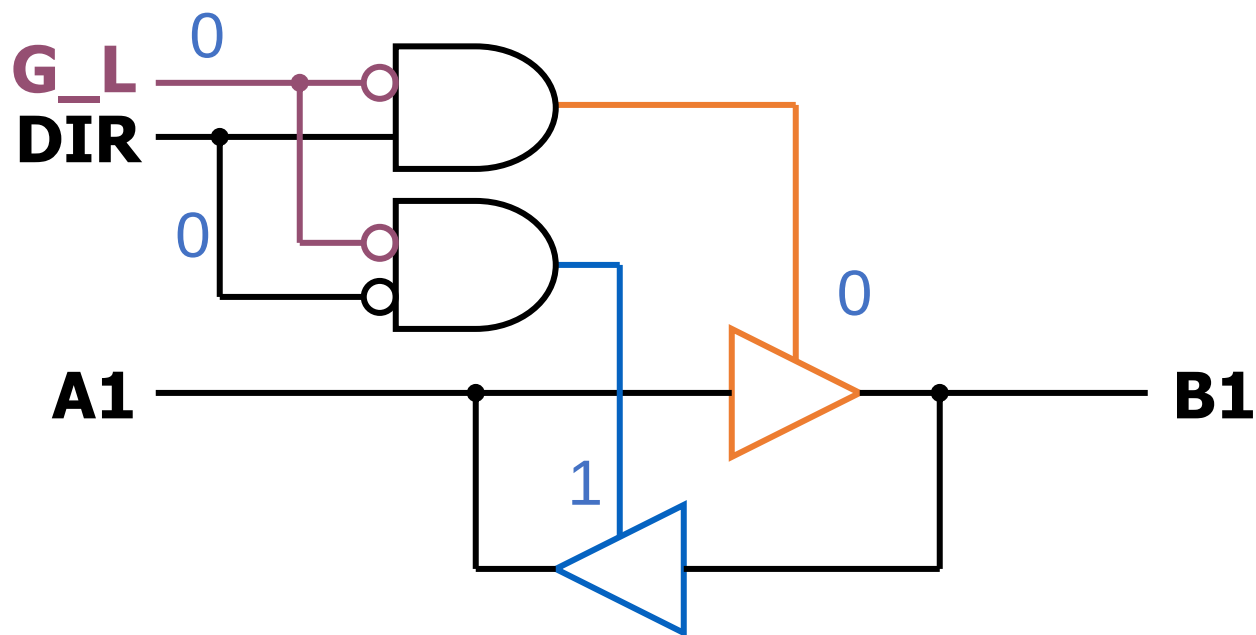
- 为了减少封装尺寸，多数常用的三态缓冲器中规模集成电路部件包含带有公共使能输入。



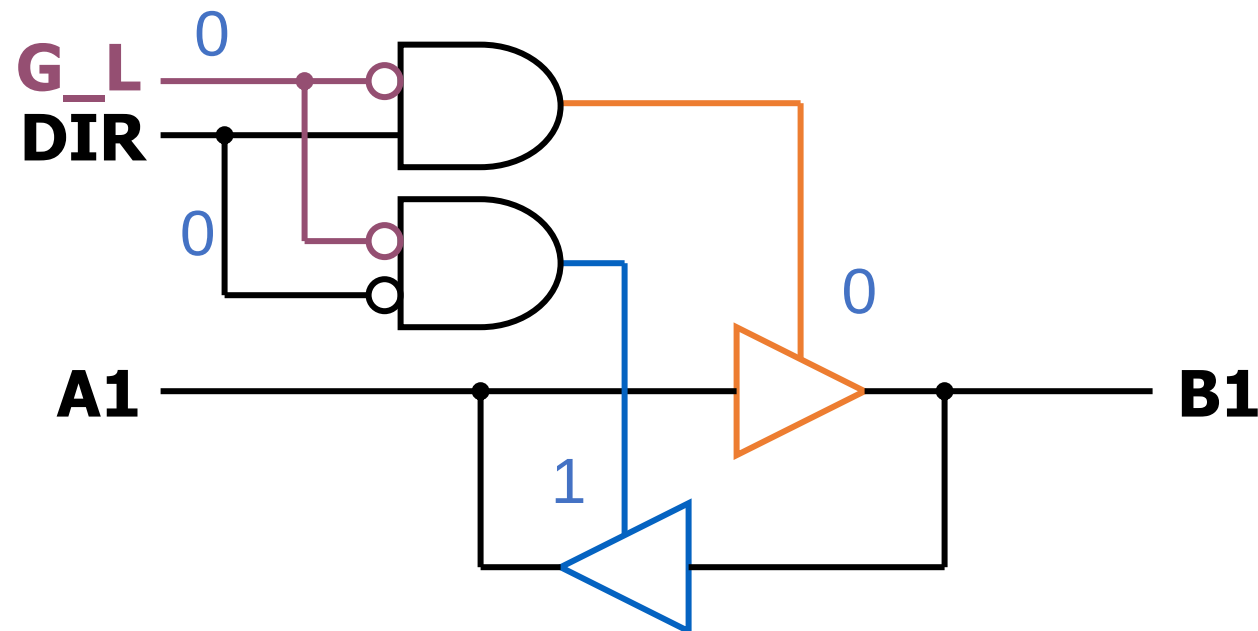
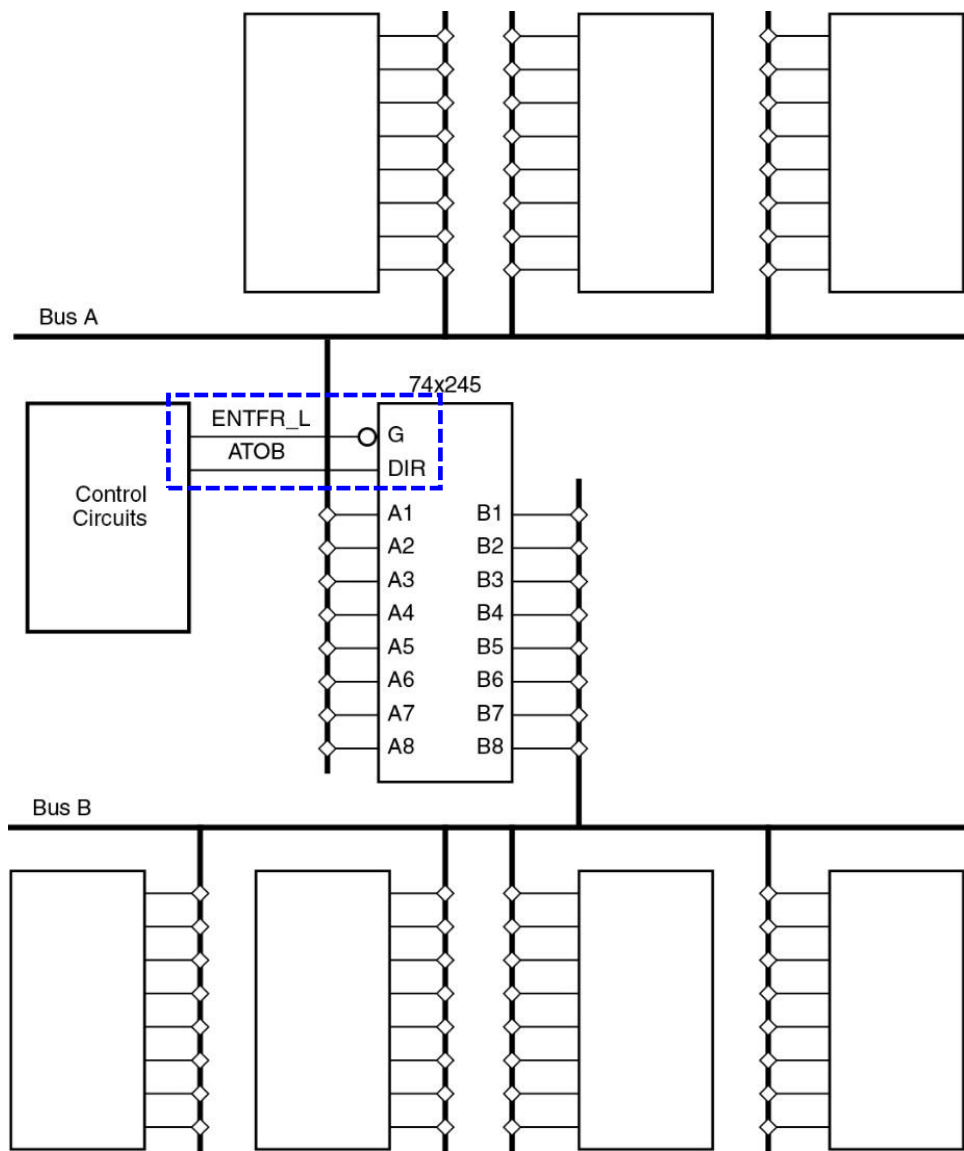
8三态缓冲器74x245

典型三态缓冲器应用——总线收发器

- 包含三态缓冲器对，每对引脚之间以相反方向连接，所以数据可以双向传输。
- 常用于两个双向总线之间，根据G_L和DIR的状态，控制操作模式。



双向总线示例



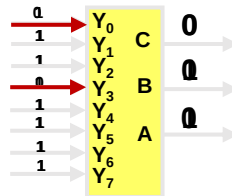
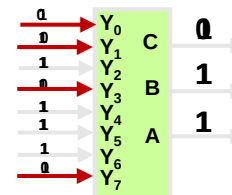
ENTRF_L	ATOB	操作
0	0	B——>A
0	1	A——>B
1	x	AB单独传输

组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

编码器

- 特点：多输入、多输出的组合逻辑电路
- 功能：将二进制码按照一定规律编排，使其具有特定含义，与译码器互逆。

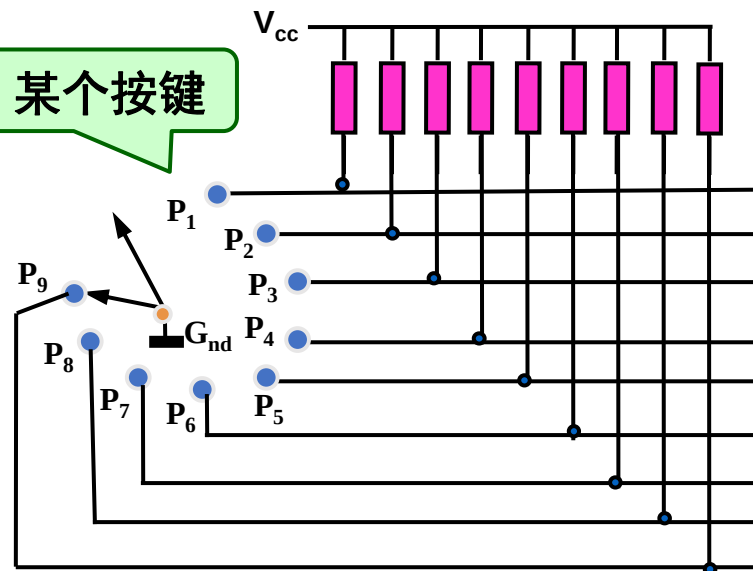
常用编码器	特点	编码演示
普通编码器 (二进制编码器)	N 位，任何时刻 N 根输入线中只能有一个输入有效， N ($N=2^n$) 中取一。 n 位二进制码	 (8 线-3 线编码器)
优先编码器	• 允许同时输入两个以上有效输入信号 • 能按照预先设定的优先级别，只对其 中优先级最高的输入进行编码。	 (8 线-3 线优先编码器)

键盘编码器

键盘编码器功能表

$P_9 \dots P_1$	按键	WXYZ
111111111	0	0000
111111110	1	0001
111111101	2	0010
111111011	3	0011
111110111	4	0100
111101111	5	0101
111011111	6	0110
110111111	7	0111
101111111	8	1000
011111111	9	1001

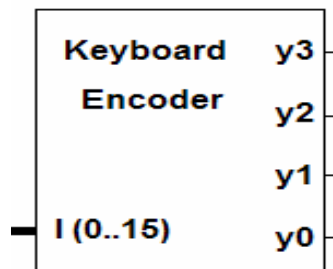
输入：某个按键



输出：
8421BCD码

扩展

7	8	9	/
4	5	6	*
1	2	3	-
0	.	+	=



0	I0	0000	8	I8	1000
1	I1	0001	9	I9	1001
2	I2	0010	/	I10	1010
3	I3	0011	*	I11	1011
4	I4	0100	-	I12	1100
5	I5	0101	+	I13	1101
6	I6	0110	.	I14	1110
7	I7	0111	=	I15	1111

$$W=(P_8 \cdot P_9)'$$

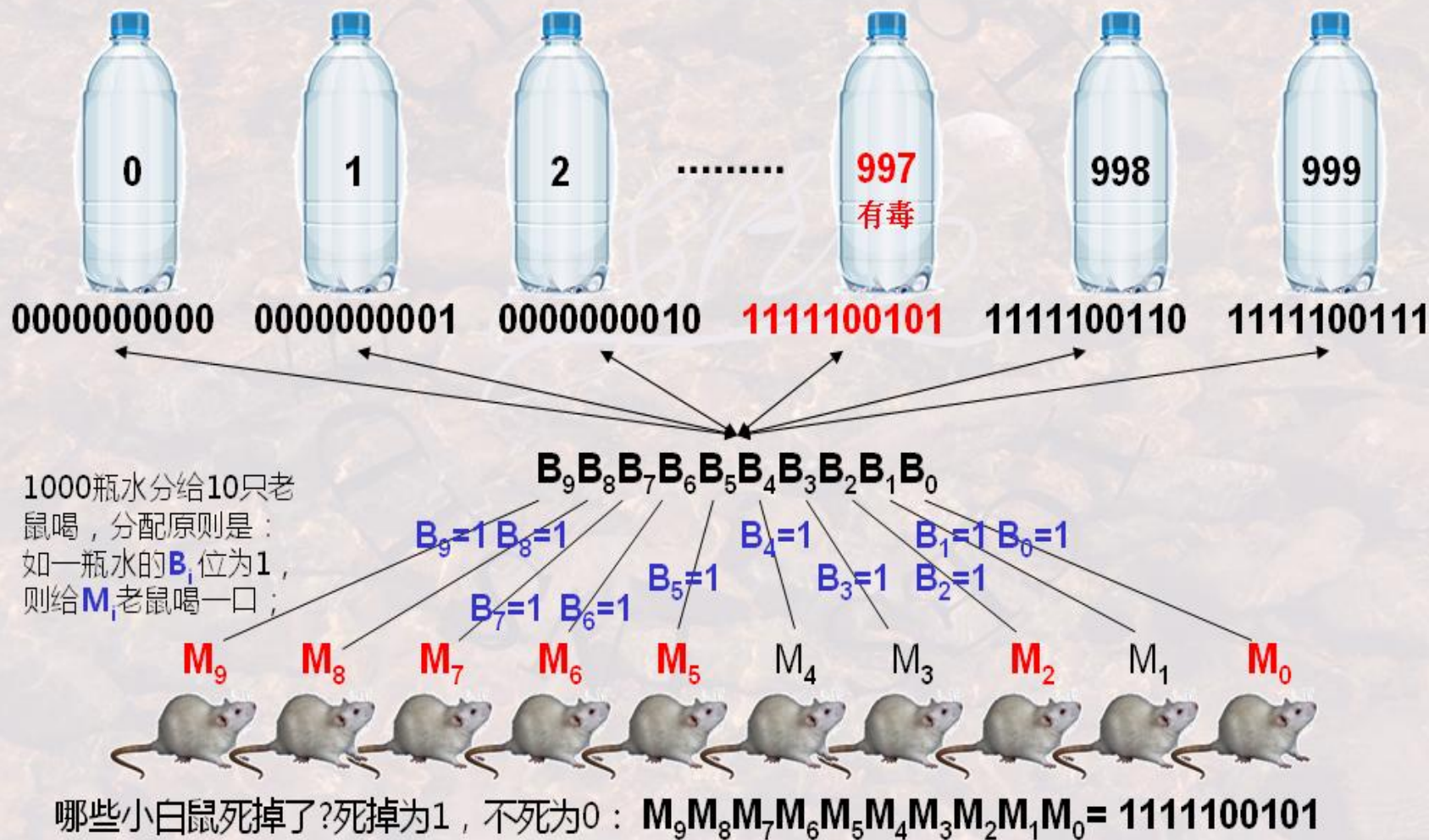
$$Y=(P_2 \cdot P_3 \cdot P_6 \cdot P_7)'$$

$$X=(P_4 \cdot P_5 \cdot P_6 \cdot P_7)'$$

$$Z=(P_1 \cdot P_3 \cdot P_5 \cdot P_7 \cdot P_9)'$$

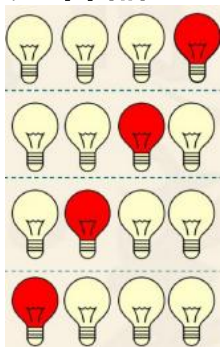
小白鼠检测毒水问题

有1000瓶水，其中有一瓶有毒，小白鼠只要尝一点带毒的水24小时内就会死亡，至少要多少只小白鼠才能在24小时内鉴别出那瓶水有毒。



优先编码器

抢答器



4:2编码器

A_3	A_2	A_1	A_0	B_1	B_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

计算机配有四个外部设备：声卡(A0), 硬盘驱动器(A1), 鼠标(A2), 网卡(A3), B_0 、 B_1 为编码输出。

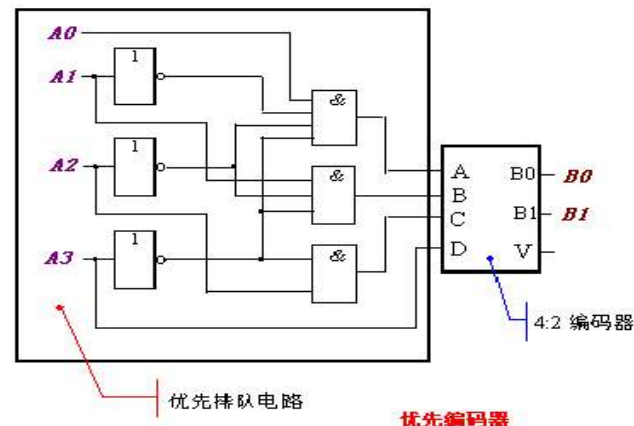


某一时刻只允许输入一个编码信号，如 A_1 ($A_1=1$) 向 CPU 请求传送数据，CPU 根据接收的编码 $B_1B_0 = 01$ ，启动硬盘驱动器，开始传送数据。

4:2优先编码器

A_3	A_2	A_1	A_0	B_1	B_0
0	0	0	1	0	0
0	0	1	X	0	1
0	1	X	X	1	0
1	X	X	X	1	1

普通编码器：无法避免错误输入（同时输入多路有效信号），容易造成混乱。



$$A = A_0 \overline{A_1} \overline{A_2} \overline{A_3}$$

$$B = A_1 \overline{A_2} \overline{A_3}$$

$$C = A_2 \overline{A_3}$$

$$D = A_3$$

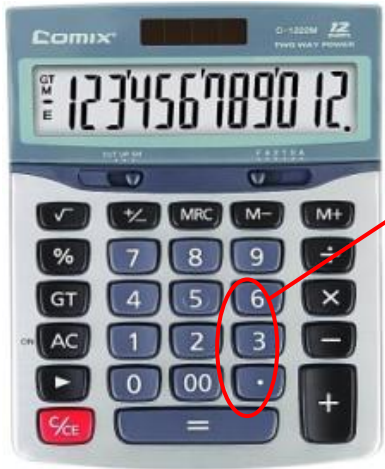
二进制编码器：

- 可以对 2^n 个输入对象编码
- 只需 n 个输出端（每个对象获得一个 n 位编码）
- 编码具有唯一性

优先编码器：

- 允许同时输入多路有效信号
- 按照预先设定的优先级，只对其中优先级最高的输入进行编码。

编码器与译码器的实际应用



键盘编码器

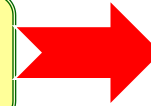


存储、运算等
处理....

计算结果
1001



显示译码器



显示器件



哪部分有疑问？

- | | |
|---|------------|
| A | 译码器 |
| B | 多路复用器/多路开关 |
| C | 三态缓冲器 |
| D | 编码器 |
| E | 优先编码器 |
| F | 无 |

提交

组合逻辑元件

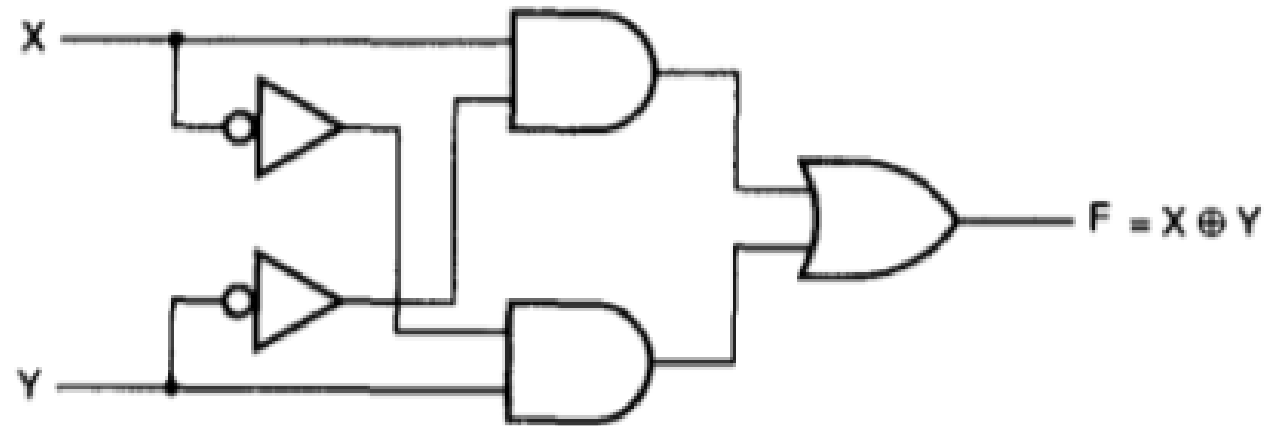
- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

异或门和异或非门

异或门是二输入门，如果两个输入不同，则输出为1。

异或非门正好相反，如果两个输入相同，则输出为1。

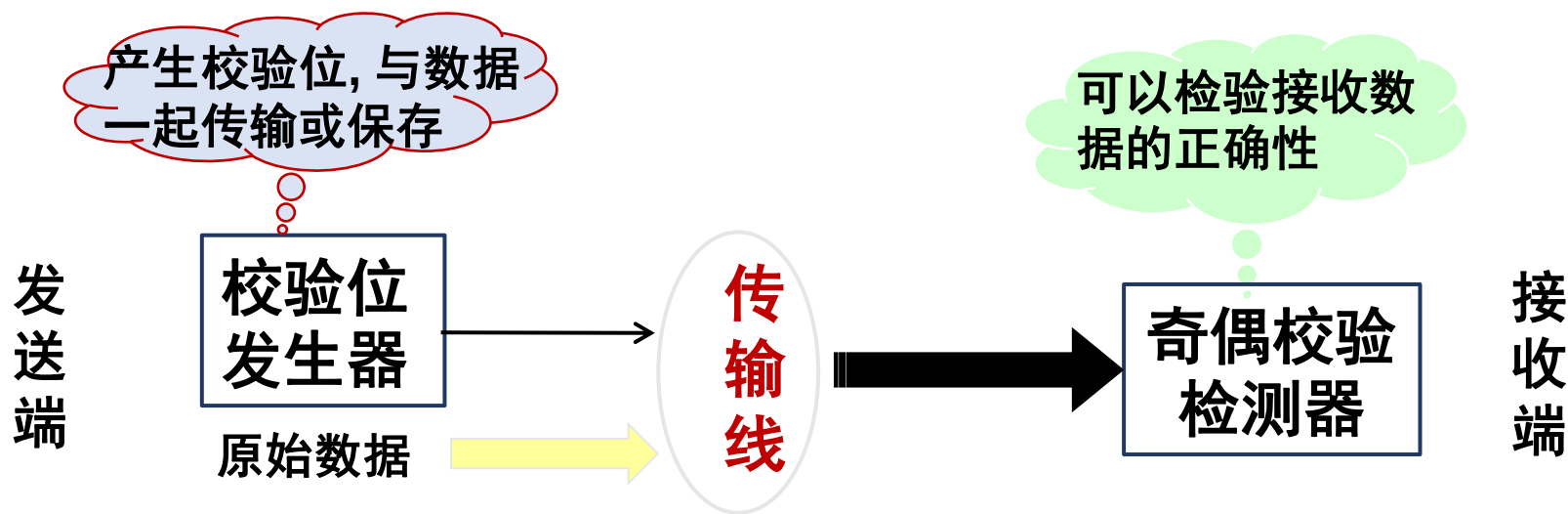
真值表			
X	Y	$X \oplus Y$	$(X \oplus Y)'$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1



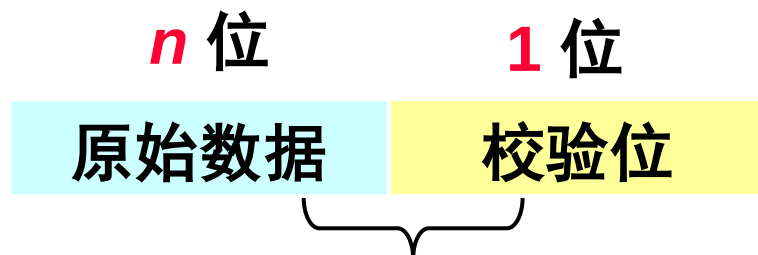
2输入异或函数的多门设计

奇偶校验器

- 用来检查数据传输和存取过程中是否产生错误的组合逻辑电路。
 - 发生一位错误的可能性一般占96%以上；
- 就是检测数据中包含“1”的个数是奇数还是偶数。
 - **偶**（Even）校验：校验码（数据位+校验位）有**偶数个1**，表示校验无误。
- 能够检测传送出错，但不能确定错误位置，所以不能纠错。
- 电路简单，容易实现。
- 广泛用于计算机的内存储器以及磁盘等外部设备中。



奇偶校验位计算方法



校验码: $n+1$ 位

偶 (Even) 校验位逻辑值的表达式:

$$P_E = A_3 \oplus A_2 \oplus A_1 \oplus A_0$$

奇 (Odd) 校验位逻辑值的表达式:

$$P_O = A_3 \oplus A_2 \oplus A_1 \oplus A_0$$

异或真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

奇偶校验器一般
由异或门构成

异或门特性

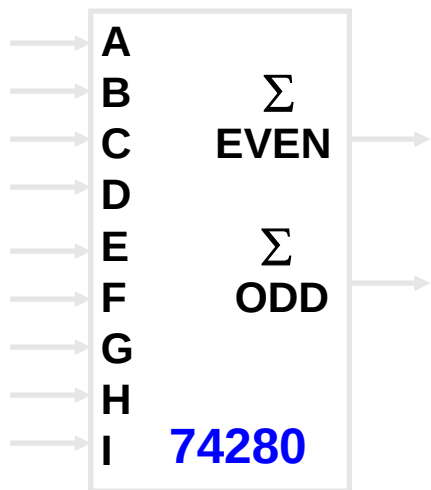
- 两个输入中有奇数个“1”，输出为1；
有偶数个“1”，输出为0。
- 扩展: n 个1位二进制数中有奇数个“1”，输出为1；
有偶数个“1”，输出为0。

4位二进制数校验码真值表

$A_3A_2A_1A_0$	P_E	P_O
0 0 0 0	0	1
0 0 0 1	1	0
0 0 1 0	1	0
0 0 1 1	0	1
0 1 0 0	1	0
0 1 0 1	0	1
0 1 1 0	0	1
0 1 1 1	1	0
1 0 0 0	1	0
1 0 0 1	0	1
1 0 1 0	0	1
1 0 1 1	1	0
1 1 0 0	0	1
1 1 0 1	1	0
1 1 1 0	1	0
1 1 1 1	0	1

奇偶校验器74LS280

奇偶校验器/产生器：
74xx180/74xx280

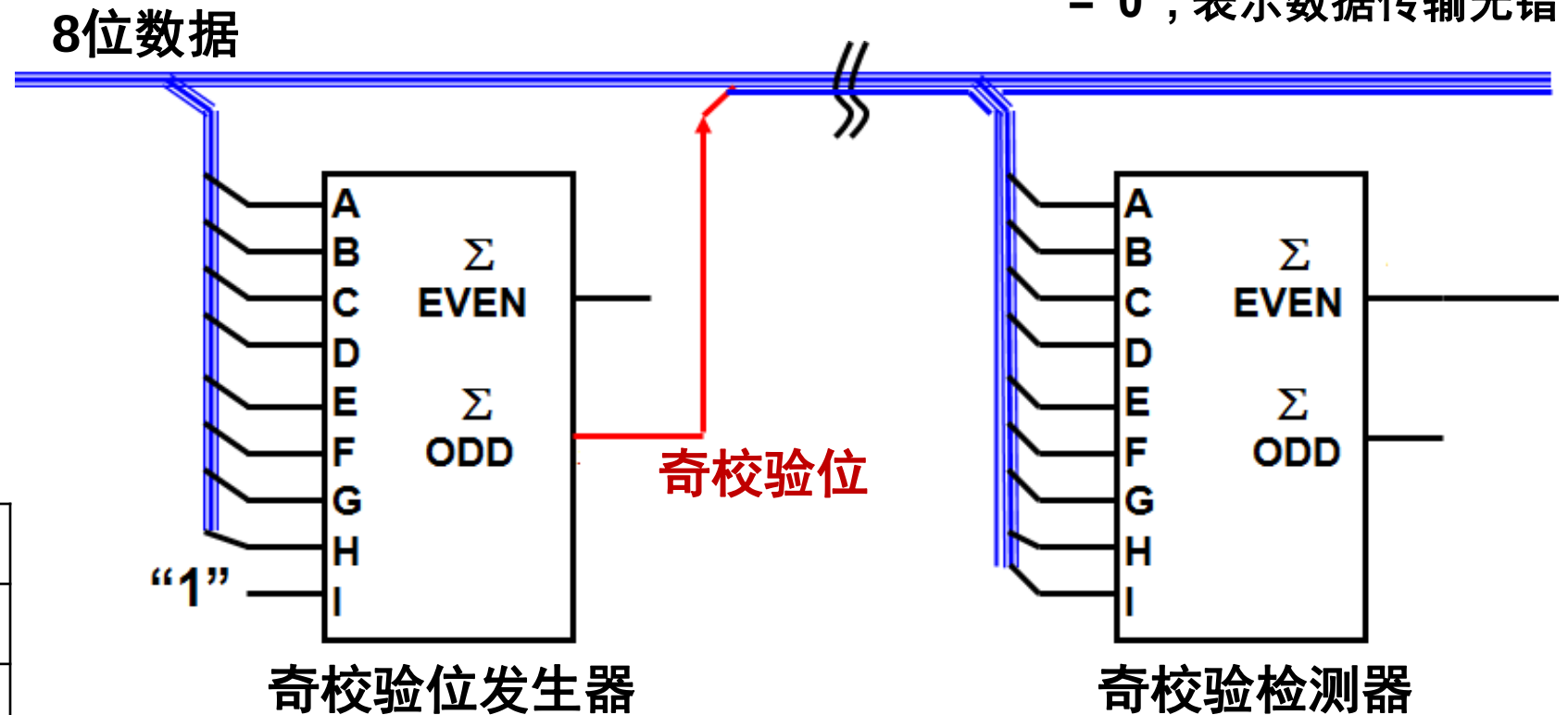


74XX280功能表

A~I	EVEN	ODD
偶数个“1”	1	0
奇数个“1”	0	1

例) 用9位奇偶校验器74LS280设计一个8位二进制码的奇校验位**发生器**和奇校验**检测器**，发生错误时输出1，否则输出0。

=“1”，表示数据传输有差错
=“0”，表示数据传输无错

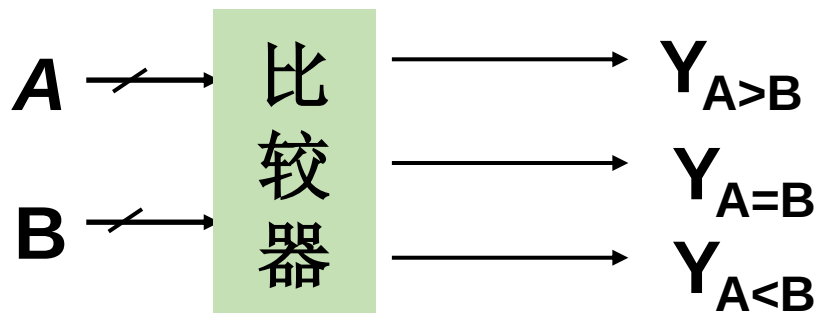


组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

数值比较器

- 计算机中对数据的基本处理方法
 - 加、减、乘、除
 - 比较运算
- 数值比较器：一种关系运算电路
 - 能对两个n位二进制数A和B 进行比较的多输入、多输出的组合逻辑电路
 - 比较结果： $Y_{A>B}$ 、 $Y_{A<B}$ 、 $Y_{A=B}$



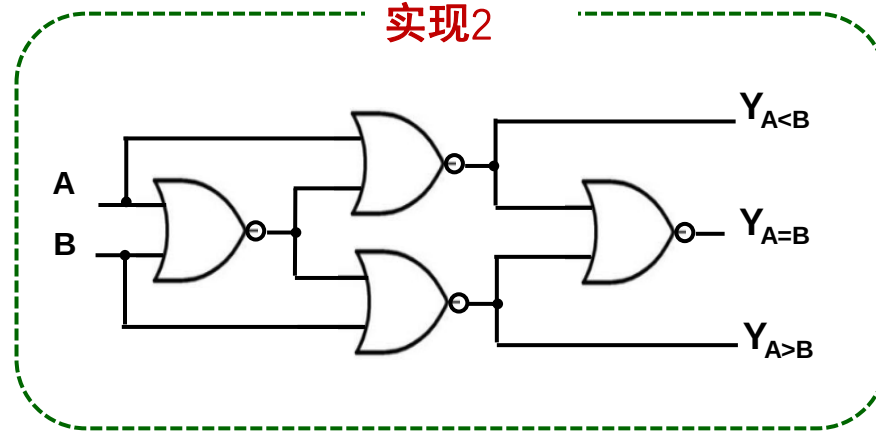
一位数值比较器

真值表

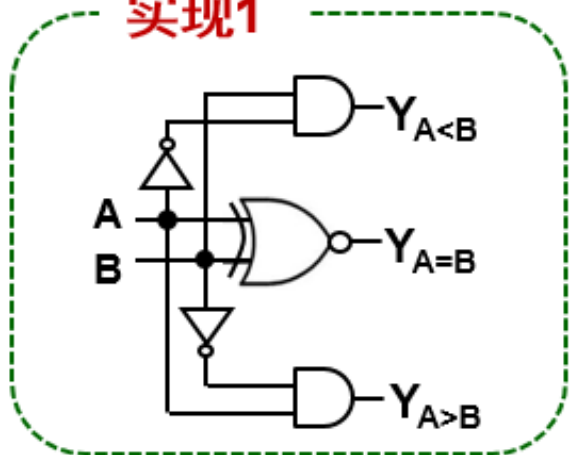
A	B	$Y_{A=B}$	$Y_{A>B}$	$Y_{A<B}$
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$$\begin{cases} Y_{A=B} = \overline{A \oplus B} \\ Y_{A>B} = A\overline{B} \\ Y_{A<B} = \overline{A}B \end{cases}$$

实现2



实现1



$$\begin{aligned} Y_{A=B} &= \overline{A\overline{B} + AB} = \overline{(A+\overline{B})(\overline{A}+B)} = \overline{(A+\overline{A}+\overline{B})(\overline{B}+A+B)} \\ &= \overline{(A+\overline{A}+\overline{B}) + (\overline{B}+A+B)} \end{aligned}$$

$$Y_{A>B} = A\overline{B} = \overline{\overline{A}B} = \overline{\overline{A} + B}$$

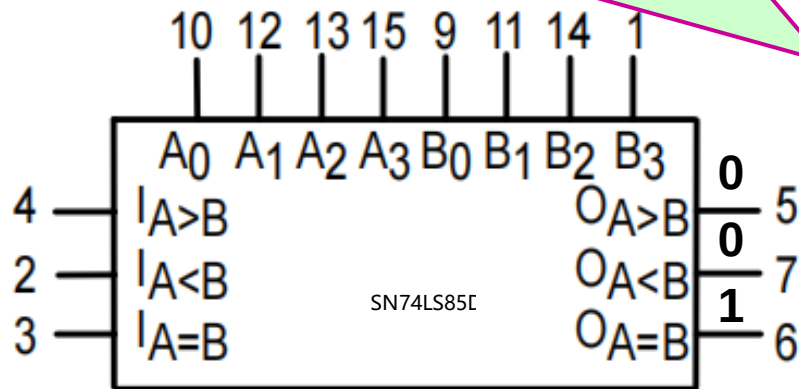
$$Y_{A<B} = \overline{A}B = \overline{A\overline{B}} = \overline{A + \overline{B}}$$

多位数值比较器

自高而低逐位比较，只有在高位相等时，才需要比较低位。

接低位芯片的比较结果，用于芯片扩展。

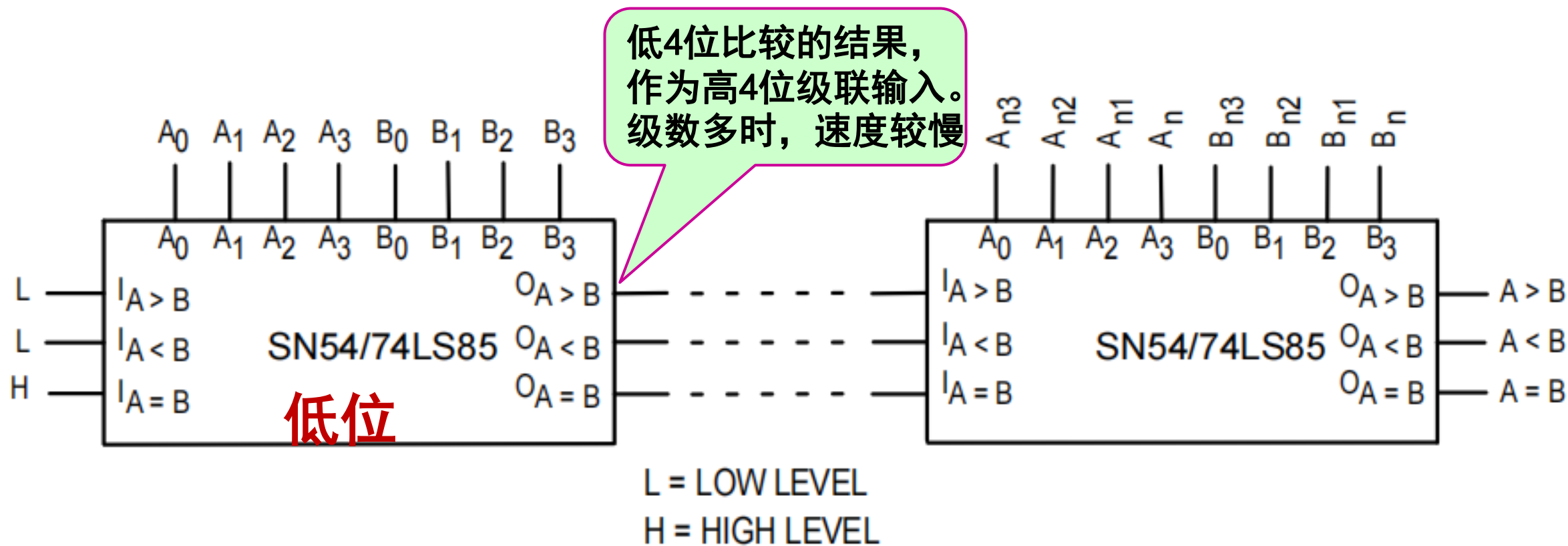
比较2个4位二进制数的大小时，3个输入端 $I_{A>B}$ 、 $I_{A<B}$ 、 $I_{A=B}$ 应接001；当 $A_3A_2A_1A_0 = B_3B_2B_1B_0$ 时，比较器的输出 $Y_{A>B}Y_{A<B}Y_{A=B} = 001$



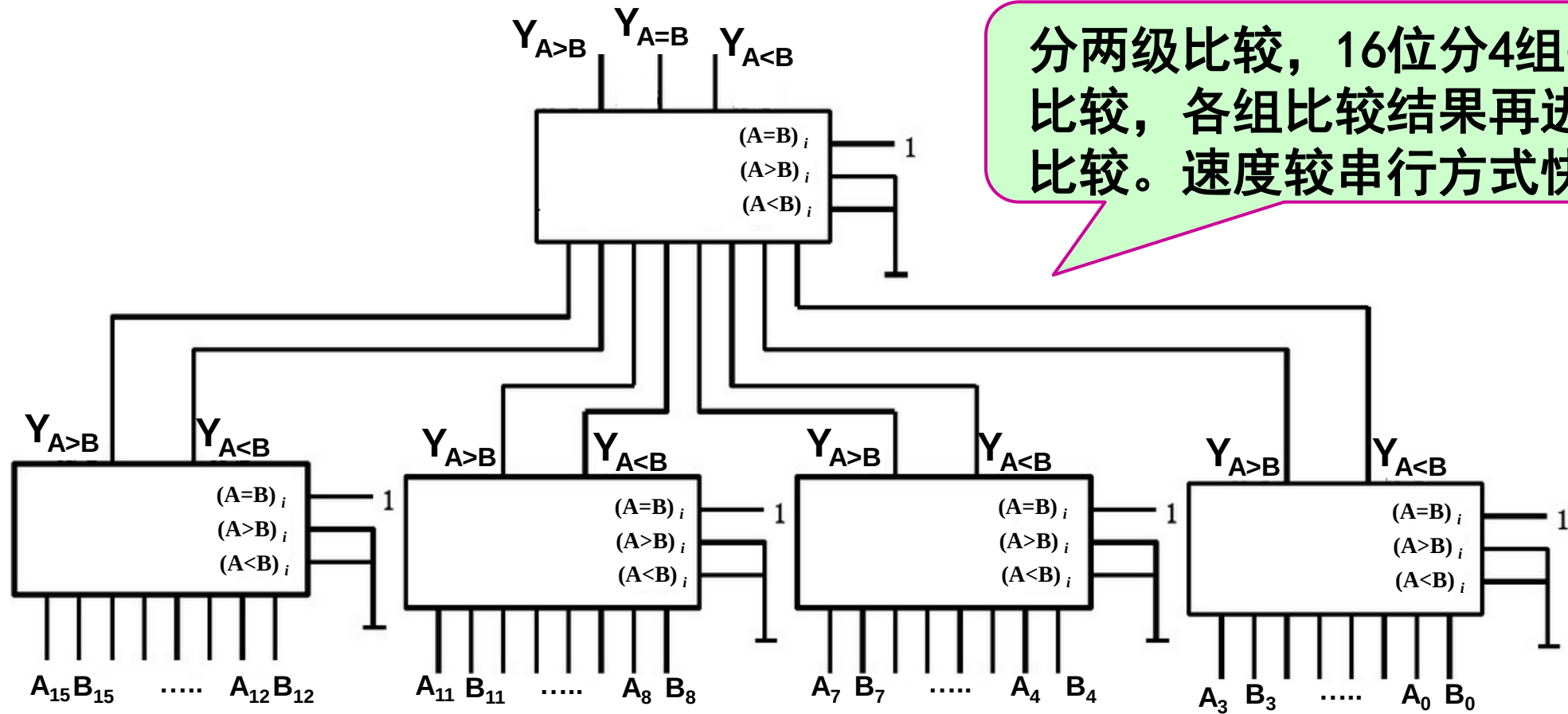
当 $A_3A_2A_1A_0 = B_3B_2B_1B_0$ 时，比较器的输出复现3个输入端 $I_{A>B}$ 、 $I_{A<B}$ 、 $I_{A=B}$ 的状态。

比较输入				级联输入I			输出O						
A ₃	B ₃	A ₂	B ₂	A ₁	B ₁	A ₀	B ₀	(A>B)	(A<B)	(A=B)	O _{A>B}	O _{A<B}	O _{A=B}
A ₃ > B ₃		X		X		X		X	X	X	1	0	0
A ₃ < B ₃		X		X		X		X	X	X	0	1	0
A ₃ = B ₃		A ₂ > B ₂		X		X		X	X	X	1	0	0
A ₃ = B ₃		A ₂ < B ₂		X		X		X	X	X	0	1	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ > B ₁		X		X	X	X	1	0	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ < B ₁		X		X	X	X	0	1	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ > B ₀		X	X	X	1	0	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ < B ₀		X	X	X	0	1	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ = B ₀		0	0	1	0	0	1
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ = B ₀		0	1	0	0	1	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ = B ₀		1	0	0	1	0	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ = B ₀		1	1	0	0	0	0
A ₃ = B ₃		A ₂ = B ₂		A ₁ = B ₁		A ₀ = B ₀		0	0	0	1	1	0

数值比较器的级联——①串行方式



数值比较器的级联—— ②并行方式



哪部分有疑问？

- ☐ A 异或门
- ☐ B 奇偶校验器
- ☐ C 一位比较器
- ☐ D 多位比较器
- ☐ E 无

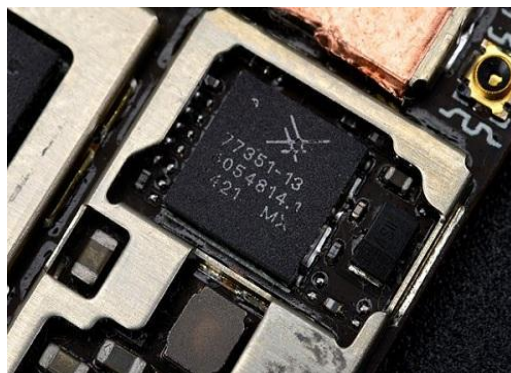
提交

组合逻辑元件

- 译码器(Decoders)
- 多路复用器(multiplexers)
- 三态器件(Three-state Buffer)
- 编码器(Encoders)
- 异或门和奇偶校验功能
- 比较器
- 逻辑器件

逻辑器件

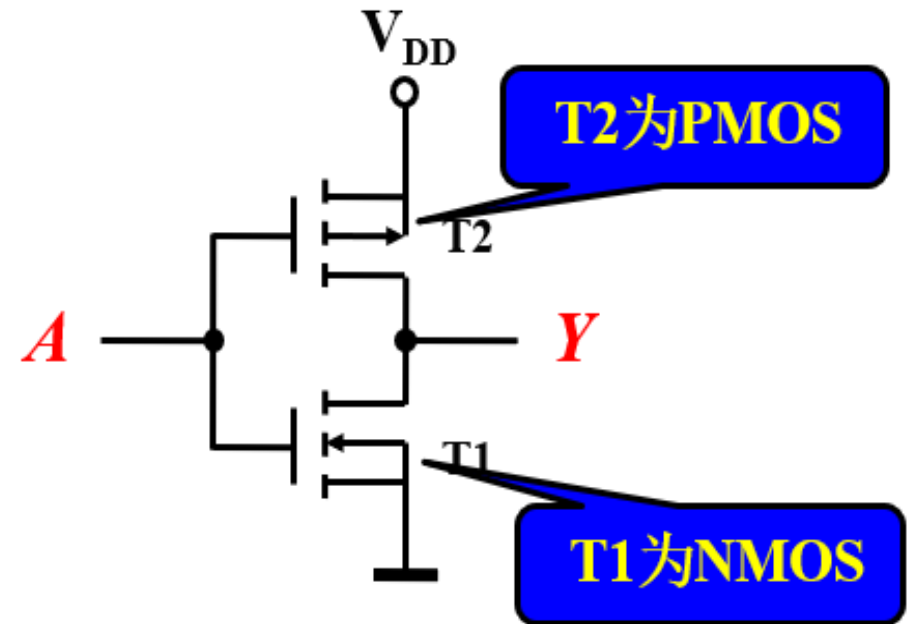
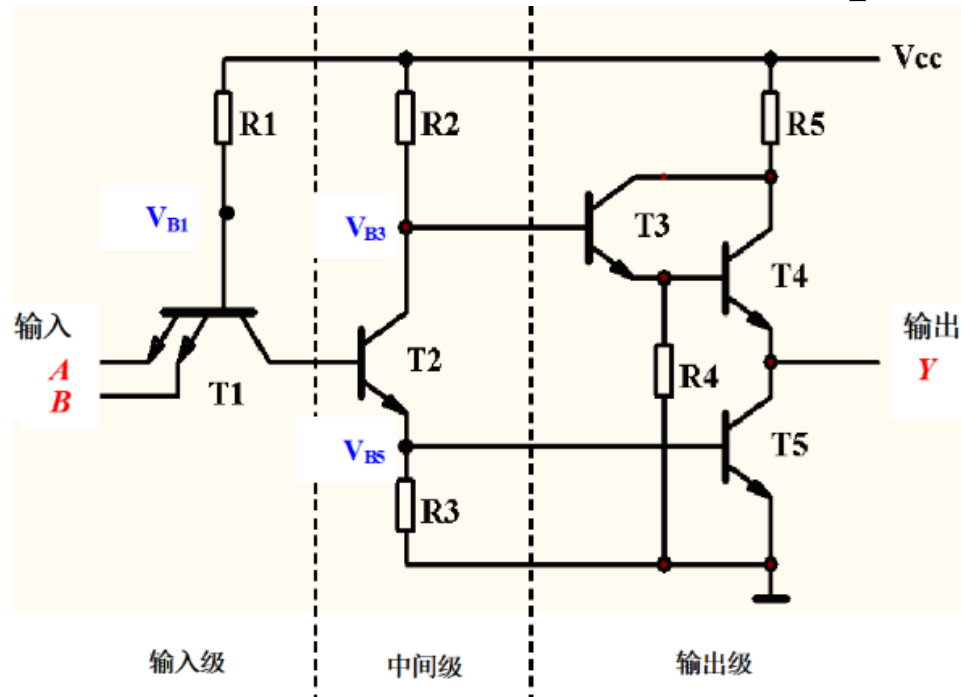
- 固定逻辑器件
 - 永久性，完成一种或一组功能——一旦制造完成，就无法改变。
- 可编程逻辑器件
 - 能够为客户提供范围广泛的多种逻辑能力、特性、速度和电压特性的标准成品部件，而且此类器件可在任何时间改变，从而完成许多种不同的功能。



固定逻辑器件

- 集成逻辑门电路

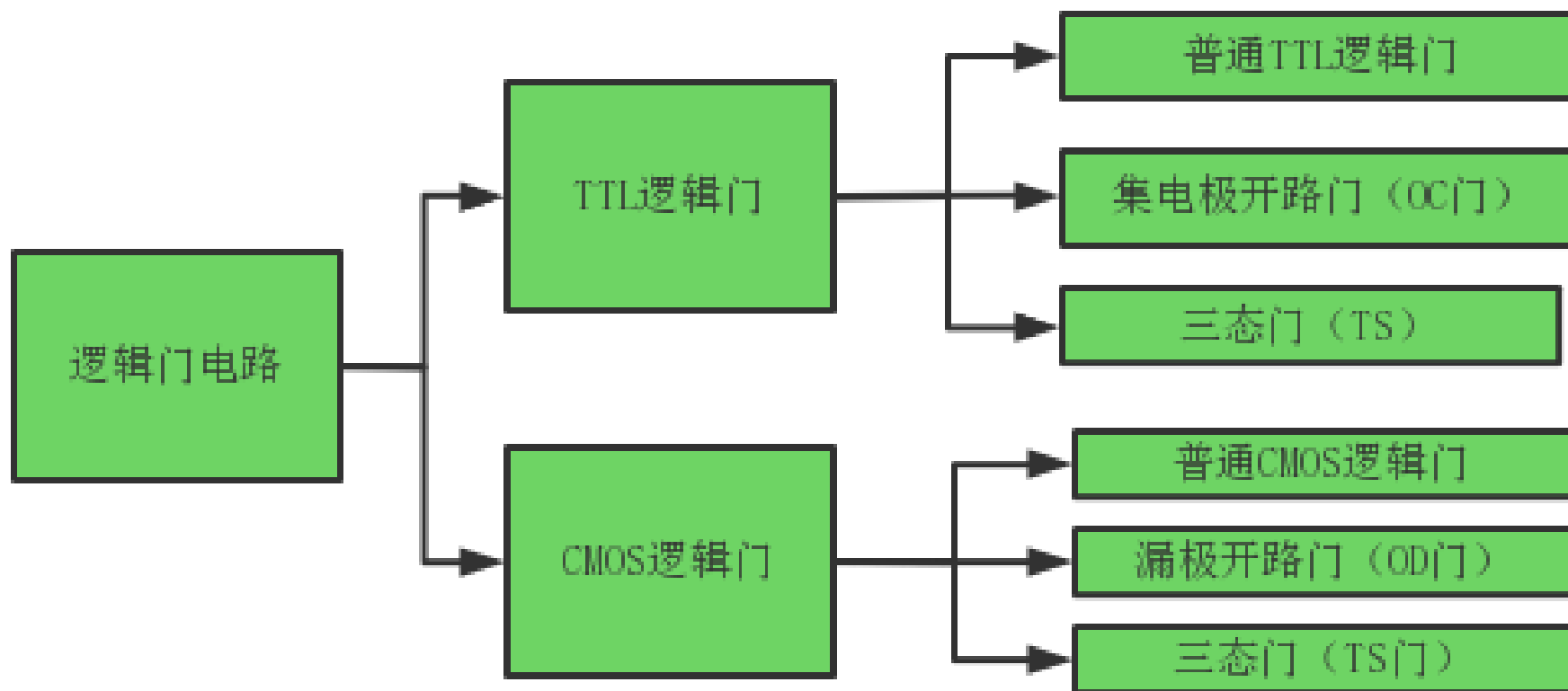
- TTL集成门电路（Transistor-Transistor Logic）
- CMOS集成门电路（Complementary Metal Oxide Semiconductor）



- ROM(Read-Only Memory)

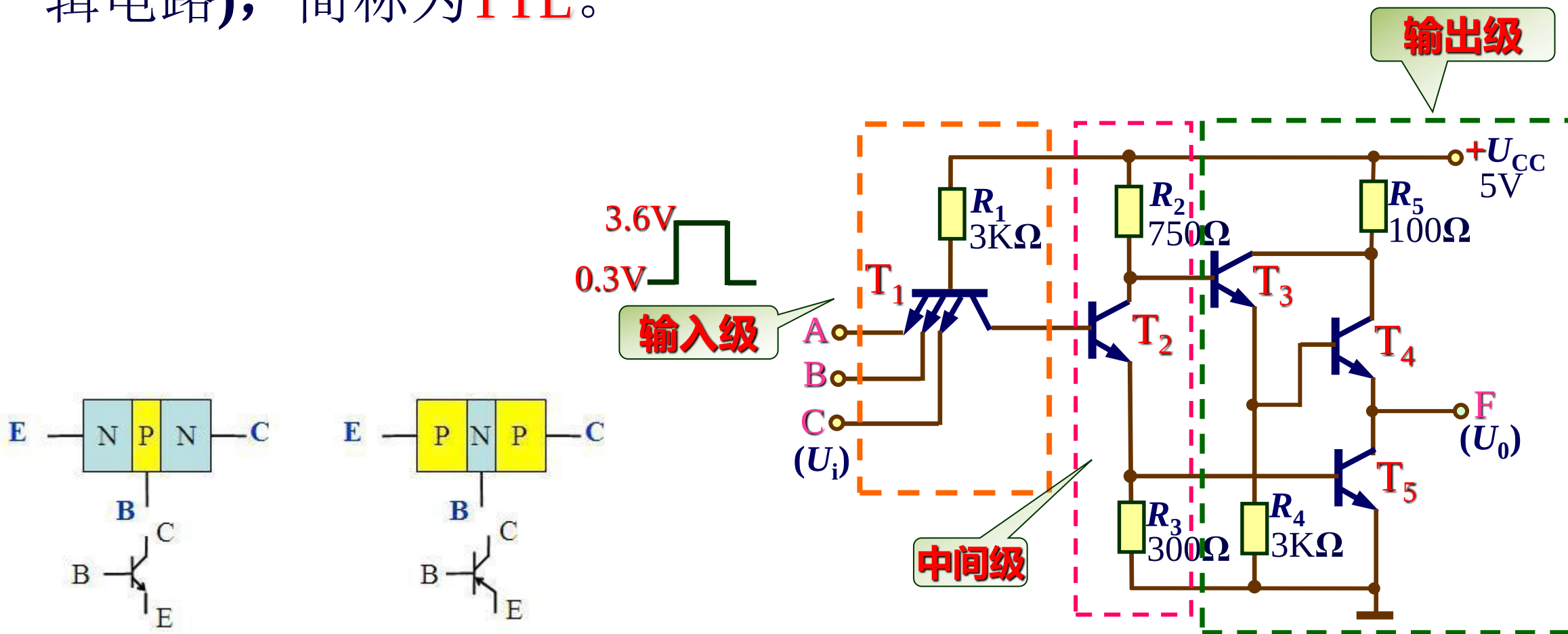
集成逻辑门电路

- TTL集成逻辑门：功耗大，不适于制造大规模集成电路。
- CMOS集成逻辑门：功耗非常低，发热量小，易于集成。



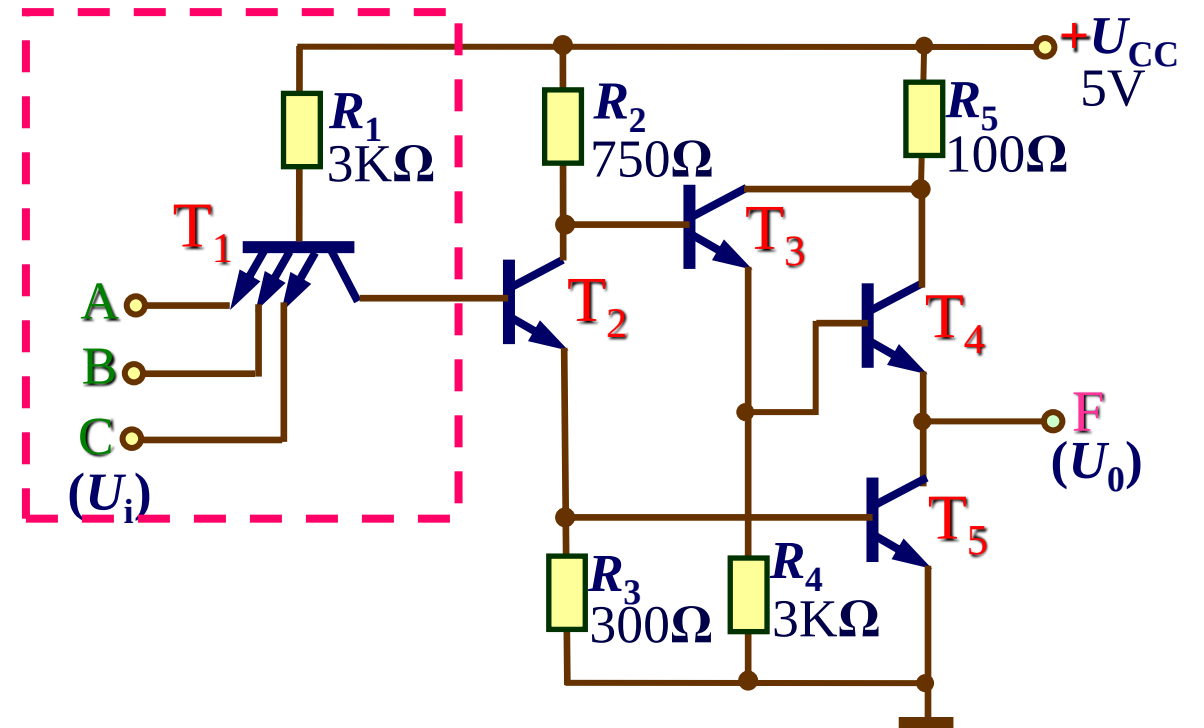
TTL集成电路 (Transistor-Transistor Logic)

- 输入端和输出端都采用了半导体晶体管 (晶体管-晶体管-逻辑电路), 简称为**TTL**。



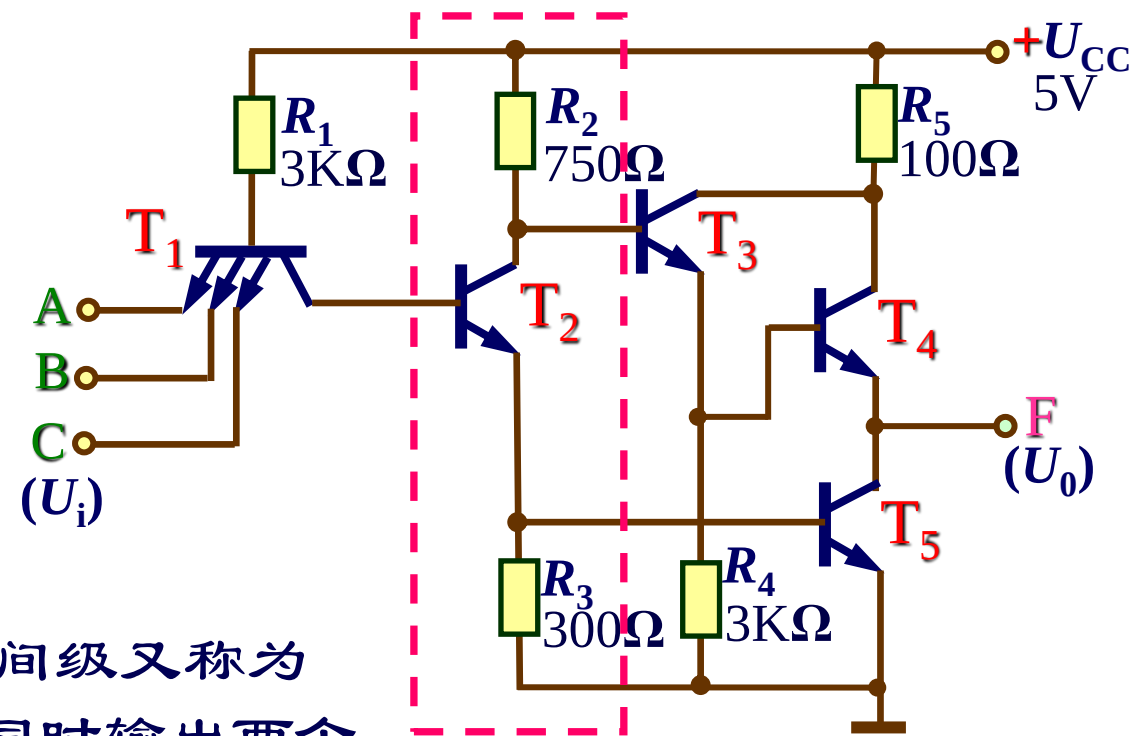
TTL与非门内部电路组成结构图

TTL集成电路——输入级



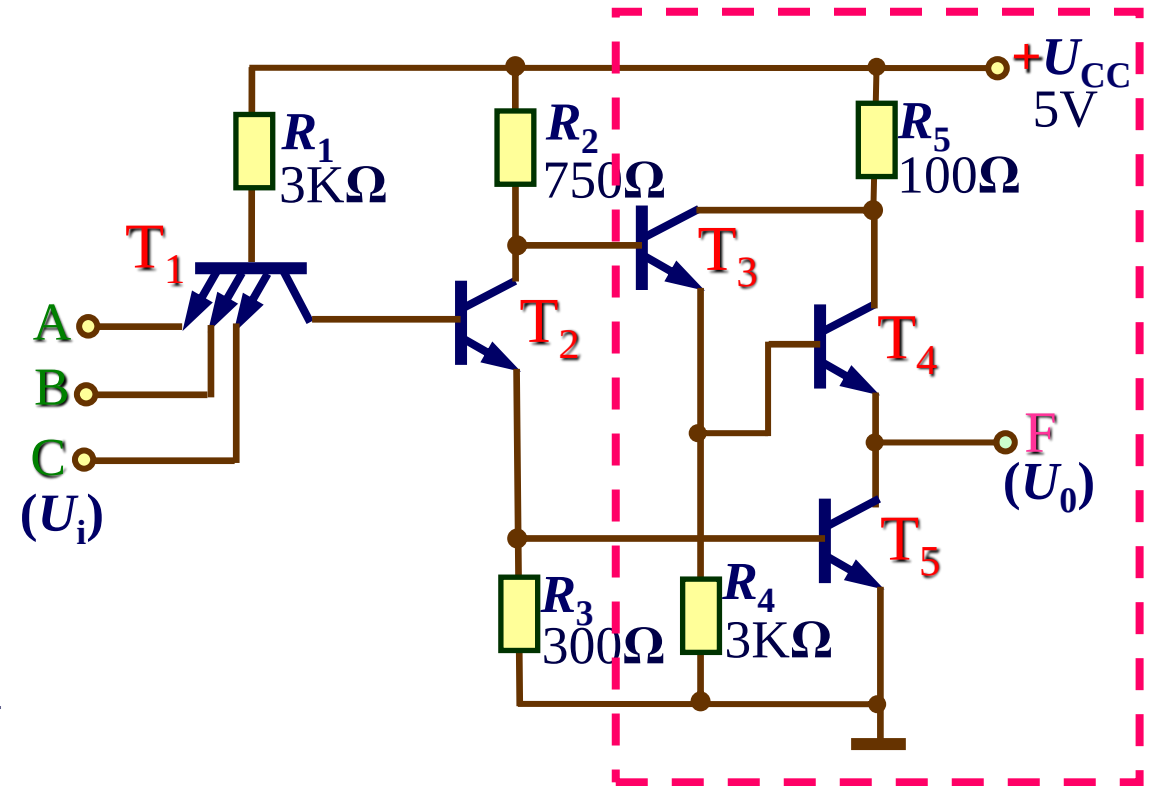
输入级由多发射极晶体管 T_1 和电阻 R_1 组成。所谓多发射极晶体管，可看作由多个晶体管的集电极和基极分别并接在一起，而发射极作为逻辑门的输入端。多个发射极的发射结可看作是多钳位二极管，其作用是限制输入端可能出现的负极性干扰脉冲。 T_1 的引入，不但加快了晶体管 T_2 储存电荷的消散，提高了TTL与非门的工作速度，而且实现“与”逻辑作用。

TTL集成电路——中间级



中间级由电阻 R_2 、 R_3 和三极管 T_2 组成。中间级又称为倒相极，其作用是从 T_2 的集电极和发射极同时输出两个相位相反的信号，作为输出级里的三极管 T_3 和 T_5 的驱动信号，同时控制输出级的 T_4 、 T_5 管工作在截然相反的两个状态，以满足输出级互补工作的要求。三极管 T_2 还可将前级电流放大以供给 T_5 足够的基极电流。

TTL集成电路——输出级

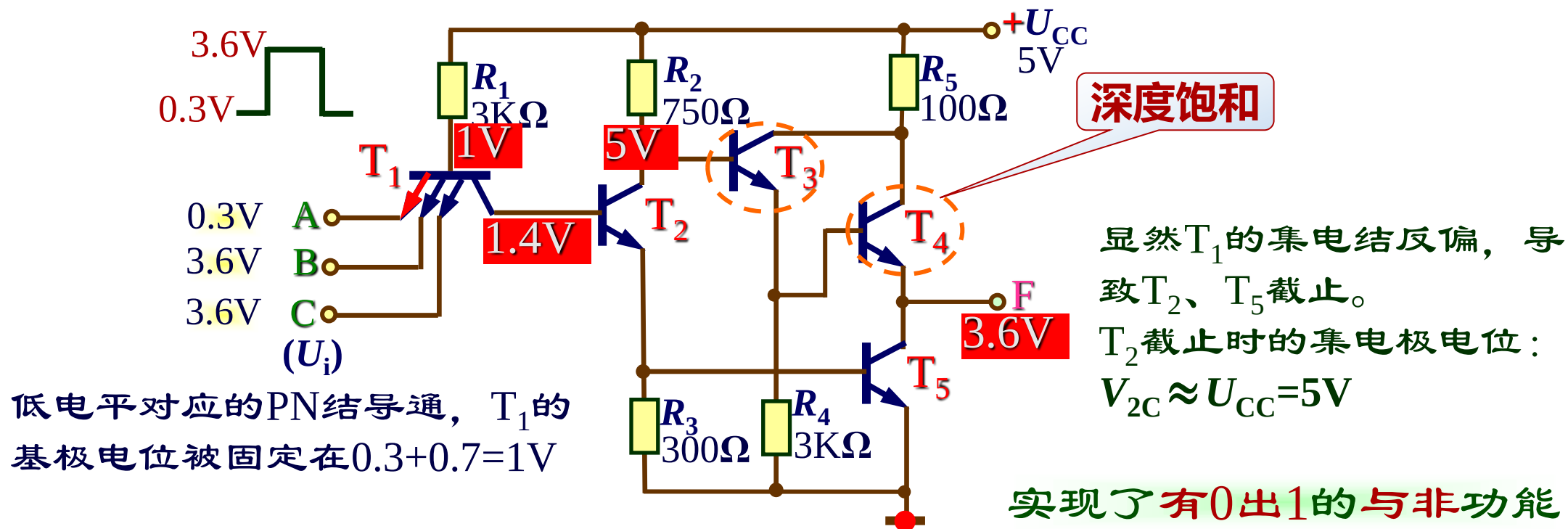


输出级是由晶体管 T_3 、 T_4 和 T_5 ，电阻 R_4 和 R_5 组成推拉式的互补输出电路。

T_5 导通时 T_4 截止， T_5 截止时 T_4 导通。由于采用了推挽输出(又称图腾输出)，该电路不仅增强了带负载能力，还提高了工作速度。

TTL与非门工作过程

- 输入端至少有一个为低电平时的工作情况

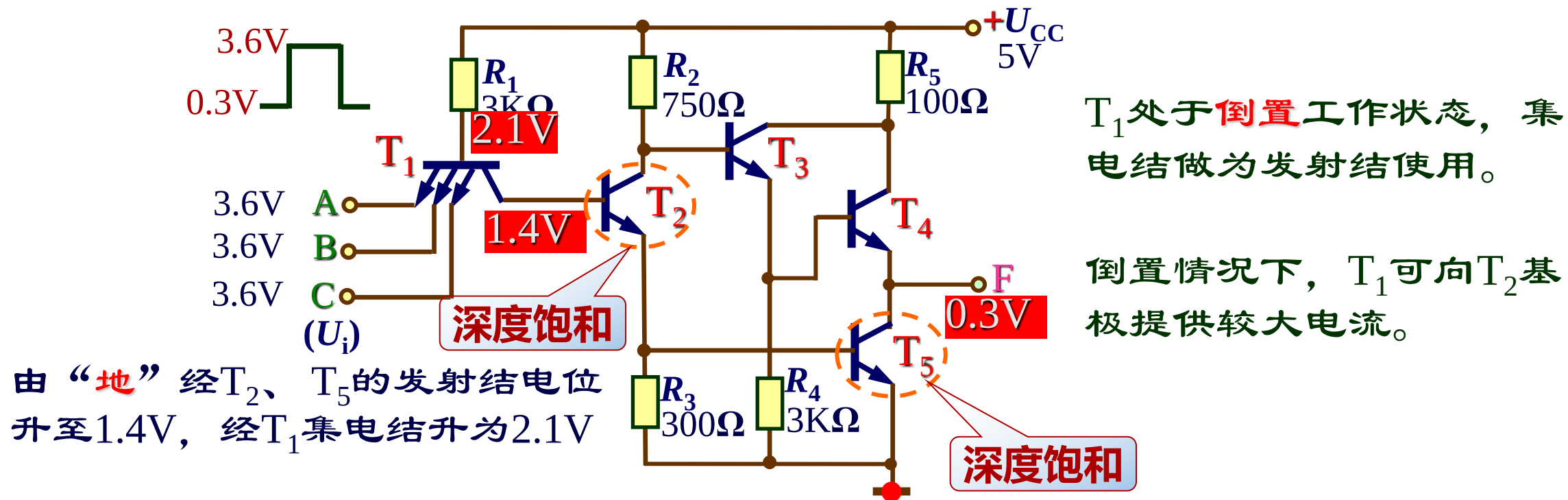


T_2 集电极+5V的电位足以使 T_3 、 T_4 导通并处于深度饱和状态。因 R_2 和 I_{B3} 都很小, 均可忽略不计, 所以与非门输出端F点的电位:

$$V_F = U_{CC} - I_{B3}R_2 - U_{BE3} - U_{BE4} \approx 5 - 0 - 0.7 - 0.7 \approx 3.6V$$

TTL与非门工作过程——续

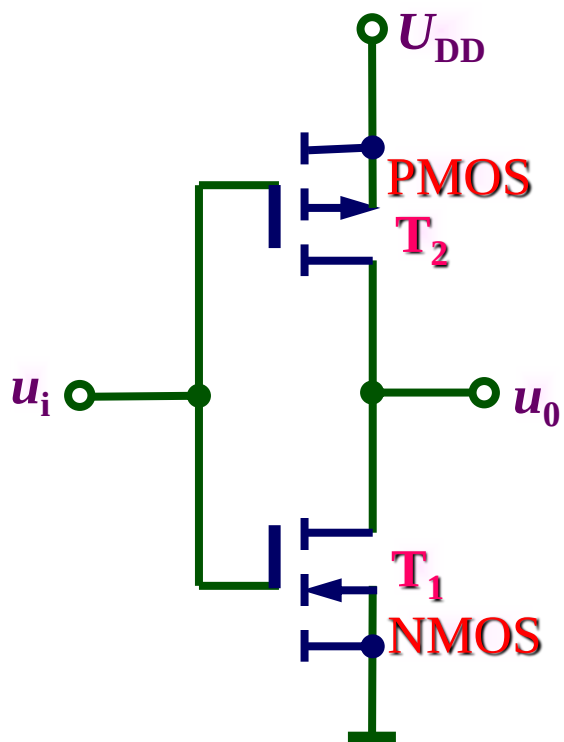
- 输入端全部为高电平时的工作情况：



T_2 深度饱和后，其发射极电流在电阻 R_3 上产生的压降又为 T_5 提供足够的基极电流使 T_5 饱和导通，从而使与非门输出F点的电位等于 T_5 的饱和输出典型值 **$F=0.3V$**

实现了全1出0的与非功能

CMOS集成电路——CMOS反相器



① $u_i = 0V$ 时， T_1 截止， T_2 导通。

输出电压 $u_0 = U_{DD}$ ，高电平；

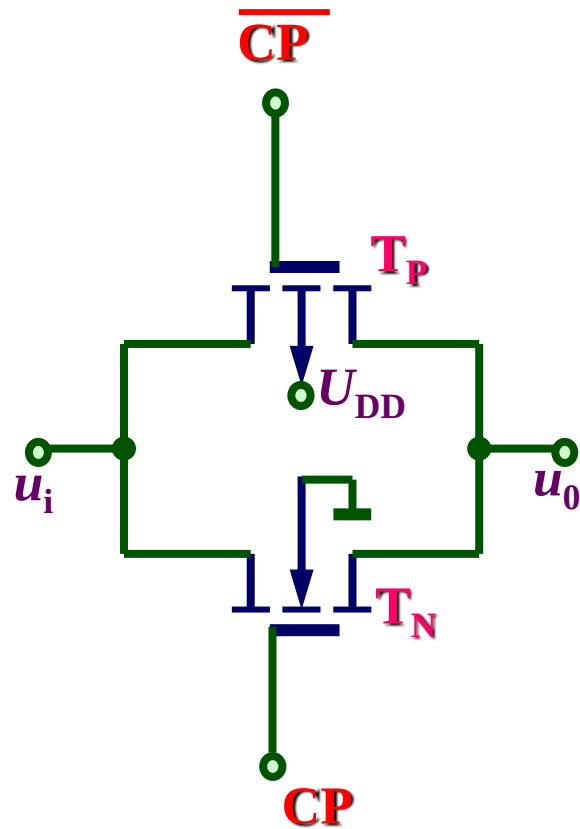
② $u_i = U_{DD}$ 时， T_1 导通， T_2 截止。

输出电压 $u_0 = 0V$ ，低电平。

实现了 **见0出1、见1出0** 的**非门**功能！

T_1 工作管为 **N** 沟道增强型 MOS 管， T_2 负载管为 **P** 沟道增强型 MOS 管，两管的漏极接在一起作为电路的输出端，两管的栅极接在一起作为电路的输入端， T_1 、 T_2 源极与其衬底相连，一个接地，一个接电源。

CMOS传输门



② 当 $CP = \text{“0”}$ 时，无论输入电压 u_i 在 $0 \sim 10V$ 之间如何变化，栅极和源极之间的电压都无法满足管子导通沟道产生的条件，因此两管都截止，输入信号不能传输到输出端，称传输门关断。此时相当于模拟开关断开。传输门在数字电路中起开关作用，所以也称作**模拟开关**。

CMOS集成电路的特点及使用注意事项

- 工作速度比TTL电路低；
- 带负载能力比TTL电路差；
- 集成度比TTL电路的集成度高；
- 抗干扰能力强；
- 功耗比TTL电路小得多。门电路的功耗只有几个 μW ，中规模集成电路的功耗也不会超过 $100\mu\text{W}$ 。
- 电源电压允许范围较大，约在3~18V；
- 适合于特殊环境下工作；
- 容易受静电感应而击穿，在使用和存放时应注意静电屏蔽，焊接时电烙铁应接地良好，尤其是CMOS电路多余不用，应根据需要接地或接高电平。

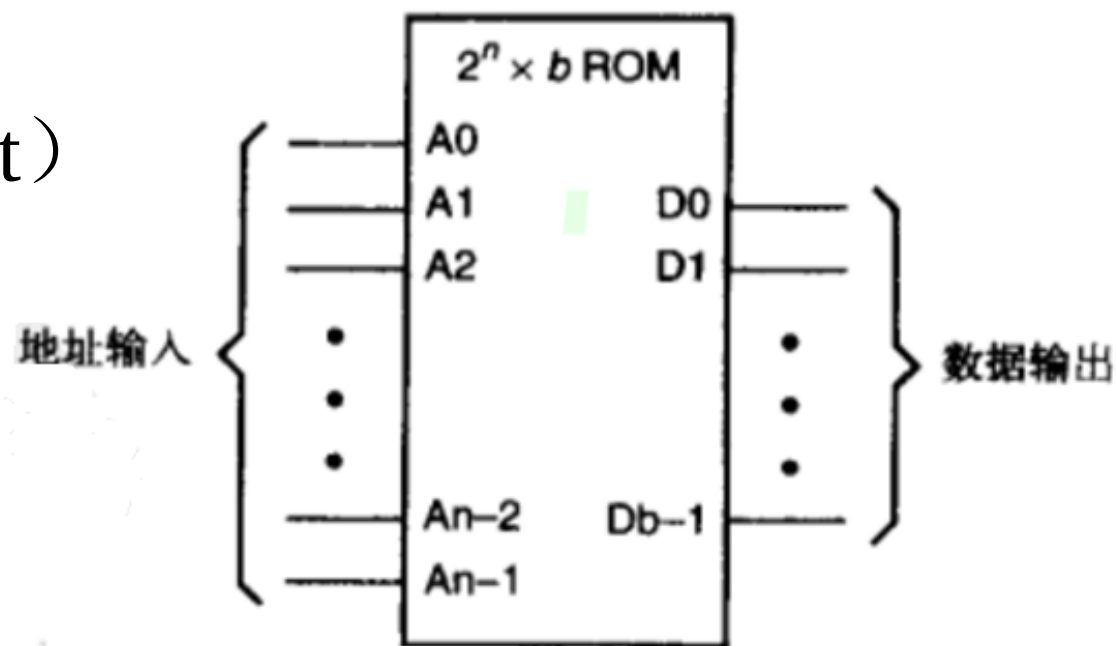
常用数字集成电路芯片-74系列

“XXX”表示芯片的类型，是一串数字(如00，08，20，138，245，373，573，4066等)，只要数字相同，其逻辑功能就相同，只是性能有差异。

TTL型	IC编号	V _{OH}	V _{IH}	V _{OL}	V _{IL}	传递延迟	功率消耗	
标准型	74XX	2.4V	2.0V	0.4V	0.8V	9ns	10mW	
低功耗型	74LXX	2.4V	2.0V	0.3V	0.8V	33ns	1 mW	
高速型	74HXX	2.4V	2.0V	0.4V	0.8V	6ns	23 mW	
萧特基型	74SXX	2.7V	2.0V	0.5V	0.8V	3ns	23 mW	
低功耗萧特基型	74LSXX	2.7V	2.0V	0.5V	0.8V	9.5 ns	2 mW	
高级萧特基型	74ASXX	3.0V	2.0V	0.5V	0.8V	1.5 ns	8 mW	
高级低功耗萧特基型	74ALSXX	3.0V	2.0V	0.4V	0.8V	4 ns	1.2 mW	
CMOS型	IC编号	V _{OH}	V _{IH}	V _{OL}	V _{IL}	传递延迟	功率消耗	
早期CMOS	CD40XX CD45XX						0.01 mW	
第一个以CMOS设计且与TTL引脚兼容的版本	74Cxx		3.5V	0.1V	1.5V	7 ns	0.01 mW	与TTL引脚相容
以高速CMOS做的IC	74HCxx	4.9V	3.5V	0.1V	1.5V	7 ns	0.01 mW	与TTL引脚相容
高速CMOS	74HCTxx	4.9V	2.0V	0.1V	0.8V	8 ns	0.01 mW	与TTL电性能兼容
高速CMOS,设计用来与74ALS竞争	74ACxx	4.9V	3.5V	0.1V	1.5V	6 ns	0.01 mW	与TTL电性能不兼容
高性能CMOS	74ACTxx	4.9V	3.5V	0.1V	1.5V	6 ns	0.01 mW	与TTL不兼容，但电性能相同

ROM (Read-Only Memory)

- ROM是一种具有 n 个输入- b 个输出的组合逻辑电路。
- 输入被称为**地址输入** (address input)
 - 通常命名为 $A_0, A_1, \dots, A_{n-1}$
- 输出被称为**数据输出** (data output)
 - 通常命名为 $D_0, D_1, \dots, D_{b-1}$
- 输入不同的 n 位地址 A , 在 D 端输出不同的 b 位数据。



$2^n * b$ 位ROM

用ROM实现2-4译码器——1

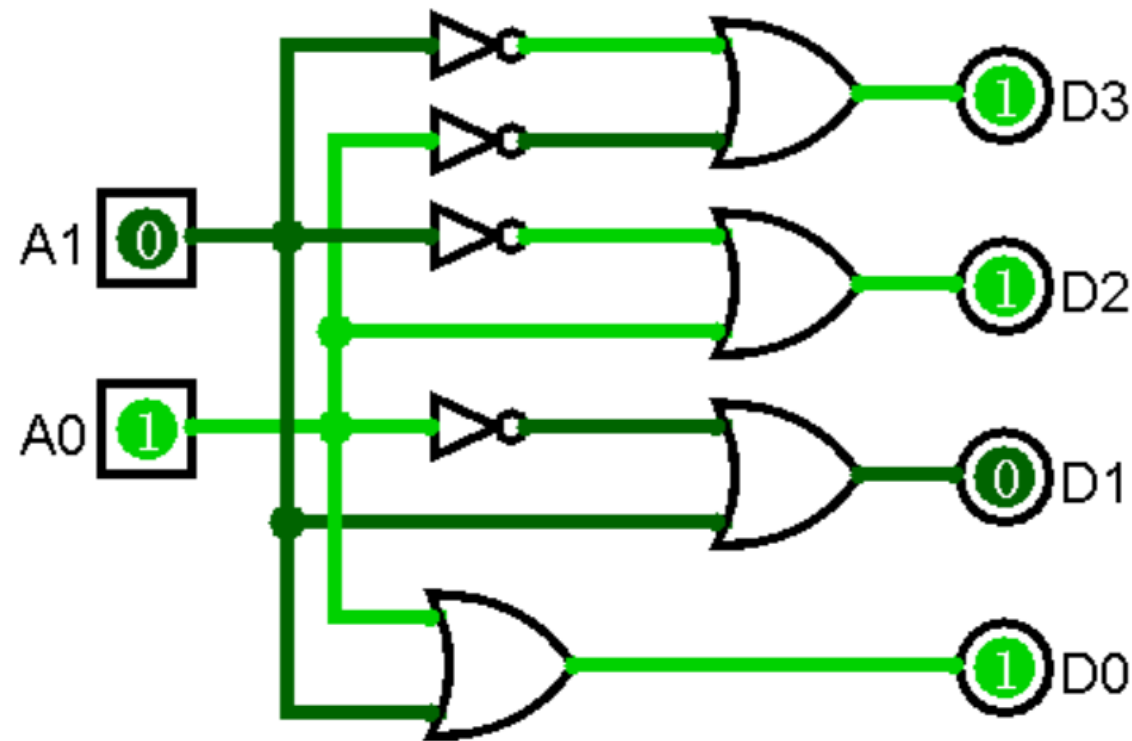
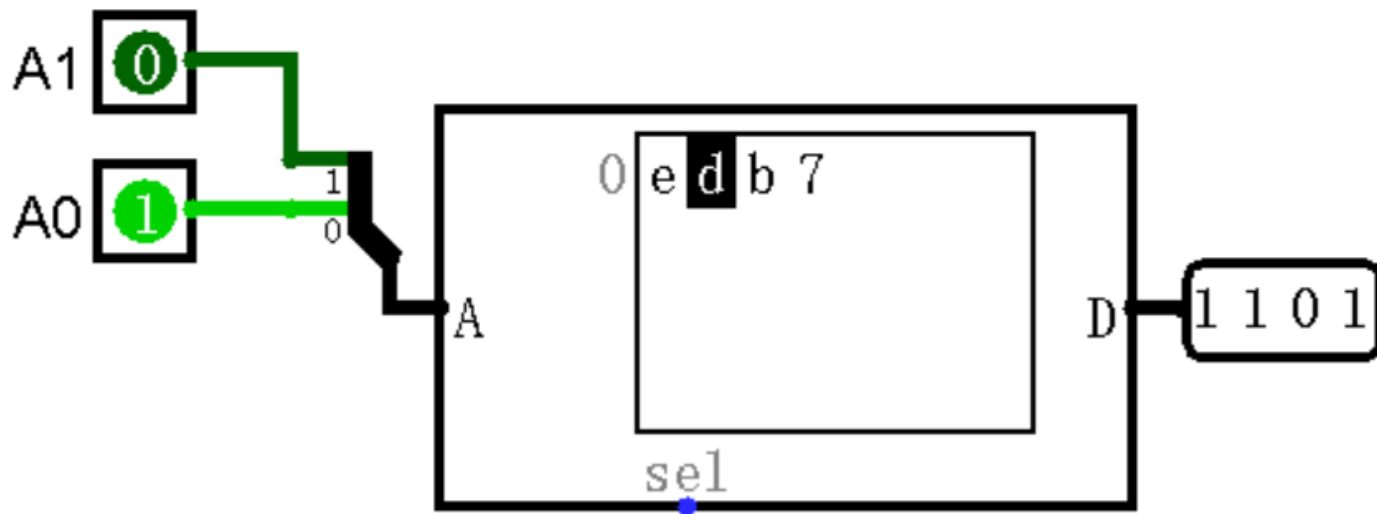
- ROM可“存储” n输入、b输出的组合逻辑功能的真值表。
- 2输入、4输出的组合功能的真值表，可以被存储在一个 $2^2 * 4$ 的ROM中。
- 忽略延迟，ROM的数据输出等于真值表中由地址输入所选择的那行输出。
(ROM 的每一行存储一组输出，即真值表的一行)

输入		输出			
A1	A0	D3	D2	D1	D0
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

2-4译码器真值表

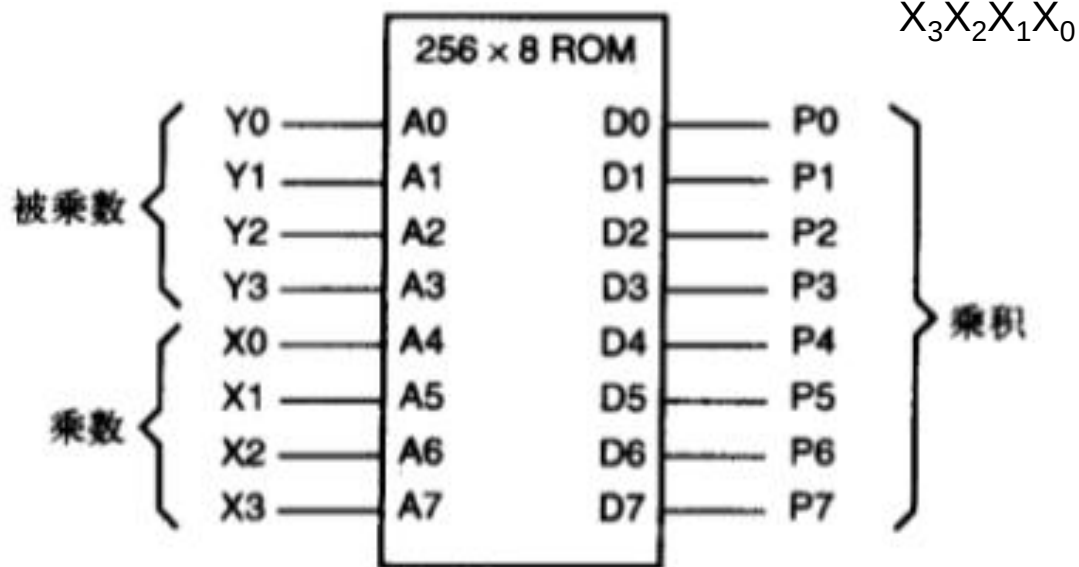
用ROM实现2-4译码器——2

- 两种不同的方式来构建译码器：
 - 使用分立的门
 - 用包含真值表的4 * 4 ROM
- 实现并不是唯一的。



用ROM实现4位*4位无符号二进制数乘法

- 多少种组合？
- 乘积最多为几位？



地址	$Y_3Y_2Y_1Y_0$															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
10	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
20	00	02	04	06	08	0A	0C	0E	10	12	14	16	18	1A	1C	1E
30	00	03	06	09	0C	0F	12	15	18	1B	1E	21	24	27	2A	2D
40	00	04	08	0C	10	14	18	1C	20	24	28	2C	30	34	38	3C
50	00	05	0A	0F	14	19	1E	23	28	2D	32	37	3C	41	46	4B
60	00	06	0C	12	18	1E	24	2A	30	36	3C	42	48	4E	54	5A
70	00	07	0E	15	1C	23	2A	31	38	3F	46	4D	54	5B	62	69
80	00	08	10	18	20	28	30	38	40	48	50	58	60	68	70	78
90	00	09	12	1B	24	2D	36	3F	48	51	5A	63	6C	75	7E	87
A0	00	0A	14	1E	28	32	3C	46	50	5A	64	6E	78	82	8C	96
B0	00	0B	16	21	2C	37	42	4D	58	63	6E	79	84	8F	9A	A5
C0	00	0C	18	24	30	3C	48	54	60	6C	78	84	90	9C	A8	B4
D0	00	0D	1A	27	34	41	4E	5B	68	75	82	8F	9C	A9	B6	C3
E0	00	0E	1C	2A	38	46	54	62	70	7E	8C	9A	A8	B6	C4	D2
F0	00	0F	1E	2D	3C	4B	5A	69	78	87	96	A5	B4	C3	D2	E1

基于ROM的设计方法的优点

- 通常可以用高级程序语言来计算存储在ROM中的内容。

```
#include <stdio.h>

/*Procedure to print d as a hex digit. */
void PrintHexDigit(int d)
{
    if (d<10) printf("%c", '0'+d);
    else printf("%c", 'A'+d-10);
}

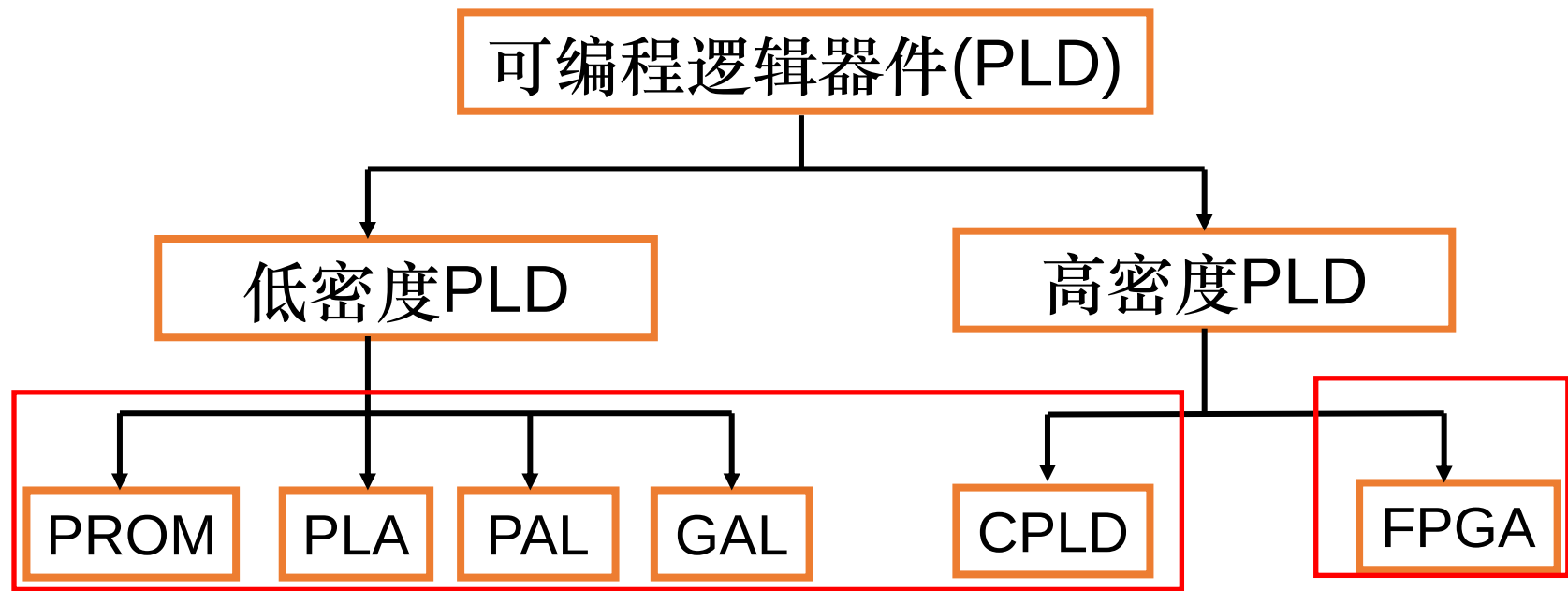
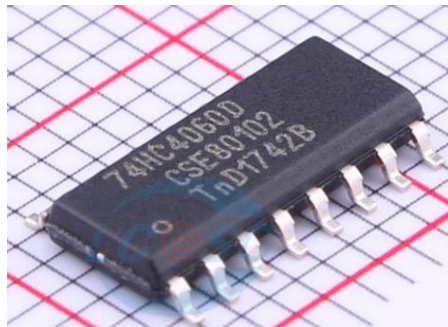
/*Procedure to print i as two hex digits. */
void PrintHex2(int i)
{
    PrintHexDigit((i / 16) % 16);
    PrintHexDigit(i % 16);
}

void main()
{
    int x, y;

    for (x=0; x<=15; x++) {
        PrintHex2(x*16); printf(":");
        for (y=0; y<=15; y++) {
            printf(" ");
            PrintHex2(x*y);
        }
        printf("\n");
    }
}
```

可编程逻辑器件(Programmable Logic Device)

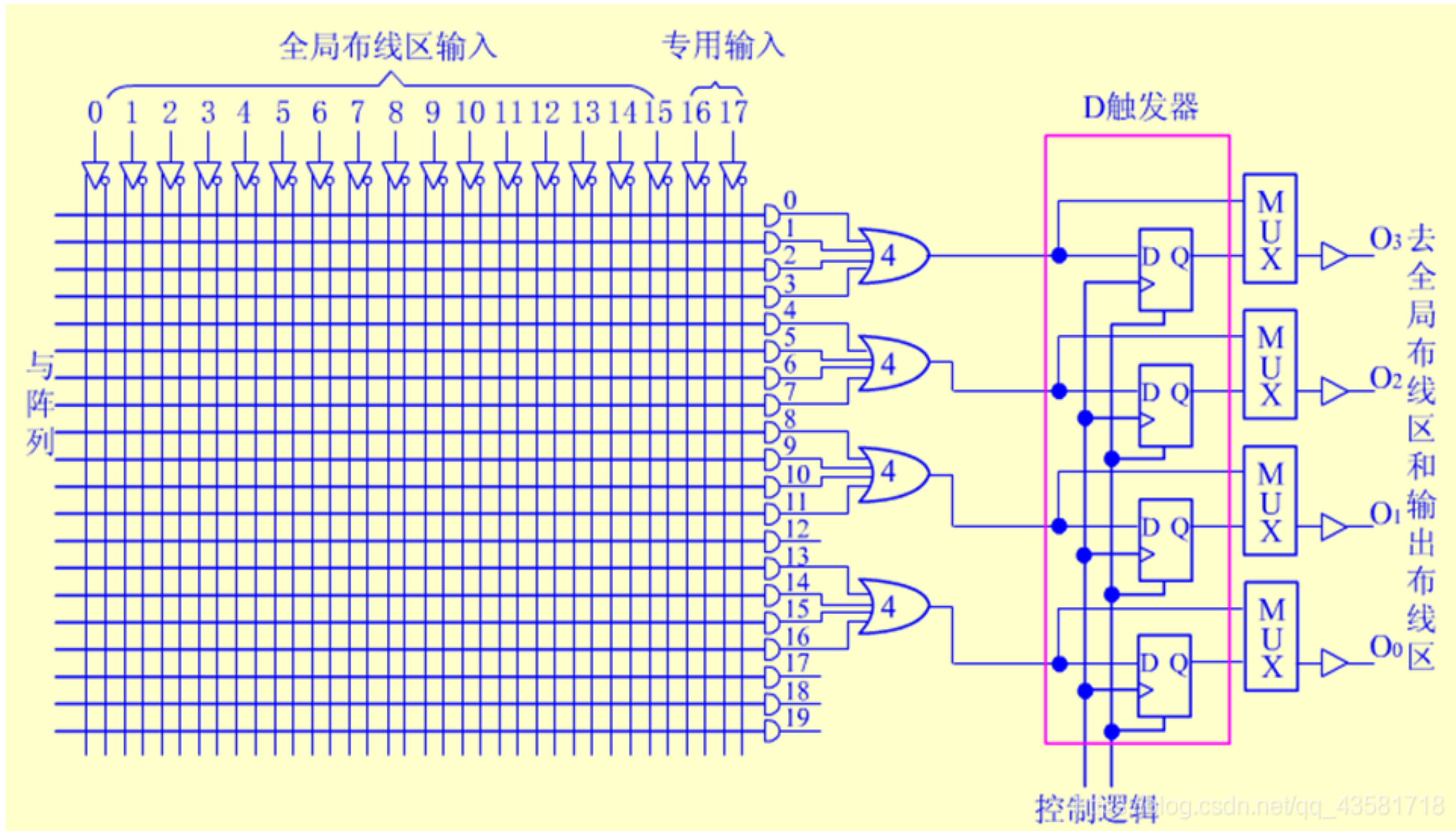
- 固定逻辑器件
- 可编程逻辑器件



乘积项结构

查找表结构

可编程逻辑器件——乘积项结构



FPGA (Field Programmable Gate Array)

- FPGA: 现场可编程门阵列
- FPGA能完成任何数字逻辑功能，上至高性能计算，下至简单的74系列电路，也常用于ASIC流片前的原型验证。



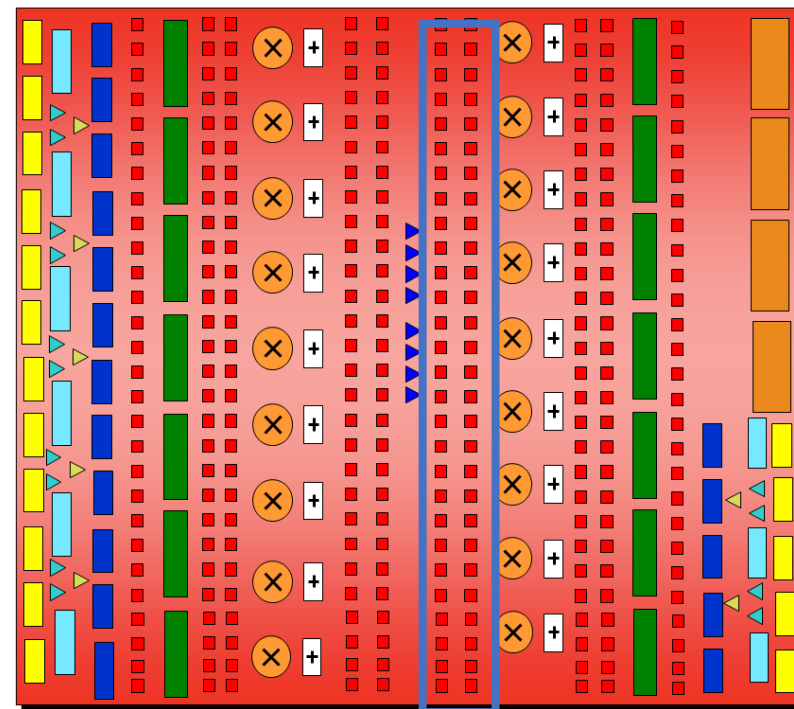
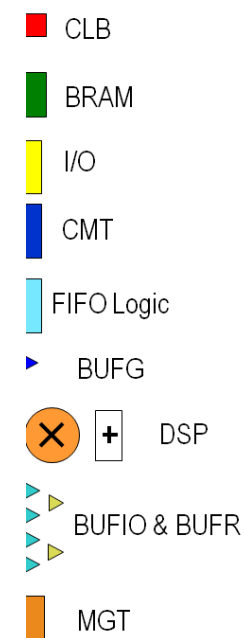
嵌入式硬件算法加速



协处理器

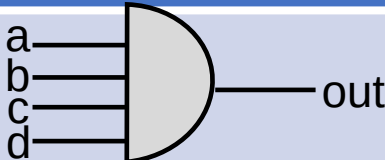
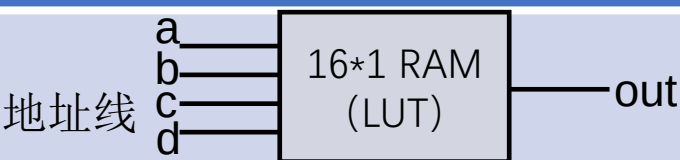
FPGA内部结构

- 内部资源分类:
- 逻辑资源: CLB、块存储(block RAM)、DSP等;
- 连接资源: 可编程互联线 (PI)、输入输出块 (IOB) 等;
- 其他资源: 全局时钟网络、时钟管理模块等
- 高端FPGA还集成ARM核、PCIE核等
- 资源分布采用ASMBL架构, 相同资源以列方式排布。



FPGA中的查找表（LookUp Table）

- LUT本质就是RAM(随机访问存储器)，主流FPGA是5或6输入LUT。
- LUT 把逻辑函数的“输入”当作地址，把“输出”当作该地址下 RAM 中存储的内容。例：使用LUT实现一个4与门电路逻辑功能

实际逻辑电路		LUT的实现方式	
			
a、b、c、d	逻辑输出	地址	RAM中存储的内容
0000	0	0000	0
0001	0	0001	0
.....	0		0
1111	1	1111	1

- a,b,c,d由FPGA管脚输入后进入可编程连线，然后地址线连到LUT，LUT中已经事先写入了真值表，通过地址查找到相应的数据然后输出

哪部分有疑问？

- ☐ A ROM实现组合逻辑函数
- ☐ B 可编程逻辑器件
- ☐ C FPGA查找表
- ☐ D 无

提交